

Dynamic Quality-of-Service for Mixed Workloads in a Data-Sovereign Private Cloud

Bachelor Thesis

ZHAW School of Engineering
Institute of Computer Science

Submitted by:

Michael Voemel

Supervised by:

Dr. Leonardo Militano

5. June 2026

Abstract

Co-locating web applications and AI training jobs on shared private hardware creates a resource-contention problem: static CPU partitioning either wastes idle capacity or leaves latency-sensitive services unprotected.

The migration of enterprise workloads to private, data-sovereign infrastructure creates a resource efficiency challenge: latency-sensitive web applications and throughput-intensive Artificial Intelligence (AI) training jobs must share the same bare-metal nodes, yet static CPU limits leave idle capacity unused while unrestricted co-location degrades service quality. This thesis designs, implements, and empirically evaluates a feedback-driven Proportional-Integral (PI) Quality-of-Service controller for a three-node bare-metal Kubernetes cluster. The controller continuously observes the 95th-percentile (P95) HTTP response latency of a co-located microservice application via Prometheus and dynamically resizes the CPU limits of Ray distributed AI training workers (Ray is an open-source distributed computing framework) through the Kubernetes in-place pod vertical scaling API, without restarting any container.

A five-condition repeated-measures experiment (three runs each) quantifies the noisy-neighbor effect and evaluates the controller against static limits. Unrestricted co-location raises mean P99 latency 3.3-fold and reduces Service Level Objective (SLO) compliance from 89.9 % to 12.0 %. The PI controller matches and marginally improves on static limits in SLO compliance while simultaneously reducing AI training time by 29 %, recovering CPU capacity during low-load phases that a fixed cap would leave stranded.

The dual objective of SLO protection and AI throughput maximization is thus achieved with a single feedback loop requiring no workload-specific knowledge beyond the observed latency signal. The principal limitation is that SLO compliance is bounded by application-internal CPU demand at load-phase transitions, which no external CPU scheduling policy can fully prevent. The entire control loop executes within the private cluster, satisfying the data-sovereignty requirement imposed by the Swiss Federal Act on Data Protection (nFADP).

Keywords: dynamic quality-of-service, PI controller, Kubernetes, noisy-neighbor effect, mixed workloads, data sovereignty, CPU throttling

Preface

This bachelor thesis originated from a practical requirement to build a scalable web application. Initially, my plan was to deploy this application within a standard Proxmox virtualized environment (an open-source hypervisor platform). However, my supervisor, Dr. Leonardo Militano, challenged me to elevate the project's academic rigor by integrating computationally intensive AI workloads alongside the web application.

As I researched modern data center architectures, I realized that statically isolating these contrasting workloads on separate physical nodes was highly inefficient. This prompted the central shift in my thesis; the academic rationale for this pivot is detailed in Section 1.3.

This pivot required overcoming a steep technological learning curve, including mastering bare-metal Kubernetes provisioning, Infrastructure as Code (IaC) with Ansible, and the mathematics of control theory. Moving from being a developer familiar only with hypervisors to engineering a custom Kubernetes controller that dynamically adjusts container CPU limits via the Kubernetes in-place resize API has been an immensely challenging and rewarding experience.

I would like to express my sincere gratitude to my supervisor, Dr. Leonardo Militano, for his continuous guidance, constructive feedback, and for pushing me out of my technical comfort zone. His insistence on innovation ultimately shaped this project into an architecture of which I am particularly proud.

Michael Voemel

Winterthur, 5. June 2026

Contents

1	Introduction	6
1.1	Background	6
1.2	Objectives	7
1.3	Deviation from the Original Task Description	8
1.4	Overview of the Thesis	8
2	Theoretical Foundations	10
2.1	Data Sovereignty and Cloud Paradigms	10
2.2	Abstraction Layers: Virtualization vs. Containerization	11
2.3	Microservices Architecture	12
2.4	Orchestrating Mixed Workloads and the "Noisy Neighbor" Problem	13
2.5	GPU Acceleration and Multiplexing	14
2.6	Distributed Computing Frameworks and Network Constraints	15
2.7	Distributed Deep Learning and Data Parallelism	16
2.7.1	Parallelism Strategies	16
2.7.2	DistributedDataParallel	16
2.7.3	AllReduce and the Barrier Problem	17
2.7.4	CPU Preprocessing as the Throughput Bottleneck	18
2.8	Quality of Service (QoS) in Containerized Environments	19
2.9	Dynamic Resource Management and Control Theory	19
2.9.1	The Control Loop Pattern	19
2.9.2	Proportional-Integral (PI) Control in Computing	20
3	Approach / Methods	22
3.1	Architecture Design and Hardware Topology	22
3.2	Infrastructure-as-Code Provisioning and Cluster Bootstrapping	23
3.2.1	Network Configuration	24
3.2.2	OS and Container Runtime Prerequisites	24
3.2.3	Cluster Bootstrap	25
3.2.4	Observability Stack and Operators	25
3.3	Workload A: Ticket Application	26
3.3.1	Service Decomposition and Responsibilities	28

3.3.2	Event Bus and Asynchronous Communication	28
3.3.3	SLA/SLO Definition and Measurement	30
3.3.4	Load Generation: WaveShape Profile	31
3.4	Workload B: Distributed AI Training Job	32
3.4.1	CPU Resource Policy and In-Place Resize	33
3.4.2	Script Delivery and Reproducibility	35
3.5	Dynamic QoS Controller	36
3.5.1	Control Loop Architecture	36
3.5.2	PI Control Law and Parameter Selection	37
3.5.3	Kubernetes Resize Subresource Actuation	39
3.5.4	Prometheus Metrics Exported	40
3.6	Experimental Design	40
3.6.1	Metrics Collected and Export Methodology	41
3.6.2	Statistical Approach	43
4	Results	44
4.1	Isolation Baselines	44
4.1.1	Workload A Alone (no-contention-wla)	44
4.1.2	Workload B Alone (no-contention-wlb)	44
4.2	Co-location without CPU Limits (no-limits)	46
4.3	Co-location with Static CPU Limits (static-limits)	47
4.4	Co-location with Dynamic QoS Controller (qos-controller)	49
4.5	Cross-Condition Comparison	51
5	Discussion and Outlook	56
5.1	Evaluation Against Research Question and Hypotheses	56
5.2	The Noisy Neighbor Effect in Practice	57
5.3	Limitations of the PI Controller	59
5.4	The Stranded Resource Problem	60
5.5	Reaction Time and Control Loop Latency	61
5.6	Comparison with Hyperscaler Approaches	62
5.7	Outlook and Future Work	63
A	Project Management	76
A.1	Task Description	76
A.2	Repository Structure	78
A.3	Challenges	80
A.4	Schedule	81
A.5	Risk analysis	81

B Infrastructure Setup	84
B.1 01-network-setup.yml	84
B.2 02-system-baseline.yml	85
B.3 03-containerd.yml	86
B.4 04-kubernetes-components.yml & 04b-kubelet-bind.yml	87
B.5 05-cluster-bootstrap.yml	87
B.6 06-telemetry-stack.yml	88
B.7 07-gpu-operator.yml	89
B.8 07b-ingress-nginx.yml	90
B.9 07c-kuberay-operator.yml	91
C Workload A Reference	93
C.1 HTTP API Reference	93
C.1.1 Authentication Service (auth-srv)	93
C.1.2 Tickets Service (tickets-srv)	93
C.1.3 Orders Service (orders-srv)	95
C.1.4 Payments Service (payments-srv)	97
C.1.5 Expiration Service (expiration-srv)	97
C.2 Kafka Event Reference	98
C.2.1 Event Topics	98
C.2.2 Consumer Group IDs	98
C.3 Shared Middleware	98
C.3.1 Authentication Middleware	98
C.3.2 Metrics Middleware	100
C.4 Load Test Script	101
D Workload B Reference	104
D.1 Training Script (ray-job.py)	104
E Experiment Scripts	110
E.1 Prometheus Metrics Export Script	110
E.2 Workload A Isolation (08a)	115
E.3 Workload B Isolation (08b)	115
E.4 No-Limits Co-location (09)	116
E.5 Static-Limits Co-location (10)	116
E.6 QoS-Controller Co-location (11)	117
F Supplementary Results	118
F.1 Cross-Condition Summary Table	118
F.2 Per-Run Measurements	118

F.3 Per-Run Time-Series Plots	118
---	-----

Chapter 1

Introduction

1.1 Background

The migration to hyperscale public cloud providers has historically offered organizations significant advantages in scalability, operational agility, and reduced capital expenditure. However, recent industry trends indicate a growing shift toward "cloud repatriation," where organizations move workloads from public clouds back to on-premises or private cloud infrastructures [1, 2].

This movement is primarily driven by the unpredictability of cloud costs and the need for tighter operational control. According to the 2024 Barclays CIO Survey, an unprecedented 83% of enterprise CIOs plan to repatriate at least some workloads from the public cloud back to private or hybrid infrastructure [3] (see Figure 1.1). Rather than abandoning the cloud entirely, the industry is seeing a targeted shift where highly predictable, steady-state workloads are brought back on-premises to reduce excessive subscription costs, while elastic workloads remain in the public domain.

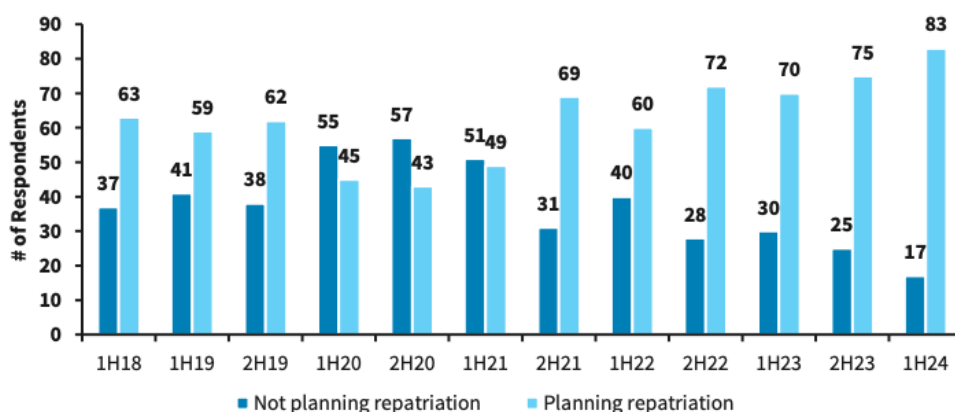


Figure 1.1: The Cloud Repatriation Trend: percentage of CIOs planning to repatriate workloads back to private infrastructure (Data based on the Barclays 2024 CIO Survey [3]).

Simultaneously, the revised Swiss Federal Act on Data Protection (nFADP, September 2023) and the risk of extraterritorial data access under the US Clarifying Lawful Overseas Use of Data Act (CLOUD Act) make maintaining digital sovereignty a strategic imperative [4, 5]. The regulatory implications and why a fully self-hosted architecture is required are examined formally in Chapter 2.

Concurrently, modern business operations increasingly rely on heterogeneous software ecosystems. Organizations must deploy standard, latency-sensitive web applications alongside resource-intensive artificial intelligence (AI) and big data workloads [6]. Hyper-scale providers manage these mixed workloads through massive hardware redundancy and sophisticated, proprietary orchestration layers. Reproducing this dual-capability ecosystem on constrained, private infrastructure introduces a concrete resource-sharing problem: latency-sensitive and throughput-intensive workloads compete directly for the same CPUs, memory, and network bandwidth. The standard approach for managing this contention in a private context relies heavily on strict physical hardware partitioning or static container limits. While static limits protect web applications from interference, they result in severe resource underutilization. When a web application experiences low traffic, its reserved CPU and memory remain idle, effectively stranding compute capacity that a co-located AI workload could otherwise utilize to accelerate training [6, 7].

1.2 Objectives

This thesis pursues three tightly coupled objectives. The first is to provision a reproducible, data-sovereign bare-metal Kubernetes cluster using Infrastructure as Code (IaC) with Ansible, establishing a private cloud foundation that operates entirely within national jurisdiction and does not depend on any hyperscale provider.

The second objective is to deploy two contrasting reference workloads on the same physical nodes: a latency-sensitive microservice web application (Workload A) and a throughput-intensive distributed AI training job (Workload B), thereby deliberately inducing the hardware contention that motivates the research.

The third, and central, objective is to design, implement, and empirically evaluate a custom feedback-driven QoS Controller. The controller continuously observes the P95 HTTP response latency of the web application via Prometheus and applies a Proportional-Integral (PI) control law to dynamically resize the CPU limits of the co-located AI workers through the Kubernetes in-place pod resize API. The research question this controller is designed to answer is: *Can a feedback-driven PI controller dynamically reclaim idle CPU capacity for a co-located AI workload while protecting the HTTP latency SLO of a web application running on the same bare-metal Kubernetes nodes?*

1.3 Deviation from the Original Task Description

The genesis of this bachelor thesis originated from a practical, real-world requirement: the need to build a highly scalable web application. Initially, the infrastructure planning was heavily skewed toward traditional hypervisor virtualization using Proxmox VE (Virtual Environment), as this was the extent of the author’s prior systems administration experience.

Upon preliminary review, the academic supervisor noted that deploying a standard web application on a hypervisor lacked the technical innovation required for a computer science bachelor thesis. To increase academic rigor, the original task description (see Appendix A.1) was formulated to include the orchestration of computationally intensive AI workloads alongside the web application, relying on static scheduling policies to ensure physical isolation.

However, subsequent research into modern data center architectures revealed that bare-metal Kubernetes, rather than Proxmox, is the established industry standard for scalable application orchestration. It also became evident that relying solely on static isolation (pinning workloads to separate physical nodes) presented the same limitation and failed to address the operational reality of constrained private clouds: resource efficiency.

Consequently, the thesis was deliberately pivoted to introduce an active software engineering component. Both workloads were intentionally co-located on the same physical nodes to induce hardware contention. Instead of static limits, a custom Quality-of-Service (QoS) Controller was engineered to dynamically resize co-tenant CPU limits at runtime via the Kubernetes API, driven by real-time latency feedback from Prometheus.

Due to this pivot, the "Chaos Testing" (simulating physical node failures) outlined in the original task description was omitted. High availability and node redundancy are native, solved problems in Kubernetes. Evaluating them does not contribute to the core research question of this thesis, which is strictly focused on dynamic workload interference mitigation and resource utilization efficiency.

1.4 Overview of the Thesis

This thesis is structured to guide the reader from theoretical foundations to practical implementation and empirical validation.

- **Chapter 2** establishes the theoretical background regarding data sovereignty, mixed workloads orchestration, and the mathematical principles of control systems applied to computing resources.
- **Chapter 3** details the methodological approach, including the hardware topology, the deployment of the reference workloads, and the software architecture of the

custom Dynamic QoS Controller.

- **Chapter 4** presents the empirical results of the system evaluation, contrasting the performance and utilization metrics of a static-limit baseline against the dynamic intervention of the custom controller under interference testing.
- **Chapter 5** discusses the findings in relation to the research question, evaluates the controller’s SLO compliance improvement and AI throughput gains against the static-limit baseline, addresses the controller’s limitations, and provides an outlook on future architectural improvements.

The complete repository structure, including all source files and Kubernetes manifests, is documented in Appendix A.2.

Chapter 2

Theoretical Foundations

2.1 Data Sovereignty and Cloud Paradigms

The fundamental premise of this thesis is the establishment of a data-sovereign architecture. Digital sovereignty refers to an organization’s or nation’s ability to maintain exclusive control over its digital assets, ensuring that data processing, storage, and governance remain strictly within its jurisdictional boundaries [5].

In the context of the revised Swiss Federal Act on Data Protection (nFADP), organizations are subjected to strict doctrines such as "Privacy by Design" and "Privacy by Default" [4]. Organizations that process highly sensitive personal data face significant compliance risks when relying on foreign-owned hyperscale cloud providers. Even if a hyperscaler operates data centers physically located within Switzerland, the corporate domicile of the provider can expose the hosted data to extraterritorial legal frameworks, such as the US CLOUD Act [5]. This legislation allows foreign law enforcement to compel technology providers to surrender data housed on their servers, effectively rendering local data localization strategies obsolete.

Consequently, Swiss regulatory authorities and privacy commissioners (such as Priva-tim) have emphasized that leveraging international cloud infrastructure for sensitive data is legally problematic unless the responsible entity implements zero-knowledge, end-to-end encryption where the cloud provider retains absolutely no access to the cryptographic keys [8, 9]. Because hyperscalers natively require access to the underlying data to facilitate platform services and indexing, satisfying this requirement is technologically prohibitive. A self-hosted, private cloud architecture structurally eradicates these compliance vectors by guaranteeing that the legal jurisdiction governing the data is perfectly congruent with the physical location and ownership of the hardware.

2.2 Abstraction Layers: Virtualization vs. Containerization

Choosing an abstraction layer between the bare-metal hardware and the workloads is the foundational architectural decision; it determines both the performance overhead the workloads pay and how easily the QoS controller can manipulate their resource limits.

The foundational technological decision in architecting a private cloud is determining the abstraction layer that interfaces with the physical hardware. The two prevailing paradigms are hypervisor-based virtualization and bare-metal container orchestration.

Type-1 hypervisors (such as KVM, the Kernel-based Virtual Machine) execute directly on the physical hardware, abstracting the underlying compute components into isolated Virtual Machines (VMs). The primary advantage of this paradigm is operational elasticity; administrators can easily execute snapshots, enforce strict hardware boundaries, and perform live migrations. However, hypervisor-based virtualization introduces a fundamental "hypervisor tax." Even with modern hardware-assisted virtualization extensions, a Type-1 hypervisor incurs approximately 10% CPU overhead relative to native bare-metal execution; I/O-intensive workloads suffer more pronounced penalties, with throughput reductions of up to 30% under database and network-heavy loads [10, 11].

This performance delta widens dramatically under the extreme I/O demands of AI training. Furthermore, exposing physical GPUs to virtual machines necessitates complex PCIe passthrough configurations. Pass-through complicates Non-Uniform Memory Access (NUMA) topologies, as every memory-related operation can incur penalties if the virtual machine must move data across different processor domains to access the physical device [12].

Conversely, bare-metal container orchestration entirely circumvents the hypervisor layer. Containerized workloads interact directly with the host operating system kernel via Linux Control Groups (cgroups) and namespaces. Because containers share the host kernel rather than emulating hardware, their overhead is substantially smaller: empirical benchmarks report approximately 5% performance degradation relative to bare metal for CPU-bound workloads [10, 11]. For AI and data-intensive workloads, this near-native execution path ensures that hardware accelerators are not gated by hypervisor translation layers. The cumulative effect of these abstraction-layer overheads becomes particularly apparent when Kubernetes itself is deployed atop VMs rather than bare metal: VM-based Kubernetes deployments achieve between 82% and 96% of bare-metal performance for standard stateless applications [13].

2.3 Microservices Architecture

The microservices architectural style defines an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms [14]. This stands in fundamental contrast to the traditional monolithic architecture, in which all business logic, data access layers, and user interface components are compiled and deployed as a single, indivisible artifact. In a monolith, a modification to any feature (regardless of its scope) necessitates the redeployment of the entire application, creating organizational bottlenecks and constraining the frequency of releases. As is well established in the literature [14, 15, 16], this tight coupling of components within a monolith also introduces scalability limitations: when one component requires additional resources, the entire application must be scaled, even if the bottleneck is localized to a single functional area.

The defining characteristic of the microservices pattern is the decomposition of application functionality along business capability boundaries [14]. Each service encapsulates a single, cohesive domain of responsibility and is developed, tested, deployed, and scaled entirely independently. This boundary-driven decomposition yields two critical operational advantages. First, services can be deployed individually; a change to the payments logic requires no redeployment of the authentication or ticketing services. Second, each service can be scaled horizontally in isolation: a service experiencing high request volume can have its replica count increased without provisioning additional resources for services that remain lightly loaded [16]. Newman describes this granular elasticity as one of the primary motivations for adopting the microservices pattern in latency-sensitive, user-facing systems [15].

Inter-service communication in a microservices architecture must be designed to prevent the tight coupling that characterizes monolithic systems. Synchronous, request-response HTTP between services introduces cascading failure modes: if a downstream service is slow or unavailable, the upstream caller is blocked, propagating the latency degradation across the entire call chain [15]. The predominant solution is asynchronous, event-driven communication via a persistent message broker. In this model, a service publishes a domain event to a durable topic and immediately returns; downstream consumers process the event independently and at their own pace, achieving temporal decoupling and resilience to partial failures [14]. Apache Kafka is the canonical implementation of this pattern, guaranteeing ordered, durable delivery of events across multiple independent consumer groups without contention. Complementing this communication model is the principle of polyglot persistence: each microservice owns its dedicated data store and selects the storage technology most appropriate for its access patterns [15]. This eliminates the shared-schema anti-pattern of monolithic systems, in which a single database becomes both a coordination point and a source of structural coupling between otherwise

independent components.

The ticket reservation platform constituting Workload A (described in Section 3.3) in this thesis is a canonical instantiation of the microservices pattern. Its five services (authentication, ticket inventory, order management, payment processing, and reservation expiration) map directly onto five distinct business capability boundaries, each with its own persistence layer and independent horizontal scaling policy [14]. The services communicate exclusively via Kafka events: a ticket purchase triggers an ordered chain of domain events across the order, payment, and expiration services without any direct HTTP coupling between them. This design makes the application inherently latency-sensitive; the user-facing HTTP purchase flow traverses multiple service boundaries, and any stall (whether from resource contention, event backlog, or scheduling jitter) manifests directly as increased end-to-end response latency. The application is deliberately engineered to maintain significant CPU utilization under load (rather than being purely I/O-bound), so that co-located resource contention manifests as measurable latency degradation rather than being absorbed by idle I/O wait time. It is precisely this interaction between a CPU-sensitive, latency-critical web application and a co-located throughput-heavy AI workload that the noisy-neighbor problem, discussed in the following section, formalizes.

2.4 Orchestrating Mixed Workloads and the "Noisy Neighbor" Problem

Integrating standard web applications alongside AI data processing tasks on shared infrastructure introduces severe orchestration challenges due to their opposing architectural requirements.

Standard web applications (e.g., e-commerce microservices) are highly volatile, latency-sensitive, and require high availability. They process thousands of brief, unpredictable HTTP requests and require low-latency CPU scheduling and network responsiveness [6]. Conversely, AI workloads, particularly deep learning models, are monolithic and throughput-oriented. They demand sustained, uninterrupted access to compute accelerators and maximum memory bandwidth over extended periods.

When these heterogeneous workloads are co-located without strict orchestration policies, the infrastructure suffers from the "noisy neighbor" phenomenon [17]. This phenomenon is not merely a software scheduling issue, but a physical hardware bottleneck. Even if a container orchestrator pins a web application to CPU Core 0 and an AI workload to CPU Core 1, the two processes still share underlying hardware pathways, such as the L3 processor cache, the memory bus, and the Network Interface Card (NIC).

As illustrated in Figure 2.1, when an intensive AI training job processes massive multidimensional tensors, it can easily saturate the memory bus. Consequently, when the

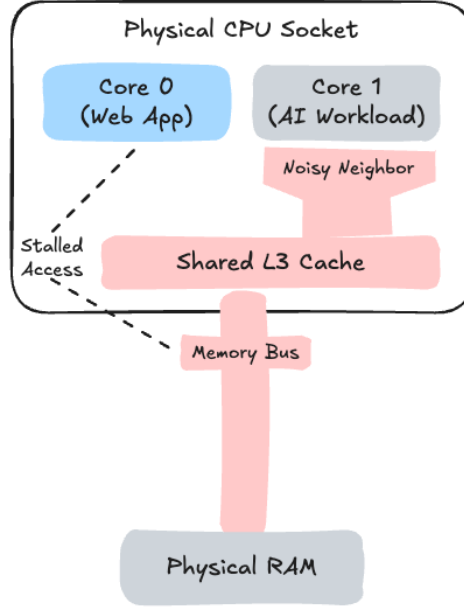


Figure 2.1: The physical mechanics of the Noisy Neighbor problem: independent CPU cores contending for shared L3 Cache and Memory Bus bandwidth.

web application receives an HTTP request and attempts to fetch data from RAM, its CPU requests stall while waiting for the memory bus to become available. This interference instantly degrades the responsiveness of co-located web applications, violating their Service Level Agreements (SLAs).

In this thesis, the noisy-neighbor role is played by Workload B: a distributed deep learning training job (described in Section 3.4) that saturates multiple CPU cores and both GPU workers through a computationally intensive preprocessing pipeline. Its co-location with Workload A on the same bare-metal nodes provides the experimental basis for evaluating the QoS controller.

2.5 GPU Acceleration and Multiplexing

The integration of GPU acceleration introduces an additional layer of scheduling complexity. Enterprise-grade data center GPUs (such as the NVIDIA A100 or H100) feature hardware-level Multi-Instance GPU (MIG) capabilities, allowing a single physical card to be securely partitioned into fully isolated mini-GPUs with dedicated memory and compute cores [18].

Consumer-grade GPUs, however, lack MIG support. Executing multiple distinct workloads on consumer hardware requires software-level multiplexing. The default mechanism in Kubernetes is Compute Unified Device Architecture (CUDA) Time-Slicing, which leverages the GPU hardware scheduler to rapidly context-switch between applications [18]. While this allows oversubscription, it provides no memory isolation, leading to potential

out-of-memory crashes if one process consumes the shared video RAM (VRAM). Furthermore, temporal context switching incurs a measurable computational penalty.

A superior alternative for maximizing hardware utilization without severe context-switching latency is the CUDA Multi-Process Service (MPS). MPS operates as a transparent scheduling funnel, injecting CUDA commands from multiple independent processes into a single execution queue [19]. This enables spatial multiplexing, allowing kernels from fundamentally different processes to execute concurrently on the physical silicon, significantly boosting total GPU utilization. Since both GPU workers in the bare-metal cluster used for this thesis are consumer-grade cards (an RTX 3090 and an RTX 3080) that lack MIG support, and each training worker is assigned its own physical GPU, neither time-slicing nor MPS is required.

2.6 Distributed Computing Frameworks and Network Constraints

Workload B is a distributed training job coordinated by Ray. Understanding why Ray was chosen over Spark, and what properties of its architecture affect CPU utilization on shared nodes, requires a brief comparison of the two frameworks.

When an AI computational task exceeds the capacity of a single machine, the workload must be mathematically distributed. The two predominant frameworks for this are Apache Spark and Ray.

While Apache Spark is highly mature and optimized for data-parallel batch analytics, its coarse-grained DAG execution model is less suited for the fine-grained, stateful, and dynamically scheduled tasks required by deep learning [20]. Ray abandons the rigid batch-processing paradigm in favor of dynamic task graphs and a universal actor model. Crucially, Ray’s distributed shared-memory object store enables zero-copy deserialization of tensor and array data within a node, significantly reducing data movement overhead for preprocessing-heavy workloads [20].

The efficacy of these frameworks is heavily dictated by physical network constraints. In distributed training, nodes must continuously synchronize calculated mathematical gradients across the inter-node network before the subsequent training step can commence. In an infrastructure operating over constrained network interconnects, such as the 2.5 Gbps links of the bare-metal cluster used in this thesis, massive gradient payloads can require several seconds of pure transmission time, during which the highly capable GPUs sit entirely idle [21]. In Workload B, the ResNet-50 gradient payload is modest and inter-node communication is not the dominant bottleneck; the workload is deliberately CPU-bound, as described in Section 2.7.

2.7 Distributed Deep Learning and Data Parallelism

The QoS controller throttles Ray workers’ CPU limits. For this throttle to affect AI training throughput (and for the thesis experiment to be meaningful) the training job must be CPU-bottlenecked rather than GPU-bottlenecked. The following subsections explain the distributed training architecture (parallelism strategies, DDP synchronization, and AllReduce) and then show why, in Workload B’s configuration, the CPU is the dominant bottleneck.

2.7.1 Parallelism Strategies

Distributed deep learning training addresses a fundamental scaling problem: a single GPU has finite compute and memory, yet both model size and dataset scale continue to grow. Two principal strategies partition the training problem across multiple devices. In *model parallelism*, the model itself is partitioned across devices; either by assigning different layers to different accelerators (pipeline parallelism) or by splitting individual weight tensors across devices (tensor parallelism). In *data parallelism*, each device holds a complete model replica and independently processes a disjoint shard of the training batch; after the backward pass, gradient updates are aggregated across all replicas so that every device remains numerically identical. When a model’s parameter count fits within the memory of a single accelerator, data parallelism is the preferred strategy: it requires no inter-device communication during the forward or backward pass, only a single gradient synchronization step per iteration, and delivers near-linear throughput scaling as long as communication overhead remains low relative to computation time [22].

Workload B in this thesis applies data parallelism across two GPU workers. ResNet-50 has approximately 25 million parameters, corresponding to roughly 100 MB of single-precision weights; well within the capacity of both GPU workers deployed on the cluster. Model parallelism would introduce inter-device communication on the critical path of every forward pass and add implementation complexity without benefit; a straightforward two-worker data-parallel configuration is therefore used, with each worker processing an independent shard of each synthetic batch. The concrete deployment is described in Section 3.4.

2.7.2 DistributedDataParallel

PyTorch’s `DistributedDataParallel` (DDP) module is the standard implementation of synchronous data-parallel training. Each participating process (termed a *rank*) holds a full copy of the model parameters and an exclusive slice of the training data. During the forward pass, every rank computes its portion of the loss independently, with no inter-rank communication. During the backward pass, PyTorch’s autograd engine computes

local gradients for each parameter; DDP intercepts these gradients via autograd hooks and invokes an AllReduce operation (a collective communication primitive that gathers gradient tensors from all participating ranks, computes their element-wise average, and redistributes the result to every rank; described in detail in Section 2.7.3) [21]. Because every rank receives the same averaged gradient, a single synchronized optimizer step preserves exact model consistency without parameter servers or sequential coordination.

To reduce communication latency, DDP employs *gradient bucketing*: parameters are grouped into fixed-size buckets, and each bucket’s AllReduce is triggered as soon as all gradients within it become available during the backward pass. This overlaps collective communication for earlier layers with gradient computation for later layers, hiding a significant fraction of the communication cost behind useful computation [21]. The synchronization nevertheless imposes a strict barrier: every rank must complete its backward pass before any rank can receive the averaged gradients and advance to the optimizer step. A single slow rank (one whose backward pass is delayed, for instance because its GPU was starved of input data) stalls the entire training cohort until the barrier is cleared.

Workload B applies this configuration with two GPU workers distributed across the cluster nodes. Gradient synchronization occurs over NCCL (NVIDIA’s Collective Communications Library) via the Kubernetes pod network. At startup, each worker logs its `world_rank` and `world_size`; a `world_size` of two in these logs confirms that both workers successfully joined the distributed process group.

2.7.3 AllReduce and the Barrier Problem

The AllReduce collective must sum (or average) N independent gradient tensors and deliver the result to all N participants simultaneously. A naive implementation via a central parameter server creates a bandwidth bottleneck that scales linearly with N . The ring-allreduce algorithm eliminates this bottleneck by arranging workers in a logical ring and decomposing the operation into two phases. In the *reduce-scatter* phase, each worker sends a fragment of its gradient tensor to its right neighbor and accumulates the incoming fragment from its left neighbor; after $N - 1$ steps, each worker holds the fully reduced sum for one contiguous fragment of the parameter space. In the *all-gather* phase, each worker propagates its reduced fragment around the ring so that all workers end with the complete averaged gradient. The total data transmitted per worker equals $\frac{2(N-1)}{N} \cdot M$ bytes for a gradient of size M , which converges to $2M$ as N grows, making communication cost independent of worker count and determined solely by the slowest network link in the ring [23].

NCCL provides a highly optimized GPU implementation of ring-allreduce. NCCL selects the fastest available transport automatically (NVLink for intra-node GPU pairs, PCIe, or the IP network for inter-node communication) and executes each collective inside

a single CUDA kernel that interleaves data transfer with partial reduction arithmetic. PyTorch uses NCCL as the default backend for CUDA-enabled distributed training, meaning that DDP’s AllReduce calls in Workload B are transparently dispatched to NCCL over the Kubernetes pod network interconnecting the two GPU worker nodes.

Because ring-allreduce is synchronous, any stall on one worker propagates to all others: the ring cannot advance until every participant has transmitted its fragment for the current step. A preprocessing delay on one rank (caused, for instance, by CPU throttling that slows the augmentation pipeline and leaves the GPU waiting for the next batch) delays that rank’s backward pass and therefore holds the AllReduce barrier open for all ranks. If the delay exceeds PyTorch’s distributed watchdog timeout (configurable via the `timeout` parameter of `init_process_group`, defaulting to 10 minutes for the NCCL backend), the watchdog aborts the NCCL communicator and terminates the job [24]. In deployments where CPU contention is sufficient to exhaust the preprocessing budget and hold the AllReduce barrier open beyond this threshold, the mechanism can terminate an entire multi-node training job. This makes the CPU resource governance a correctness concern, not only a performance concern.

2.7.4 CPU Preprocessing as the Throughput Bottleneck

A common assumption in distributed training is that the GPU is the system bottleneck. In practice, data augmentation pipelines for computer vision tasks frequently exceed the GPU in computational cost: when the time to prepare a batch on the CPU is longer than the time to execute a forward and backward pass on the GPU, the GPU sits idle between steps waiting for the next batch (a CPU-induced stall that negates the hardware investment in accelerators) [25]. The *prefetch-queue* pattern addresses this by decoupling CPU and GPU timelines: a background daemon thread runs the preprocessing pipeline continuously, pushing completed batches into a bounded queue; the training loop consumes from the front of the queue while the daemon fills new entries at the back. The bounded queue capacity applies back-pressure, preventing unbounded memory consumption when the GPU temporarily outpaces the CPU. However, this pattern only masks the bottleneck while the CPU can sustain the preprocessing rate required to keep the queue non-empty; if CPU throughput is reduced (for example, by a reduced Kubernetes CPU limit) the queue drains and GPU stalls recur.

Workload B is explicitly constructed to operate in this CPU-bottlenecked regime. Its preprocessing pipeline is designed so that batch preparation time consistently exceeds GPU forward-and-backward time, keeping CPU cores saturated throughout the entire training run and not only during data preparation phases. The full pipeline design is described in Section 3.4. The consequence for resource governance is that reducing available CPU cycles drains the prefetch queue and causes GPU stalls, making CPU limit

adjustment a direct lever on AI training throughput; a property exploited by the QoS controller described in Section 3.5.

2.8 Quality of Service (QoS) in Containerized Environments

To overcome the physical hardware contention described in Section 2.4, infrastructure must employ Quality of Service (QoS) mechanisms. Traditionally a networking concept used to prioritize specific packets (e.g., guaranteeing bandwidth for VoIP over file downloads), QoS in a containerized operating system refers to the prioritization of computational resources, specifically CPU, Memory, and Disk I/O.

In a Kubernetes cluster, QoS is categorized into three primary classes based on pod configuration: **Guaranteed**, **Burstable**, and **BestEffort** [26]. However, the actual enforcement of these classes relies entirely on the underlying Linux host kernel, specifically through Linux Control Groups (cgroups) [27].

Cgroups limit, account for, and isolate the resource usage of a collection of processes [28]. For CPU prioritization, the kernel utilizes the Completely Fair Scheduler (CFS) [29]. When a container orchestrator sets a CPU limit, it modifies the `cpu.max` parameter within the cgroup virtual filesystem [27]. This mechanism acts as a strict bandwidth controller. By dynamically patching the pod’s CPU limit via the Kubernetes API (which the kubelet propagates to these cgroup files within a single CFS scheduling period) a system can instantly restrict a background task’s access to the processor (forcing the kernel to yield CPU cycles to the latency-sensitive web application) without requiring the background application to be terminated or restarted.

2.9 Dynamic Resource Management and Control Theory

While cgroups provide the physical mechanism to throttle processes, determining *when* and *how much* to throttle requires intelligent, dynamic resource management. This is achieved by implementing control loops that autonomously observe system health and manipulate allocations accordingly.

2.9.1 The Control Loop Pattern

Modern orchestration relies on the Operator or Controller pattern; a non-terminating background process that continuously tracks the state of the cluster via an API. The logic of a controller strictly follows a three-step cycle:

1. **Observe:** The controller reads the current state of the system and specific workload metrics (e.g., via a time-series database like Prometheus).
2. **Analyze:** It compares the observed state against a desired state or predefined Service Level Agreement thresholds.
3. **Act:** If a discrepancy exists, the controller executes API calls to modify the system configuration (e.g., patching a pod’s CPU resource limits via the Kubernetes in-place resize API) to realign the system [30].

2.9.2 Proportional-Integral (PI) Control in Computing

To prevent erratic oscillation when adjusting resources dynamically, the "Analyze" phase of the control loop can be modeled using principles from automation control theory. Recent academic research demonstrates the efficacy of applying Proportional-Integral (PI) controllers to manage container scaling and resource provisioning based on real-time application demands [7].

In a PI-driven resource management system, the controller continuously observes a target variable, such as the 95th percentile (p95) HTTP response latency ($L_{current}$), and compares it against a predefined SLA threshold (L_{target}) to calculate the current system error $e(t)$:

$$e(t) = L_{current} - L_{target} \quad (2.1)$$

Based on this error, the controller calculates the control output $u(t)$, which dictates the precise amount of CPU quota restriction to apply to the interfering background workloads:

$$u(t) = K_p e(t) + K_i \int e(t) dt \quad (2.2)$$

Where:

- K_p represents the proportional gain, dictating the immediate, linear reaction to a sudden latency spike.
- K_i represents the integral gain, which accounts for the accumulation of past errors to eliminate steady-state offset, ensuring the latency strictly returns to the target threshold [7].

Selecting appropriate values for K_p and K_i requires tuning the controller to the specific system dynamics. Systematic approaches such as the Ziegler-Nichols method derive gain values analytically from the open-loop step response of the plant; however, for dynamic infrastructure with non-stationary workloads, empirical tuning is more reliable in practice. Gains are adjusted iteratively through controlled experiments until the controller

achieves an acceptable trade-off between response speed (governed by K_p) and steady-state accuracy (governed by K_i) without inducing oscillation. The specific values applied in this thesis and the reasoning behind them are described in Section 3.5.

By coupling the mathematical precision of PI control loops with the Kubernetes in-place pod resize API (backed by Linux cgroup enforcement) a private cloud architecture can safely co-locate highly volatile web applications with throughput-heavy AI tasks, dynamically suppressing hardware interference in real time.

Chapter 3

Approach / Methods

3.1 Architecture Design and Hardware Topology

The cluster is designed to deliberately induce hardware contention between the two workloads: isolating them on separate physical nodes would make the noisy-neighbor problem impossible to observe and the QoS controller unnecessary.

To fulfill the requirements of a data-sovereign private cloud while enabling dynamic Quality-of-Service (QoS) control, the prototype was implemented as a three-node bare-metal Kubernetes cluster. Physical isolation of workloads was deliberately abandoned in favor of intentional co-location on shared nodes, creating the hardware contention conditions required to evaluate the effectiveness of the dynamic controller.

Table 3.1 summarizes the hardware specification and cluster network address of each node. All nodes are equipped with at least one 1 TB NVMe SSD for local storage. Both processor models used in this cluster (the Ryzen 7 5800X and the Ryzen 7 5700) feature 8 physical cores and 16 logical threads. The Ray head pod runs on `cpu-node-1` alongside the Kubernetes control plane; both Workload A (web application) and Workload B (AI training workers) are intentionally scheduled exclusively on the two worker nodes to co-locate and induce contention.

Hostname	Role	CPU	RAM	GPU	Cluster IP
<code>cpu-node-1</code>	Control Plane	Ryzen 7 5800X	64 GB	—	10.0.1.11
<code>gpu-node-1</code>	Worker	Ryzen 7 5700	96 GB	RTX 3090 (24 GB)	10.0.1.12
<code>gpu-node-2</code>	Worker	Ryzen 7 5800X	96 GB	RTX 3080 (10 GB)	10.0.1.13

Table 3.1: Hardware specifications of the three-node bare-metal Kubernetes cluster.

Each node has two network interface controllers (NICs): a 1 Gbps motherboard interface on `192.168.1.0/24` used exclusively for SSH and Ansible access, and a dedicated cluster interface on the same switch (60 Gbit/s switching capacity; `10.0.1.0/24`) carrying

all inter-node Kubernetes traffic. The GPU worker nodes use 2.5 Gbps PCIe cluster interfaces; `cpu-node-1` uses a 1 Gbps connection on the same subnet. Because distributed Ray training traffic flows directly between the two GPU workers, the asymmetric bandwidth on the control-plane node does not affect the primary compute data path. The complete network configuration, including interface names, static IP assignments, and MTU 9000 Jumbo Frame setup, is described in Section 3.2.1 and Appendix B.1.

All software components are pinned to exact versions (Table 3.2) so that re-running the provisioning playbooks after a cluster reset produces an identical software stack, ensuring all experimental measurements are comparable.

Component	Version
Ubuntu	24.04 LTS
Kubernetes (kubelet, kubeadm, kubectl)	v1.34
containerd	2.2.3-1~ubuntu.24.04~noble
Helm	v3.17.0
Calico CNI	v3.29.3
NVIDIA GPU Operator	v24.9.2 (Helm chart)
kube-prometheus-stack	69.3.0 (Helm chart)
ingress-nginx	4.12.1 (Helm chart)
KubeRay operator	1.2.2 (Helm chart)

Table 3.2: Software component versions pinned across the cluster.

As depicted in Figure 3.1, both workloads are co-located on `gpu-node-1` and `gpu-node-2`, while `cpu-node-1` hosts the Kubernetes control plane and the Ray head node.

3.2 Infrastructure-as-Code Provisioning and Cluster Bootstrapping

Provisioning transforms three raw Ubuntu 24.04 machines into a fully configured Kubernetes cluster through ten sequential playbooks. Each playbook handles one layer of the stack, and all are idempotent (re-running them on an already-configured node is safe). Detailed steps are documented in Appendix B; this section explains the key engineering decisions.

To ensure the reproducibility of the private cloud environment, the entire bare-metal provisioning process was codified using Ansible. This IaC approach mitigates configuration drift and provides a declarative methodology for transitioning the hardware from a raw Ubuntu 24.04 operating system to a fully functioning, data-sovereign Kubernetes orchestration layer. The infrastructure consists of one control-plane node (`cpu-node-1`) and two GPU-accelerated worker nodes (`gpu-node-1`, `gpu-node-2`).

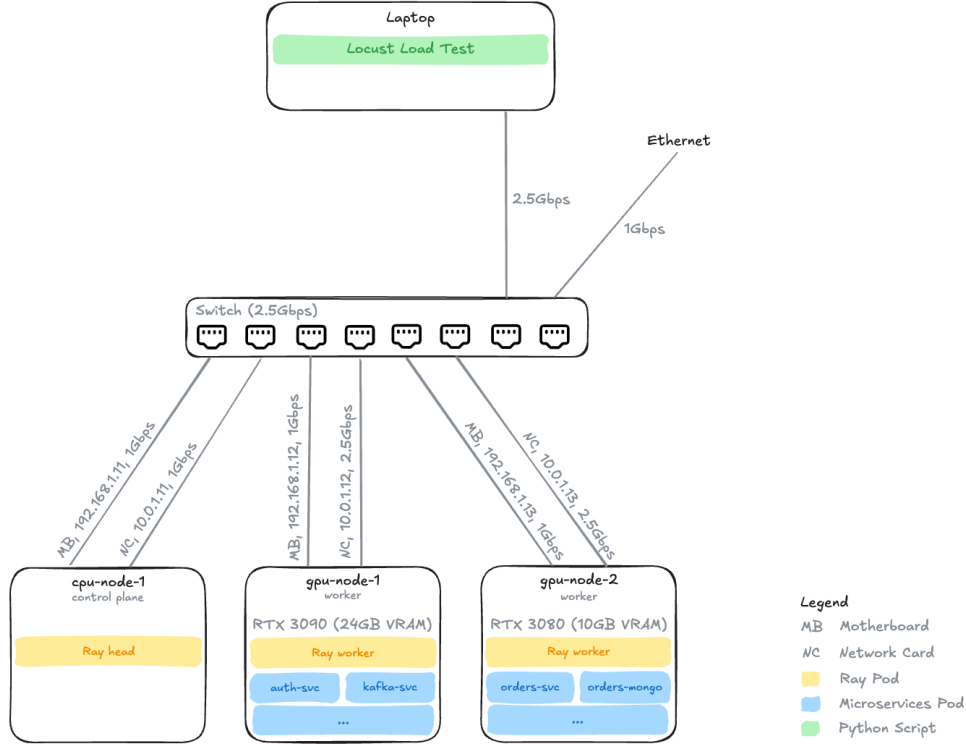


Figure 3.1: Hardware and network topology diagram showing the dual-NIC configuration, asymmetric cluster network bandwidth, and workload distribution across the control-plane and GPU nodes.

3.2.1 Network Configuration

Each node’s cluster NIC was configured with a static IP on the 10.0.1.0/24 subnet and an MTU of 9000 to enable Jumbo Frames. Jumbo Frames amortize per-frame kernel interrupt overhead across larger payloads, reducing CPU interrupt frequency during Ray DDP gradient synchronization and freeing CPU cycles for the application workloads. A dedicated Netplan file is used for the cluster interface so the management interface (192.168.1.0/24) is never altered; the cluster-network configuration can be fully reversed by deleting a single file (see Appendix B.1).

3.2.2 OS and Container Runtime Prerequisites

Two host-level changes are prerequisites for the cluster. First, Linux swap was completely disabled on all nodes: the `kubelet` refuses to start with swap active (`--fail-swapon=true` is the default), making this a hard Kubernetes prerequisite; additionally, with swap enabled, memory limits become unreliable because the kernel may page container memory to disk rather than triggering the Out-of-Memory (OOM) killer. Second, `containerd` was configured with `SystemdCgroup = true`, selecting `systemd` as the unified cgroup manager. This ensures the `kubelet` and the host OS agree on resource accounting paths, which is a hard requirement for the `kubelet` to reliably propagate the QoS controller’s

CPU limit updates to the correct `cpu.max` entry in the cgroup v2 hierarchy at runtime. All software components are pinned to exact versions with `apt-mark hold` (Table 3.2), preventing unattended upgrades from altering the runtime environment mid-experiment (see Appendix B.2–B.3).

3.2.3 Cluster Bootstrap

Kubernetes v1.34 was installed and version-pinned via the Advanced Package Tool (APT). The `kubelet` on each node was explicitly bound to its `10.0.1.x` address via a `KUBELET_EXTRA_ARGS` entry in the `systemd` environment file. Without this binding the `kubelet` defaults to the management-network address, routing API and pod traffic over the 1 Gbps interface and preventing Ray DDP AllReduce from using the faster PCIe cluster interconnect.

Calico v3.29.3 was selected as the CNI for its robust bare-metal routing support. Because the nodes are multi-homed (dual-NIC), an initial deployment without explicit interface binding caused split-brain routing: Calico built its overlay on the `192.168.1.x` management subnet while the `kubelet` expected traffic on `10.0.1.x`, resulting in ICMP blackholing. Setting `IP_AUTODETECTION_METHOD=cidr=10.0.1.0/24` forces Calico to select the correct PCIe interface on each node (see Appendix B.4–B.5).

3.2.4 Observability Stack and Operators

Three supporting stacks complete the cluster infrastructure. The `kube-prometheus-stack` bundles Prometheus (time-series metrics storage and querying), Grafana (dashboarding), and Node Exporter (host-level CPU, memory, and I/O metrics), providing the observability layer that drives both the QoS controller’s latency observations and the collection of all experimental measurements. The NVIDIA GPU Operator manages containerized GPU driver installation on each worker node and exports per-GPU utilization metrics via an embedded DCGM exporter. The KubeRay operator orchestrates the persistent `RayCluster` used by Workload B. These three stacks must be installed in a specific order.

The `kube-prometheus-stack` chart is deployed before the GPU Operator because the GPU Operator Helm chart renders the DCGM `ServiceMonitor` only when the `monitoring.coreos.com/v1 Custom Resource Definition (CRD)` is already present; deploying out of order silently omits GPU metrics from Prometheus permanently. Two Helm flags (`serviceMonitorSelectorNilUsesHelmValues=false` and `podMonitorSelectorNilUsesHelmValues=false`) enable cross-namespace `ServiceMonitor` and `PodMonitor` discovery, which is required to collect metrics from `gpu-operator`, `ingress-nginx`, `ray-system`, and `qos-system`.

The NVIDIA GPU Operator replaces manual driver and CUDA toolkit installation on each host with a containerized driver that compiles against the running kernel, keeping the host OS pristine and eliminating per-node configuration drift. The NGINX Ingress

Controller exposes Workload A on NodePort 30080. The P95 latency signal that drives the PI control law is sourced not from the NGINX per-request histogram but from the application-level `http_request_duration_seconds_bucket` metric emitted by each ticket-app microservice via a Prometheus middleware and scraped by `kube-prometheus-stack` through the `ticket-app/ticket-app-services` ServiceMonitor on port 3000 (see Section 3.3.3).

The KubeRay operator and a persistent `RayCluster` are deployed once and shared across all 15 experiment runs rather than being recreated between conditions. Ray stores the pip runtime environment (including `torch` and `torchvision` CUDA wheels) in the pod’s ephemeral `tmpfs`. Recreating the cluster between conditions would evict this cache, triggering a 5–10 minute PyTorch re-download before training could begin, inflating makespan measurements and confounding cross-condition comparisons (see Appendix B.6–B.9).

3.3 Workload A: Ticket Application

Workload A is a six-service microservices ticket reservation platform whose architecture is largely adapted from an existing course project, with the original NATS Streaming event bus replaced by Apache Kafka, serving as the latency-sensitive reference workload in all co-location experiments [31]. As shown in Figure 3.2, external HTTP requests enter through the NGINX Ingress Controller, which routes each request directly to the responsible service based on path prefix: `/api/users` to `auth-srv`, `/api/tickets` to `tickets-srv`, `/api/orders` to `orders-srv`, `/api/payments` to `payments-srv`, and all remaining paths (`/`) to the Next.js `client-srv`. Each service owns an independent MongoDB instance and communicates asynchronously via Kafka; no direct service-to-service HTTP calls exist between backend services.

In early iterations, Node.js CRUD handlers were almost entirely I/O-bound; node CPU utilization stayed at $\approx 5\%$ even at 400 concurrent users, rendering CPU contention invisible to the kernel scheduler and to the QoS controller. To make Workload A CPU-visible, the orders service computes a Password-Based Key Derivation Function 2 (PBKDF2) receipt token per order creation using `crypto.pbkdf2Sync` with 42 000 SHA-256 iterations synchronously in the request handler (see Appendix C.1.3). At peak load (84 orders per second during Phase 2), the PBKDF2 computation raises steady-state node CPU utilization from $\approx 5\%$ to $\approx 16\text{--}20\%$.

The QoS controller uses P95 latency $\leq 150\text{ ms}$ as its SLA input signal (`L_TARGET_MS`). Experiment evaluation uses the stricter P99 $< 200\text{ ms}$ threshold as the SLO metric, discussed in Section 3.3.3.

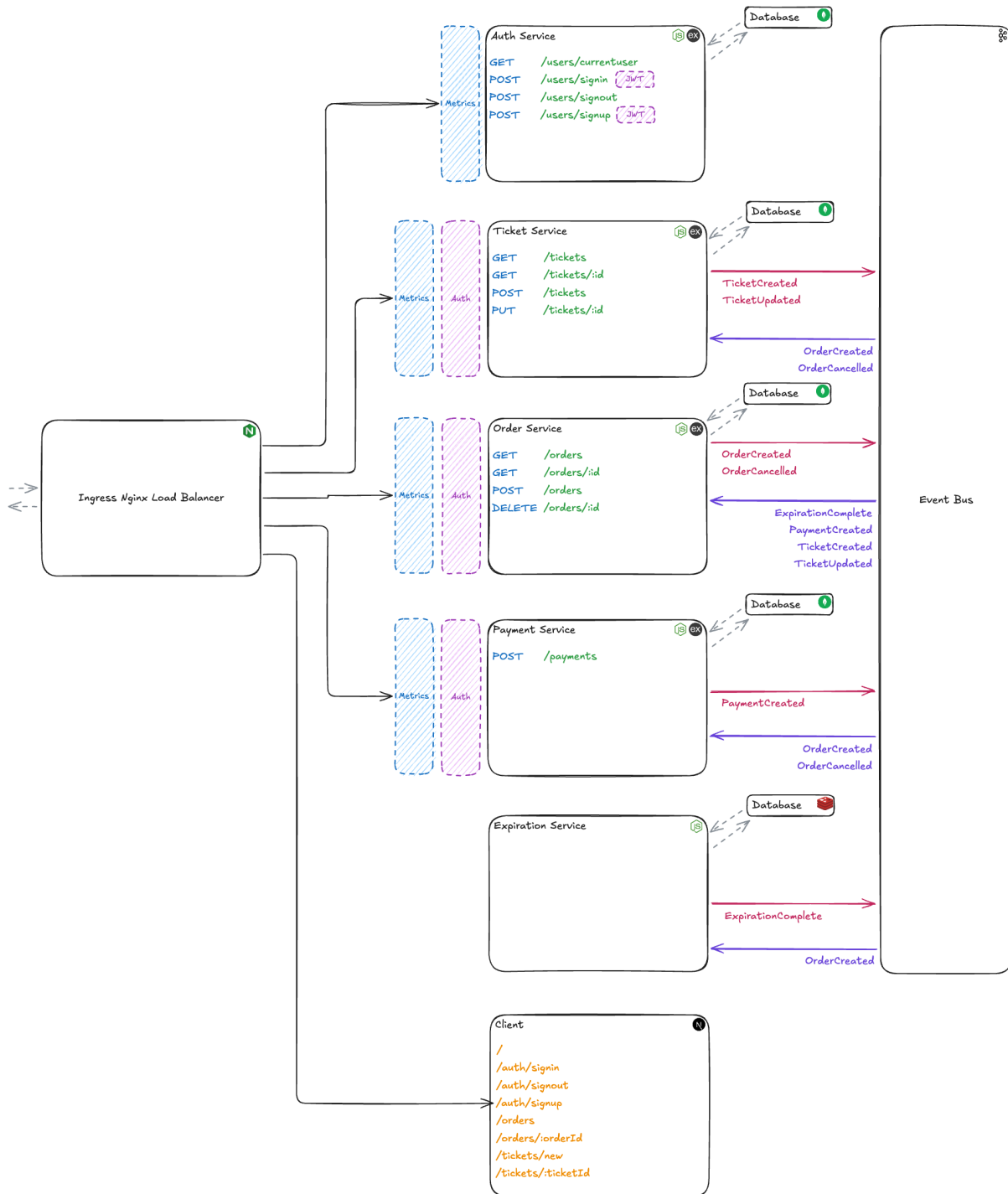


Figure 3.2: Architecture of Workload A. External traffic enters via NGINX Ingress, which routes requests by path prefix directly to the responsible backend service. Frontend routes (/) are served by the Next.js client. Services communicate asynchronously via Kafka (Event Bus); each service owns an isolated database instance. Tickets and Orders run two replicas each.

3.3.1 Service Decomposition and Responsibilities

Table 3.3 describes the six services that compose Workload A. All services are deployed as `Deployment` objects on `gpu-node-1` and `gpu-node-2` via node affinity rules. The `tickets` and `orders` services run two replicas each, doubling their CPU footprint across both worker nodes and making their resource consumption measurable under the QoS scheduling experiment. All MongoDB instances are intentionally ephemeral (no `PersistentVolume`), guaranteeing clean state between experiment runs.

Service	Technology	Responsibility	Replicas
<code>client</code>	Next.js	Frontend and event catalog; NGINX catch-all route (/)	1
<code>auth</code>	Node.js/Express	User registration, login, JWT session management	1
<code>tickets</code>	Node.js/Express	Ticket inventory, catalog, and pricing	2
<code>orders</code>	Node.js/Express	Purchase state machine (created → awaiting:payment → complete); computes PBKDF2 receipt token per order creation	2
<code>payments</code>	Node.js/Express	Mocked payment gateway with hardcoded 500 ms processing delay [†]	1
<code>expiration</code>	Node.js	Reservation countdown worker (Bull queue, Redis); cancels unpaid orders after <code>EXPIRATION_WINDOW_SECONDS</code>	1

[†]Excluded from SLO latency metrics; see Section 3.3.3.

Table 3.3: Workload A services, their technology stack, and primary responsibilities. Full HTTP API specifications are listed in Appendix C.1.

3.3.2 Event Bus and Asynchronous Communication

All backend services communicate exclusively via Kafka events; no direct inter-service HTTP calls exist. The key topics and their consumers are (full event reference in Appendix C.2):

- `ticket.created`, `ticket.updated` — consumed by the `orders` service to maintain a local projection of ticket state (price, reservation status) without cross-service queries.
- `order.created` — consumed by the `payments` service (to await payment submission) and the `expiration` service (to start the reservation countdown timer).
- `order.cancelled` — consumed by the `tickets` service to release the reservation, and by the `payments` service to abort the pending charge.
- `payment.created` — consumed by the `orders` service to transition order state to complete.

- `expiration.complete` — consumed by the orders service to cancel the order when the reservation window closes without payment.

Each Kafka listener is assigned a unique `groupId` following the `{service}-{topic-slug}` convention (e.g. `orders-ticket-created`); the expiration service, which subscribes to a single topic, uses the abbreviated `expiration-service` (full list in Appendix C.6). Sharing a `groupId` across listeners subscribed to different topics causes Kafka to assign empty partition sets during rebalance, silently dropping events.

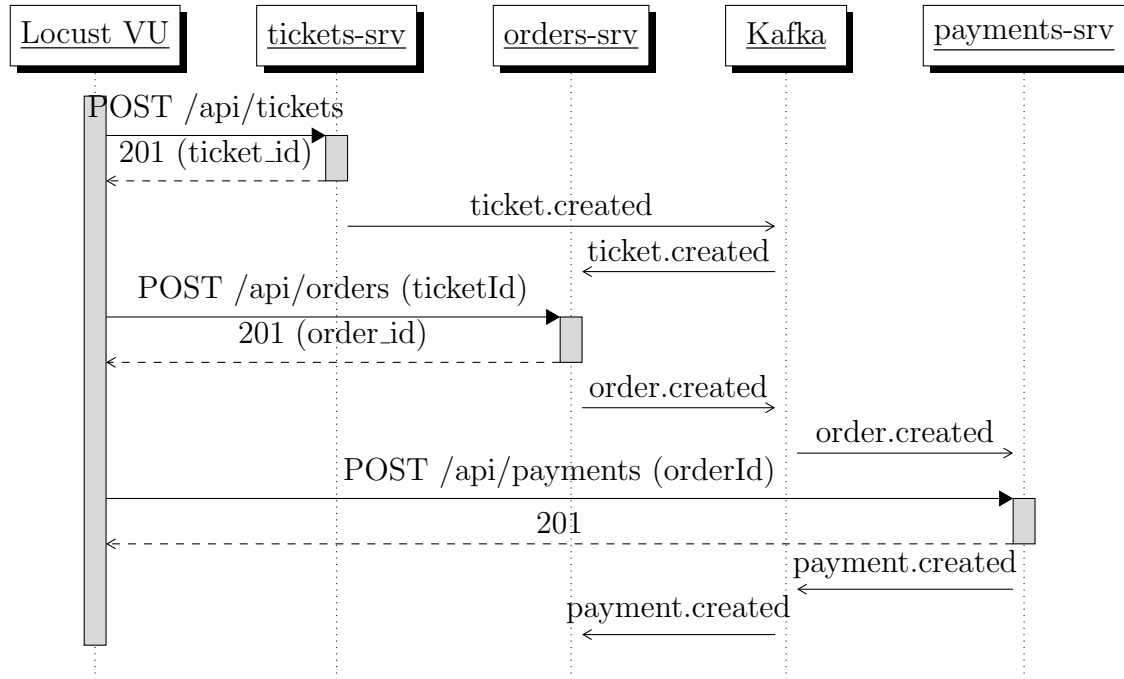


Figure 3.3: Sequence diagram of the Locust purchase flow. Filled arrows are synchronous HTTP calls (request/response); open arrows are asynchronous Kafka events. Between the ticket creation response and the order request, a `gevent.sleep(500ms)` pause allows the `ticket.created` event to propagate to `orders-srv` before the order is submitted; a second `gevent.sleep(300ms)` serves the same purpose for `order.created` before the payment request. The `expiration-srv` is omitted: it consumes `order.created` to start a countdown timer but is not part of the HTTP response path.

Coordination delays in load generation. The Locust purchase flow must respect Kafka consumer lag. After creating a ticket, the virtual user waits `gevent.sleep(500ms)` before submitting the order request; without this pause, the orders service has not yet consumed the `ticket.created` event and rejects the order as referencing an unknown ticket. A second `gevent.sleep(300ms)` after order creation gives the payments service time to consume `order.created` before the payment request arrives. `gevent.sleep` is used rather than `time.sleep` to avoid blocking the Locust greenlet pool (see Appendix C.4).

Expiration window. The original hardcoded 5-minute expiration window caused orders created during Phase 1 ($t = 0\text{--}180\text{s}$) to expire at $t \approx 480\text{s}$, coinciding with the Phase 2 traffic spike. The resulting flood of `expiration.complete` events on top of 400-user load produced P99 spikes unrelated to CPU contention, confounding cross-condition comparison. Setting `EXPIRATION_WINDOW_SECONDS=1800` (30 minutes) in the ConfigMap pushes all expirations beyond the 15-minute test window, eliminating this confound.

3.3.3 SLA/SLO Definition and Measurement

Two distinct thresholds are used: the SLA governs real-time controller behavior (the controller reacts to this signal), while the SLO is the offline evaluation metric reported in Chapter 4. Separating them allows the controller to act conservatively (throttling before the SLO is violated) while evaluation uses the stricter boundary.

Two latency thresholds govern the experiment. The **Service Level Agreement (SLA)** target is P95 HTTP latency $\leq 150\text{ ms}$ (`L_TARGET_MS = 150`); this is the signal the QoS controller reacts to in real time. The **Service Level Objective (SLO)** used for cross-condition evaluation is P99 $< 200\text{ ms}$ compliance rate; the fraction of time during a 15-minute run that P99 stays below 200 ms.

The controller target is deliberately more conservative than the SLO evaluation threshold. Under the static-limits condition the system reaches a steady-state P95 of $\approx 120\text{ ms}$ between phase transitions. Setting `L_TARGET_MS = 200 ms` would place the target above this equilibrium, causing the controller to remain idle during load ramps and only act after P99 had already breached the SLO. A target of 150 ms provides a 50 ms buffer, allowing throttling to begin before P99 crosses 200 ms.

The controller observes latency via the following PromQL query, polled every 5 seconds against the in-cluster Prometheus endpoint (the metrics middleware that produces this histogram is shown in Appendix C.3.2):

```

histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket{
    namespace="ticket-app",
    service!="payments-srv"
  }[60s])) by (le)
) * 1000

```

The payments service is excluded (`service!="payments-srv"`) because its hardcoded 500 ms mock delay permanently inflates the aggregate quantile above the 200 ms threshold regardless of CPU load, masking the true contention signal. The `rate()` window is 60 seconds. A 30-second window was tested first but produced excessive signal noise; latency quantiles computed over sparse 30-second windows fluctuate sharply between

scrape intervals, causing the controller to oscillate between `CPU_MIN` and `CPU_MAX` without converging.

3.3.4 Load Generation: WaveShape Profile

Load is generated using Locust with a custom `LoadTestShape` subclass (referred to as the *WaveShape* profile after the class name used in the implementation) that overrides any `--users` and `--spawn-rate` CLI flags (full script in Appendix C.4). The profile consists of five phases over 15 minutes, shown in Table 3.4 and Figure 3.4.

Phase	Duration	Virtual Users	Fraction of Peak
1	0–3 min	30	7.5 %
2	3–6 min	400	100 %
3	6–9 min	200	50 %
4	9–12 min	400	100 %
5	12–15 min	30	7.5 %

Table 3.4: WaveShape load profile phases.

The profile is intentionally asymmetric: Phases 1, 3, and 5 represent low-load windows during which the controller is expected to relax the Ray CPU limit toward `CPU_MAX`; Phases 2 and 4 are full-load spikes during which the controller must throttle Ray to protect the SLO.

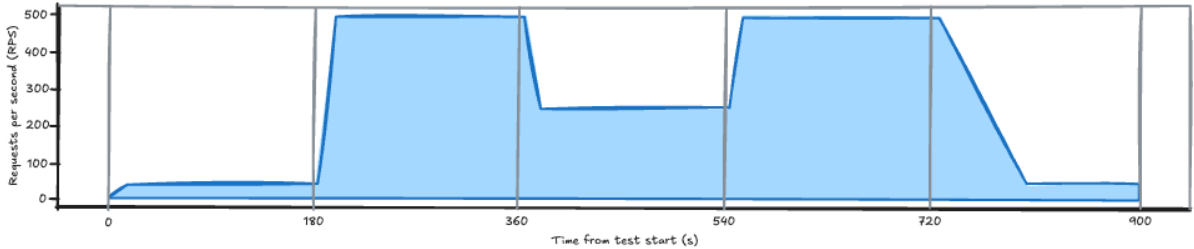


Figure 3.4: Observed requests-per-second (RPS) time series for the WaveShape load profile across a representative 15-minute run. Phase transitions at $t = 180$ s, 360 s, 540 s, and 720 s are clearly visible. Peak throughput of ≈ 500 RPS occurs during Phases 2 and 4.

Simulated user journey. Each virtual user registers once on startup via `POST/api/users/signup` with a unique email address. Subsequent iterations are drawn from two weighted tasks: a lightweight browse task (`GET /api/tickets`, weight 1) and a full purchase flow (weight 3) consisting of ticket creation, order submission, and payment (Figure 3.3). The purchase flow uses `gevent.sleep` delays between steps to respect Kafka consumer lag, as described in Section 3.3.2.

Bcrypt spike at phase transitions. At each upward phase transition, Locust spawns new virtual users at 60 users per second. Every new user immediately executes the signup task, which calls bcrypt password hashing in the auth service. Node.js delegates blocking crypto operations to its `libuv` thread pool (default pool size: 4 threads). With 60 concurrent signup requests, 56 queue behind 4 available threads, causing signup latencies of 1000–4000 ms for approximately 30–45 seconds until the pool drains. These spikes appear in the P99 time series of every experimental condition (including the no-contention-wla baseline) confirming they are an application-level artifact independent of Ray CPU contention.

3.4 Workload B: Distributed AI Training Job

Workload B is a throughput-heavy distributed deep-learning training job designed to saturate the CPU and GPU resources of the two worker nodes, thereby inducing the noisy-neighbor effect on Workload A. The job is deliberately engineered to be CPU-bottlenecked rather than GPU-bottlenecked: the QoS controller throttles CPU limits, so only a CPU-bound workload will respond to those changes with measurable throughput reduction. The preprocessing pipeline described below is the mechanism that achieves this.

The job trains a ResNet-50 model using PyTorch Distributed Data Parallel (DDP) coordinated by the Ray Train framework (`TorchTrainer`), with two workers placed on `gpu-node-1` (RTX 3090, 24 GB VRAM) and `gpu-node-2` (RTX 3080, 10 GB VRAM) respectively. After each backward pass, DDP synchronizes gradients across both workers via an NCCL AllReduce collective over the 2.5 Gbps cluster interconnect. The `TorchTrainer` is configured with `ScalingConfig(num_workers=2, use_gpu=True, resources_per_worker={"GPU": 1, "CPU": 8})`, allocating one GPU and eight CPU cores per worker.

Before each training step, a multi-threaded CPU pipeline preprocesses a batch of synthetic images through nine sequential operations:

1. Generate a 768×768 `uint8` random tensor per image, simulating a high-resolution raw sensor frame.
2. Random 640×640 crop (up to 128px random offset per axis), requiring a memory copy of each image.
3. Random horizontal flip.
4. Color jitter: brightness and contrast perturbation (uniform within each preprocessing chunk).
5. Random rotation $\pm 30^\circ$ via a torchvision affine transform; a computationally expensive per-image bilinear warp.

6. Gaussian blur with kernel size 21; wide convolution with high CPU cost.
7. Second Gaussian blur pass with kernel size 9, adding further convolution work.
8. Center-crop to 224×224 (offset 208) to produce standard ResNet-compatible input dimensions.
9. ImageNet normalization (channel mean $[0.485, 0.456, 0.406]$, std $[0.229, 0.224, 0.225]$).

Figure 3.5 illustrates the parallel structure of the pipeline and how it feeds the GPU training loop.

The pipeline processes the full batch in parallel using a `ThreadPoolExecutor` with `NUM_PREPROCESS_THREADS` (default: 12) worker threads. Each thread independently pre-processes a chunk of `batch_size / num_threads` images with a unique deterministic seed. Because the torchvision C extensions release the Python GIL during pixel-level operations, all 12 threads execute truly in parallel, saturating up to 12 CPU cores simultaneously.

To prevent GPU idle time while the CPU pipeline runs, a background daemon thread (`_prefetch_worker`) continuously fills a bounded queue (`maxsize=2`) with pre-processed batches. The training loop calls `queue.get()` at the start of each step, blocking only if the prefetch queue is empty. This producer-consumer structure arranges for preprocessing of step $t+1$ to overlap with the GPU forward pass, backward pass, and DDP AllReduce of step t .

The primary throughput metric for cross-condition comparison is `makespan`: the wall-clock duration from RayJob submission to completion, recorded by each experiment playbook in `makespan.txt` as the interval between the `rayjob_start` timestamp (captured immediately before `kubectl wait --for=Running`) and the `rayjob_end` timestamp (captured after `kubectl wait --for=Complete`). Because each evaluation run is preceded by a warm-up run that primes the Ray pip runtime cache (see Section 3.4.2), the torch wheel download does not inflate the makespan of the three evaluation runs.

3.4.1 CPU Resource Policy and In-Place Resize

Each Ray worker pod is configured with `requests.cpu: 8` across all conditions, reserving eight logical cores on the node for the Kubernetes scheduler. The `limits.cpu` value varies by experimental condition as shown in Table 3.5. Setting `requests.cpu: 8` ensures that the scheduler can place both workers on the two GPU nodes without over-committing CPU; the limit then controls the maximum CPU bandwidth the worker may actually consume.

Changing `limits.cpu` between conditions exploits the Kubernetes 1.34 in-place pod vertical scaling feature, which became generally available in this release. Rather than deleting and recreating worker pods (which would evict the ephemeral pip cache; see Section 3.2.4), each experiment playbook issues a `kubectl patch --subresource resize`

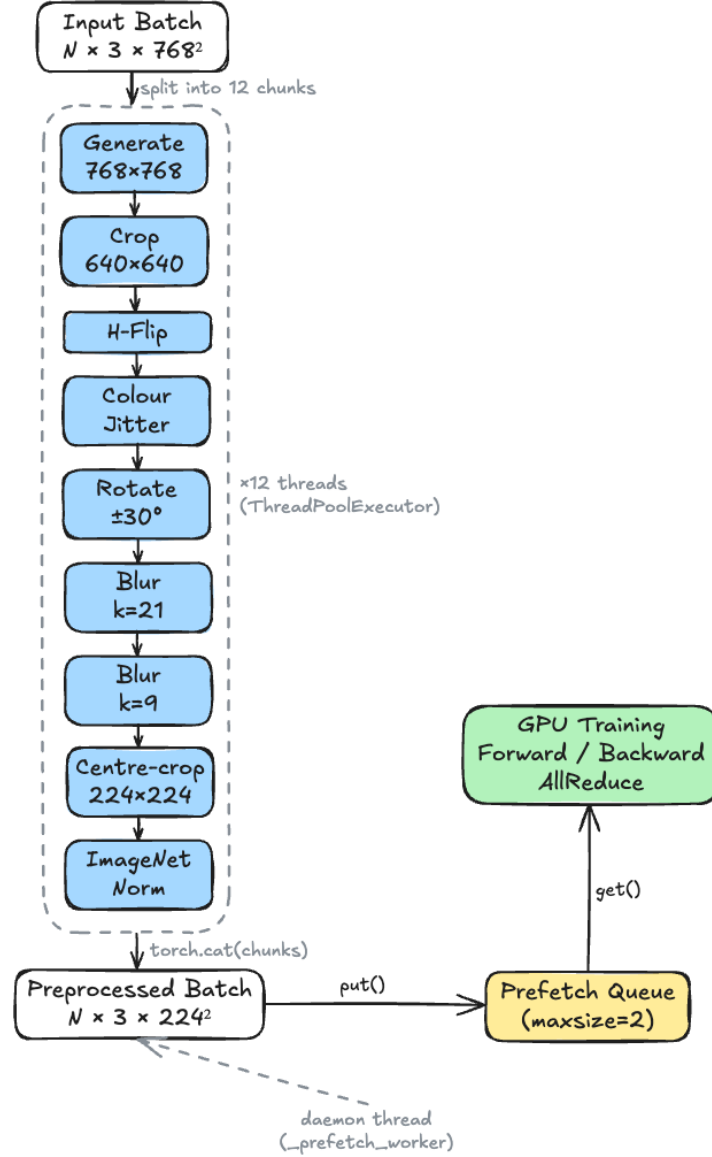


Figure 3.5: CPU preprocessing pipeline for Workload B. Each training step draws a pre-processed batch from the bounded queue (maxsize=2); the daemon thread continuously refills it using 12 parallel ThreadPoolExecutor workers. Because torchvision C extensions release the GIL, all 12 threads run truly in parallel, saturating up to 12 CPU cores. The queue allows preprocessing of step $t+1$ to overlap with GPU work of step t .

Condition	limits.cpu	Notes
no-contention-wlb	16	Workload B in isolation; no co-tenant
no-limits	16	Co-located; no throttling applied
static-limits	6	Fixed cap; no controller
qos-controller	8	Reset to 8 before test; PI controller adjusts dynamically within [2, 10]

Table 3.5: Ray worker CPU resource policy per experimental condition.

command against the already-running worker pods. The kubelet accepts the updated resource specification without restarting the container process; it propagates the new limit to the Linux cgroup subsystem, overwriting the `cpu.max` entry in the pod's `cpu` cgroup within a few CFS scheduling periods (typically under one second). The kernel CFS bandwidth controller then enforces the revised cap on all threads within the cgroup immediately.

The static-limits cap of six cores is chosen to create a measurably different throttle than the dynamic condition while still leaving the GPU preprocessing pipeline partially operational. At `limits.cpu: 6`, each worker is capped at 37.5% of the 16-thread node, providing headroom for the co-located ticket-app pods without stranding the GPU workers entirely. Throughput decreases proportionally to the reduced CPU bandwidth, since preprocessing time is the bottleneck.

3.4.2 Script Delivery and Reproducibility

The training script (`workload-b/ray-job.py`) is not embedded in a container image. Instead, each experiment playbook copies it directly into the running Ray head pod before submitting the job:

```
HEAD_POD=$(kubectl get pods -n ray-system -l ray.io/node-type=head \
-o jsonpath='{.items[0].metadata.name}')
kubectl cp workload-b/ray-job.py ray-system/${HEAD_POD}:/tmp/ray-job.py
kubectl apply -f kubernetes/workload-b/rayjob.yml
```

The `RayJob` manifest specifies `entrypoint: python /tmp/ray-job.py`, so the submitted job always runs the freshly copied file. This delivery pattern decouples code changes from image releases: modifying the training logic requires only a file copy and a job resubmit, not a Docker build, registry push, or Kubernetes image pull. The four primary tunable parameters (`NUM_EPOCHS`, `STEPS_PER_EPOCH`, `BATCH_SIZE`, `NUM_PREPROCESS_THREADS`) are passed as environment variables in the `runtimeEnvYAML` field of the `RayJob` manifest, making parameter changes equally lightweight; `SEED` is not set explicitly in the manifest and defaults to 42 in the script (full script in Appendix D.1).

All stochastic operations in the preprocessing pipeline (tensor generation, crop offsets, flip decisions, color jitter magnitudes, and rotation angles) are seeded through a `torch.Generator` initialized with a deterministic offset derived from a root `SEED` environment variable (default: 42). Each thread and each training step use a unique seed derived from this root, ensuring that the preprocessing workload is identical across all three repetitions of each condition.

Because the pip runtime cache is ephemeral and cleared on pod restart (see Section 3.2.4), one warm-up run must be submitted and discarded after any cluster restart before the three evaluation runs begin. The procedure is: redeploy the `RayCluster` via

playbook 07c-kuberay-operator.yml, run one warm-up 08b isolation run (discarded), then proceed with the three evaluation runs for each condition.

3.5 Dynamic QoS Controller

The Dynamic QoS Controller is a Python daemon deployed as a single-replica `Deployment` in the `qos-system` namespace. It implements a proportional-integral (PI) feedback loop that executes every $T = 5$ s: observing the current P95 HTTP latency of Workload A from Prometheus, computing a signed error against a 150 ms target, and adjusting the CPU resource limits of the co-located Ray worker pods accordingly. All tunable parameters (gains, target latency, actuator bounds, and timing constants) are injected via environment variables from a Kubernetes `ConfigMap`, decoupling policy tuning from container image releases. The controller additionally exposes its internal signals as a Prometheus metrics endpoint on port 8080, scraped by the cluster’s `kube-prometheus-stack` via a `ServiceMonitor`; this enables real-time dashboarding in Grafana and post-hoc analysis of control-loop behavior alongside the infrastructure and application metrics collected during each experiment run.

3.5.1 Control Loop Architecture

The control loop instantiates the Observe–Analyze–Act pattern introduced in Section 2.9.1; the three phases below document the concrete implementation of each step, illustrated in Figure 3.6.

Observe. The controller queries the in-cluster Prometheus endpoint every $T = 5$ s using the PromQL expression introduced in Section 3.3.3. This query returns the 95th-percentile HTTP response time (in milliseconds) across all ticket-app microservices except payments, averaged over a 60-second sliding window. If Prometheus returns no data or a NaN value (for example, during cluster startup before the first scrape window completes), the cycle is skipped without modifying the integral accumulator.

Analyze. Given the observed latency L_k , the controller computes the signed error $e_k = L_k - L_{\text{target}}$, conditionally updates the integral accumulator (Section 3.5.2), and derives the control output $u_k = K_p e_k + K_i I_k$ in units of CPU cores (gain values and selection rationale in Table 3.6). A positive u_k indicates the system is above target and Ray CPU capacity should be reduced; a negative u_k indicates available headroom and capacity can be restored. If $|e_k| < \delta_{\text{db}} = 10$ ms the cycle exits without issuing any patch, preventing churn within the natural noise floor of the P95 histogram.

Act. If the deadband condition is exceeded and the per-patch cooldown $\tau_c = 10$ s has elapsed since the previous actuation, the controller lists all running Ray worker pods by label selector `ray.io/node-type=worker` and computes a new CPU limit for each:

$$C_{\text{new}} = \text{clamp}(\text{round}(C_{\text{current}} - u_k), C_{\text{min}}, C_{\text{max}}) \quad (3.1)$$

A pod is patched only if its computed limit differs from the current value. Each patch is issued against the Kubernetes resize subresource (Section 3.5.3), and the `qos_patches_total` counter is incremented for each successful actuation.

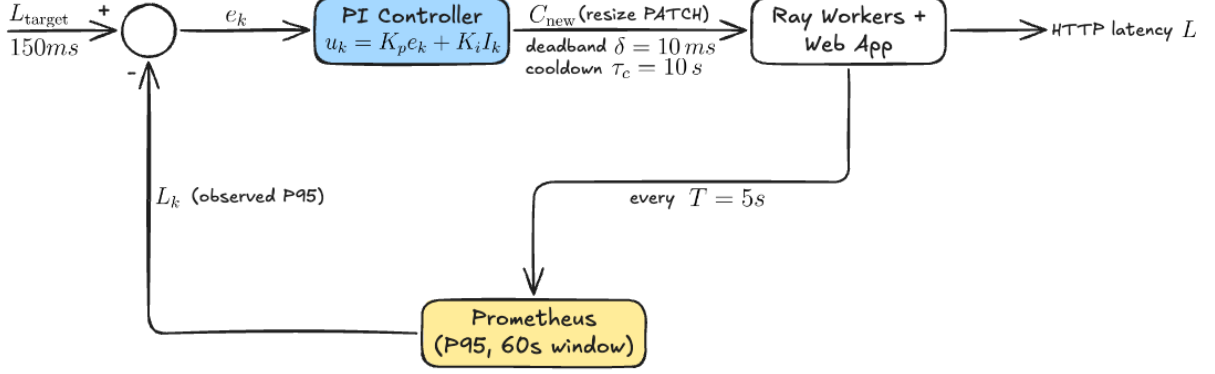


Figure 3.6: Closed-loop block diagram of the Dynamic QoS Controller. Every 5 s the PI controller computes an error $e_k = L_k - L_{\text{target}}$ from the Prometheus-observed P95 latency and adjusts the Ray worker CPU limits via the Kubernetes resize subresource. A deadband of ± 10 ms and a per-patch cooldown of 10 s suppress actuation under noisy or rapidly changing conditions.

3.5.2 PI Control Law and Parameter Selection

The controller implements a discrete-time proportional-integral law at a fixed sampling period $T = 5$ s. The error signal, integral update, and control output at step k are:

$$e_k = L_k - L_{\text{target}} \quad (3.2)$$

$$I_k = I_{k-1} + e_k \cdot T \quad (3.3)$$

$$u_k = K_p e_k + K_i I_k \quad (3.4)$$

where L_k is the P95 latency (ms) observed at step k , $L_{\text{target}} = 150$ ms, and u_k is the signed CPU-core adjustment applied to each worker via Equation (3.1). A positive u_k reduces the CPU limit (throttle); a negative u_k increases it (restore capacity).

Anti-Windup

Two complementary mechanisms prevent integral saturation. First, the integral is clamped symmetrically to a windup limit:

$$I_{\max} = \min\left(\frac{C_{\max} - C_{\min}}{K_i}, 2000\right) = 2000 \text{ ms} \cdot \text{s} \quad (3.5)$$

This cap ensures the accumulated integral can never drive the actuator more than one full range beyond its saturation boundary. Second, *conditional integration* is applied: when the actuator was saturated at $C_{\min}$ or $C_{\max}$ during the preceding cycle, the integral term is frozen for the current step. This prevents the transient P95 spikes caused by PBKDF2 computations at phase transitions (where P95 briefly reaches ≈ 1500 ms for 30–45 s) from accumulating a large positive integral that would keep Ray throttled long after the spike subsides.

Parameter Selection

Table 3.6 lists all tunable parameters with their chosen values and the rationale derived from iterative tuning across multiple test runs.

Parameter	Value	Rationale
L_{target}	150 ms	Static-limits steady state is ≈ 120 ms P95; using the 200 ms SLO bound left too much headroom and prevented early throttling at Phase 2 onset
K_p	0.02	A 100 ms error produces a 2-core adjustment; $K_p = 0.03$ caused over-correction on moderate (20–50 ms) errors, inducing oscillation
K_i	0.0002	Integral windup limit resolves to 2000 ms·s; chosen so integral contribution remains bounded during sustained violations without dominating the proportional response
$C_{\min}$	2 cores	Keeps the GPU preprocessing pipeline minimally active; below 2 cores, torchvision thread starvation causes GPU utilization to drop to near zero
$C_{\max}$	10 cores	At 12 cores, Phase 1 (30-user) P95 occasionally exceeded the 150 ms target due to application-level CPU demand alone, causing spurious throttling before any Ray contention was present
δ_{db} (deadband)	10 ms	P95 exhibits ± 20 –30 ms natural noise on the 60 s rate window; without a deadband, patches fire every cycle under steady-state conditions
τ_c (cooldown)	10 s	Two control periods allow the CFS scheduler to stabilise after a cgroup resize before the next P95 observation reflects the updated limit
T (period)	5 s	Balances responsiveness (detect Phase 2 spike within 5–10 s of onset) against Prometheus scrape overhead and API server load

Table 3.6: QoS controller tunable parameters, values, and selection rationale.

Oscillation Detection

After each successful patch, the controller computes the fractional change in CPU limit relative to the previous patched value: $\Delta_{\text{frac}} = |C_{\text{new}} - C_{\text{prev}}| / C_{\text{prev}}$. If $\Delta_{\text{frac}} > 0.30$ (30 %),

the `qos_oscillations_total` counter is incremented and a warning entry is emitted to the structured JSON log. This does not suppress the patch; it serves as a diagnostic signal visible in Prometheus and Grafana for post-hoc evaluation of controller stability across experiment conditions.

3.5.3 Kubernetes Resize Subresource Actuation

The Kubernetes in-place resize subresource is too recent to appear in the auto-generated Python client library; calling `client.patch_namespaced_pod()` would modify the pod spec but would not issue the resize PATCH the kubelet needs to propagate the change to the cgroup. The controller therefore bypasses the generated client and issues a raw HTTPS PATCH directly using the `requests` library, authenticating with the service-account credentials mounted automatically inside every Kubernetes pod.

Changing a running pod’s CPU limit without restarting it requires issuing an HTTP PATCH against the `/resize` subresource endpoint available as a Beta feature in Kubernetes 1.34 [32]. The official `kubernetes` Python client library is generated automatically from the Kubernetes OpenAPI specification; at the time of development the generated client did not expose this subresource as a typed method.

Inside a Kubernetes pod, service account credentials are mounted automatically at `/var/run/secrets/kubernetes.io/serviceaccount/`. The controller reads the bearer token from `token` and the cluster CA certificate from `ca.crt` at each patch call, constructing an authenticated HTTPS request to the in-cluster API server at `https://kubernetes.default.svc`:

```
PATCH /api/v1/namespaces/ray-system/pods/<pod-name>/resize
Content-Type: application/strategic-merge-patch+json
```

```
{"spec": {"containers": [{"name": "ray-worker",
  "resources": {"limits": {"cpu": "<N>"}, "requests": {"cpu": "<N>"}}]}}
```

Both `limits.cpu` and `requests.cpu` are set to the same integer value to keep the pod in the Kubernetes *Guaranteed* QoS class; patching only `limits` while leaving `requests` unchanged would demote the pod to *Burstable* and alter its scheduling priority.

RBAC. The controller’s `ServiceAccount` requires a dedicated `ClusterRole` rule for the resize subresource. The Kubernetes RBAC engine treats `pods/resize` as a distinct resource from `pods`: a rule granting `patch` on `pods` does not cover `pods/resize`, and omitting the separate rule causes a 403 `Forbidden` response. The deployed `ClusterRole` therefore contains two independent rules (`get/list` on `pods` and `patch` on `pods/resize`) each in its own entry.

Cgroup propagation. When the API server accepts the resize request, the kubelet on the target node receives the updated resource specification via its watch loop. It writes

the new CPU quota to the pod’s `cpu.max` entry in the Linux cgroup v2 hierarchy under `/sys/fs/cgroup/kubepods/.../<pod-uid>/cpu.max`. The kernel’s CFS bandwidth controller reads this value at the next scheduling period boundary and enforces the revised cap on all threads in the cgroup, typically within a few milliseconds. No container process restart occurs at any point in this path [32].

3.5.4 Prometheus Metrics Exported

The controller starts a `prometheus_client` HTTP server on port 8080 at startup. A `ServiceMonitor` in the `qos-system` namespace, labeled `release: telemetry` so the `kube-prometheus-stack` operator discovers it, scrapes this endpoint at a 5s interval, matching the control loop period so that every cycle’s output is captured. Table 3.7 lists all six exported metrics.

Metric	Type	Description
<code>qos_p95_latency_ms</code>	Gauge	P95 HTTP latency (ms) returned by the last successful Prometheus query; the primary control signal
<code>qos_error_ms</code>	Gauge	Signed error $e_k = L_k - L_{\text{target}}$; positive when above the 150 ms target, negative when below
<code>qos_integral</code>	Gauge	Current value of the PI integral accumulator I_k ; useful for diagnosing windup behavior at phase transitions
<code>qos_cpu_limit_cores{pod}</code>	Gauge	CPU limit (cores) most recently applied to each Ray worker pod, labelled by pod name; one time series per worker
<code>qos_patches_total</code>	Counter	Cumulative number of successful resize patches issued since controller startup
<code>qos_oscillations_total</code>	Counter	Cumulative number of patches whose CPU limit change exceeded the 30 % oscillation threshold

Table 3.7: Prometheus metrics exported by the QoS controller on port 8080.

After each experiment run, `scripts/export-metrics.py` queries Prometheus retroactively over the run’s time window and writes three QoS diagnostic metrics (`qos_cpu_limit_cores`, `qos_error_ms`, and `qos_p95_observed_ms`) to CSV files in the run’s results directory alongside the application and infrastructure metrics, enabling unified offline analysis in the results notebook.

3.6 Experimental Design

To answer the research question stated in Section 1.2, the evaluation uses a **5-condition** $\times$ **3-repetition** factorial structure. Each condition deploys a fixed configuration of the two workloads and runs an independent 15-minute Locust WaveShape load test; the three repetitions of each condition provide estimates of run-to-run variance. Every run

begins with a full namespace teardown and redeployment so that no residual state (cached MongoDB documents, Kafka offsets, Ray job metadata) carries over between runs. The five conditions are listed in Table 3.8, with prose descriptions of the Ansible playbooks in Appendix E.

Condition	Playbook	Workloads	Controller	Purpose
no-contention-wla	08a	Workload A only	—	Latency baseline: Workload A without any co-tenant
no-contention-wlb	08b	Workload B only	—	Throughput baseline: Workload B without any co-tenant
no-limits	09	A + B (<code>limits.cpu: 16</code>)	—	Worst-case co-location with no CPU throttling
static-limits	10	A + B (<code>limits.cpu: 6</code>)	—	Fixed-cap reference: constant 6-core ceiling
qos-controller	11	A + B (start: 8, dynamic)	PI	Feedback-driven throttling via the PI controller

Table 3.8: Five experimental conditions, their Ansible playbooks, workload configuration, and purpose.

Hypotheses

Four falsifiable hypotheses guide the evaluation:

- H1a** Co-location without CPU limits (*no-limits*) causes a statistically significant increase in mean P99 latency compared to the isolated Workload A baseline (*no-contention-wla*), confirming that the noisy-neighbor effect is observable on this hardware.
- H1b** Applying static CPU limits (*static-limits*) reduces P99 latency relative to *no-limits*, demonstrating that even a fixed resource cap provides measurable SLO protection.
- H2** The QoS controller achieves P99 SLO compliance at least as high as *static-limits*, confirming that dynamic throttling does not degrade web-application protection relative to the fixed-cap baseline.
- H3** The QoS controller achieves a shorter Ray training time (makespan) than *static-limits* by restoring CPU capacity to the Ray workers during low-load WaveShape phases when the web application has ample headroom.

3.6.1 Metrics Collected and Export Methodology

After each run, `scripts/export-metrics.py` is executed on `cpu-node-1` against the in-cluster Prometheus endpoint (full script in Appendix E.1). All time-series queries

use a 15s resolution step and a 60s sliding rate window (`rate(...)[60s]`), with the exception of `ticket_app_cpu_usage_by_node` which uses a 15s window. The script accepts a `--condition` flag and exports only the subset of metrics relevant to that condition, writing one CSV file per metric to the run’s results directory. Table 3.9 lists all exported metrics grouped by category.

Category	CSV file(s)	Source	Conditions
Web app latency	<code>p95_latency_ms</code> , <code>p99_latency_ms</code>	prom-client histogram (payments excl.)	wla, no-limits, static, dynamic
Ray CPU usage	<code>ray_worker_cpu_usage_cores</code> , <code>ray_worker_cpu_usage_by_node</code>	cAdvisor container metrics	wlb, no-limits, static, dynamic
Ray CPU limit	<code>ray_worker_cpu_limit_cores</code>	<code>kube_pod_container_resource_limits</code>	static, dynamic
Node CPU utilization	<code>node_cpu_util_pct_gpu1</code> , <code>node_cpu_util_pct_gpu2</code>	<code>node_cpu_seconds_total</code>	all
Ticket App CPU	<code>ticket_app_cpu_usage_by_node</code>	cAdvisor (<code>ticket-app</code> ns)	wla, no-limits, static, dynamic
GPU utilization	<code>gpu_util_pct</code> , <code>gpu_mem_used_mib</code>	DCGM exporter (NVIDIA GPU Operator)	all
GPU memory copy util	<code>gpu_mem_copy_util_pct</code>	DCGM exporter (NVIDIA GPU Operator)	wlb, no-limits, static, dynamic
QoS diagnostics	<code>qos_cpu_limit_cores</code> , <code>qos_error_ms</code> , <code>qos_p95_observed_ms</code>	Controller <code>/metrics</code> endpoint	dynamic only

Table 3.9: Prometheus metrics exported per run, grouped by category.

Locust output. The Locust process writes two CSV files to the run directory: `locust_stats.csv` (per-endpoint aggregate request count, failure count, mean/P50/P95/P99 latency, and mean RPS over the full 15-minute run) and `locust_stats_history.csv` (a time-series sampled at 10s intervals, recording instantaneous RPS and latency quantiles for each endpoint). These files are used for cross-condition reliability comparison.

Makespan caveat. The `makespan.txt` file records wall-clock timestamps written by Ansible: `rayjob_start` (captured immediately before `kubectl wait --for=Running`; for the *qos-controller* condition this occurs after the QoS controller pod reports `Ready`, introducing a systematic offset of approximately 5–15s relative to actual job submission) and `rayjob_end` (recorded after Locust exits). For the *no-limits* condition, the Ray job completes before the 15-minute Locust run ends; however, `rayjob_end` is recorded only after `kubectl wait` returns in the post-Locust Ansible play, so makespan reflects Locust-exit time rather than actual training completion, inflating the figure by the remaining Locust run time.

3.6.2 Statistical Approach

Each of the five conditions is repeated three times with independent deployments. The three repetitions yield a mean and standard deviation for every scalar outcome metric. Cross-condition comparisons in `results/tables/comparison.csv` are based on these means; the standard deviation quantifies run-to-run variability and is reported alongside each mean. The key scalar columns used for hypothesis evaluation are:

- `makespan_s` — primary AI throughput metric (seconds); lower is better for Workload B; inflated for the no-limits condition (see Section 4.2)
- `mean_p99_ms` — mean P99 latency across all 15 s windows in the run
- `slo_compliance_pct` — fraction of 15 s windows in which $P99 \leq 200$ ms; primary SLO metric
- `slo_violation_total_s` — total seconds during which P99 exceeded 200 ms

Known confounds. Two systematic effects inflate SLO violation metrics across all conditions and are acknowledged rather than corrected. First, the `bcrypt` phase-transition spike (Section 3.3.4) inflates `slo_violation_total_s` symmetrically across all conditions; this effect is application-level and independent of Ray co-location. Second, for the *no-limits* condition only, `makespan_s` is inflated by the Ansible timing artefact described above; it is reported with a dagger annotation in the cross-condition table (Table 4.1) and should not be compared directly to other conditions.

Chapter 4

Results

4.1 Isolation Baselines

4.1.1 Workload A Alone (no-contention-wla)

Figure 4.1 shows the P95/P99 latency, node CPU utilization, and Locust request rate for run 1 (runs 2 and 3 are reproduced in Section F.3 and are nearly identical).

Two transient P99 spikes are visible in every run: one at $t \approx 180$ s (Phase 1→2 ramp-up, 30→400 virtual users) and a second at $t \approx 540$ s (Phase 3→4, 200→400 users). Each spike lasts 30–45 s, driven by the `bcrypt` password-hashing handler (see Section 3.3) processing the burst of new user registrations when load escalates. Peak P99 during the spikes reaches 1 500–3 500 ms; outside the spikes the steady-state P99 falls to ≈ 49 ms. Node CPU utilization tracks the WaveShape profile, rising from $\approx 3\%$ at 30 users to $\approx 15\%$ at 400 users. GPU metrics are absent because no Ray workload is running.

Aggregated over three runs, the mean P95 latency is 89 ± 15 ms, the mean P99 is 273 ± 27 ms, and SLO compliance is $89.9 \pm 0.3\%$. Each run records exactly two SLO violations with a combined duration of 90 s, corresponding precisely to the two phase-transition spikes. These results establish the best-case latency floor: even without any co-tenant the application cannot achieve perfect SLO compliance, setting an inherent ceiling for all co-location conditions.

4.1.2 Workload B Alone (no-contention-wlb)

Figure 4.2 shows Ray worker CPU usage, node CPU utilization, and GPU utilization for run 1 (runs 2 and 3 are in Section F.3). The training job ramps up within the first 30 s and then runs at steady throughput for the remainder of the experiment. The GPU utilization panel reveals the CPU-bound preprocessing pattern: `gpu-node-1` (RTX 3090) oscillates between 0 % and 100 % as the prefetch queue alternately fills and empties, while `gpu-node-2` (RTX 3080) sustains near-100 % utilization throughout. The asymmetry

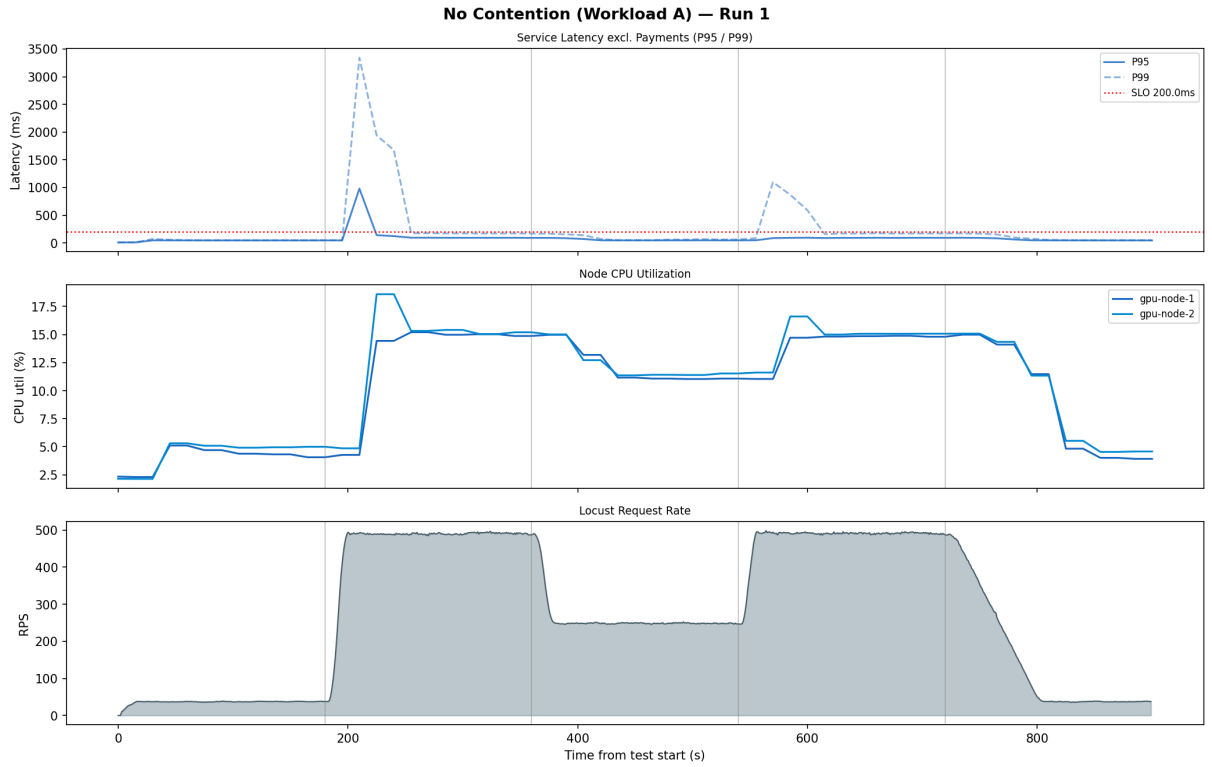


Figure 4.1: No-contention Workload A baseline — run 1 (representative). Panels show P95/P99 latency excluding payments (top), node CPU utilization (middle), and Locust request rate (bottom). Two transient P99 spikes at phase transitions ($t \approx 180$ s and $t \approx 540$ s); steady-state P99 between spikes is ≈ 49 ms. Runs 2 and 3 in Section F.3.

arises because the RTX 3090 completes each forward/backward pass faster than the CPU preprocessing pipeline can refill the next batch; the queue drains and the GPU stalls until preprocessing catches up. The slower RTX 3080 on **gpu-node-2** better matches preprocessing throughput, keeping the queue full and GPU utilization steady. Node CPU utilization reflects this asymmetry: **gpu-node-1** operates at $\approx 90\%$ while **gpu-node-2** stabilizes at $\approx 40\%$. Latency metrics are absent because no Ticket App is running.

Across three runs the mean makespan is 888 ± 2 s with mean node CPU utilization of $65.8 \pm 0.3\%$ and GPU utilization of $80.3 \pm 1.0\%$. These figures establish the unconstrained throughput ceiling: the shortest achievable Ray job duration and the highest attainable hardware utilization before any QoS intervention.

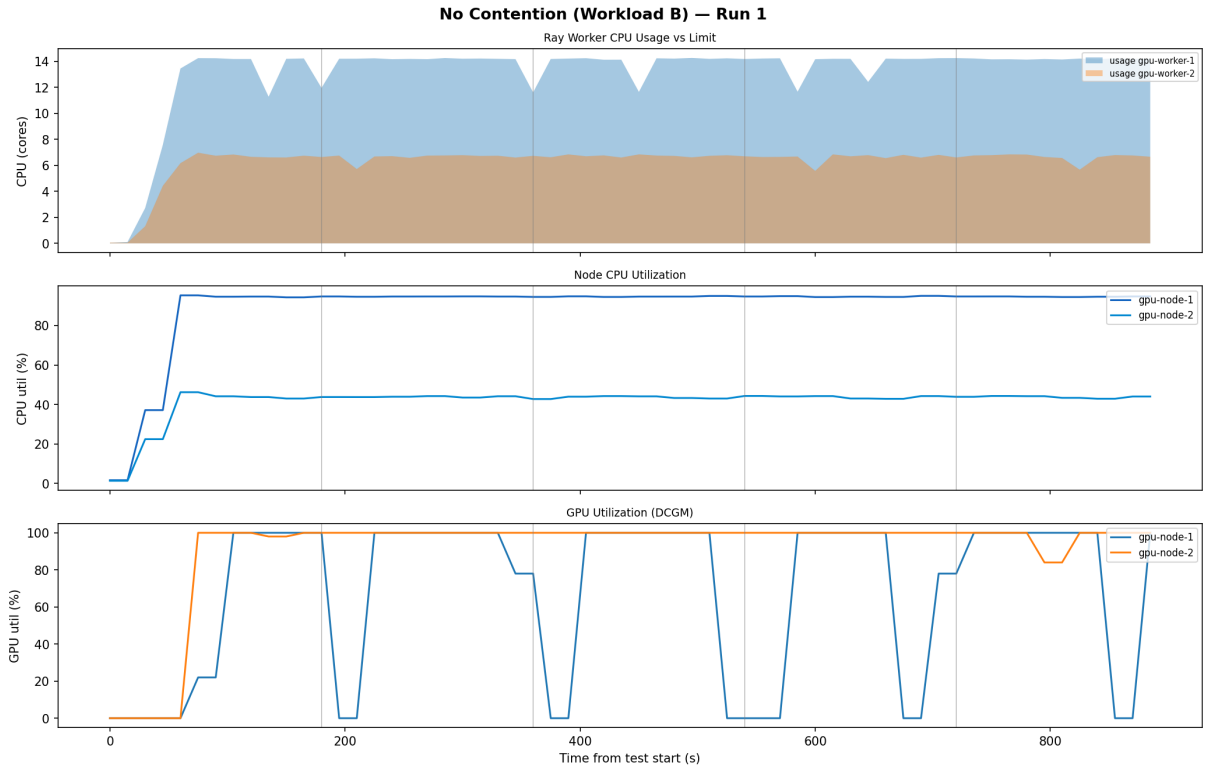


Figure 4.2: No-contention Workload B baseline — run 1 (representative). Panels show Ray worker CPU usage vs. applied limit (top), node CPU utilization (middle), and GPU utilization from DCGM (bottom). The oscillating GPU utilization on **gpu-node-1** reflects the CPU-bound preprocessing pipeline periodically stalling the prefetch queue. Runs 2 and 3 in Section F.3.

4.2 Co-location without CPU Limits (no-limits)

Figure 4.3 shows run 1 (runs 2 and 3 are in Section F.3). With Ray workers unrestricted at `limits.cpu: 16`, the two workloads compete directly for shared CPUs on **gpu-node-1** and **gpu-node-2**, producing severe and sustained latency degradation.

Latency. P95 and P99 are elevated from the very start of the run: with Ray workers saturating both worker nodes within the first 30 s (consistent with the ramp-up observed in Section 4.1.2), latency violations begin at $t \approx 30\text{--}60\text{ s}$ during Phase 1 at just 30 virtual users (the only condition in this study where violations occur below peak load). The mean P99 across runs is $906 \pm 69\text{ ms}$ (more than four times the 200 ms SLO boundary) and SLO compliance is $12.0 \pm 0.9\%$. Violations persist near-continuously through all phases; even at 200 users (Phase 3) the reduced web load is insufficient to overcome the CPU starvation imposed by unconstrained Ray workers. Each run accumulates $805 \pm 9\text{ s}$ of SLO violation (89 % of the 900 s test window), with a single brief 15 s transient recovery observed in Phase 5 (P99 drops to $\approx 193\text{ ms}$) as virtual users taper back to 30 and CPU contention momentarily subsides.

Makespan caveat. The reported makespan of $1040 \pm 1\text{ s}$ is inflated relative to actual training duration. The no-limits job completes before the 15-minute Locust run ends; however, Ansible records `rayjob.end` only after `kubect1 wait` returns in the post-Locust play, so makespan reflects Locust-exit time rather than job-completion time, inflating the figure by the remaining Locust run time.

Client throughput. Server-side latency under contention reduces client throughput: mean Locust RPS falls to 171 ± 7 versus 278 ± 1 for the no-contention Workload A baseline (-38%), and total requests drop to $154\,068 \pm 6\,139$ versus $249\,969 \pm 454$. The request failure rate rises to $40.5 \pm 0.6\%$ (from $5.1 \pm 0.5\%$ in isolation) as slow responses accumulate beyond Locust’s connection timeout.

4.3 Co-location with Static CPU Limits (static-limits)

Figure 4.4 shows run 1 (runs 2 and 3 are in Section F.3). The flat 6-core limit line is visible throughout the Ray worker CPU panel, holding constant regardless of web-traffic phase.

Latency. The SLO violation pattern mirrors the no-contention Workload A baseline: violations are concentrated at the two phase-transition spikes ($t \approx 180\text{ s}$ and $t \approx 540\text{ s}$) caused by `bcrypt` signup bursts, not by Ray CPU contention. Between transitions the steady-state P99 sits well below 200 ms, confirming that the 6-core cap is sufficient to prevent Ray from crowding out the web application during sustained load. Mean P99 is $385 \pm 33\text{ ms}$ and SLO compliance is $49.1 \pm 3.5\%$ across three runs, with a violation total of $455 \pm 17\text{ s}$.

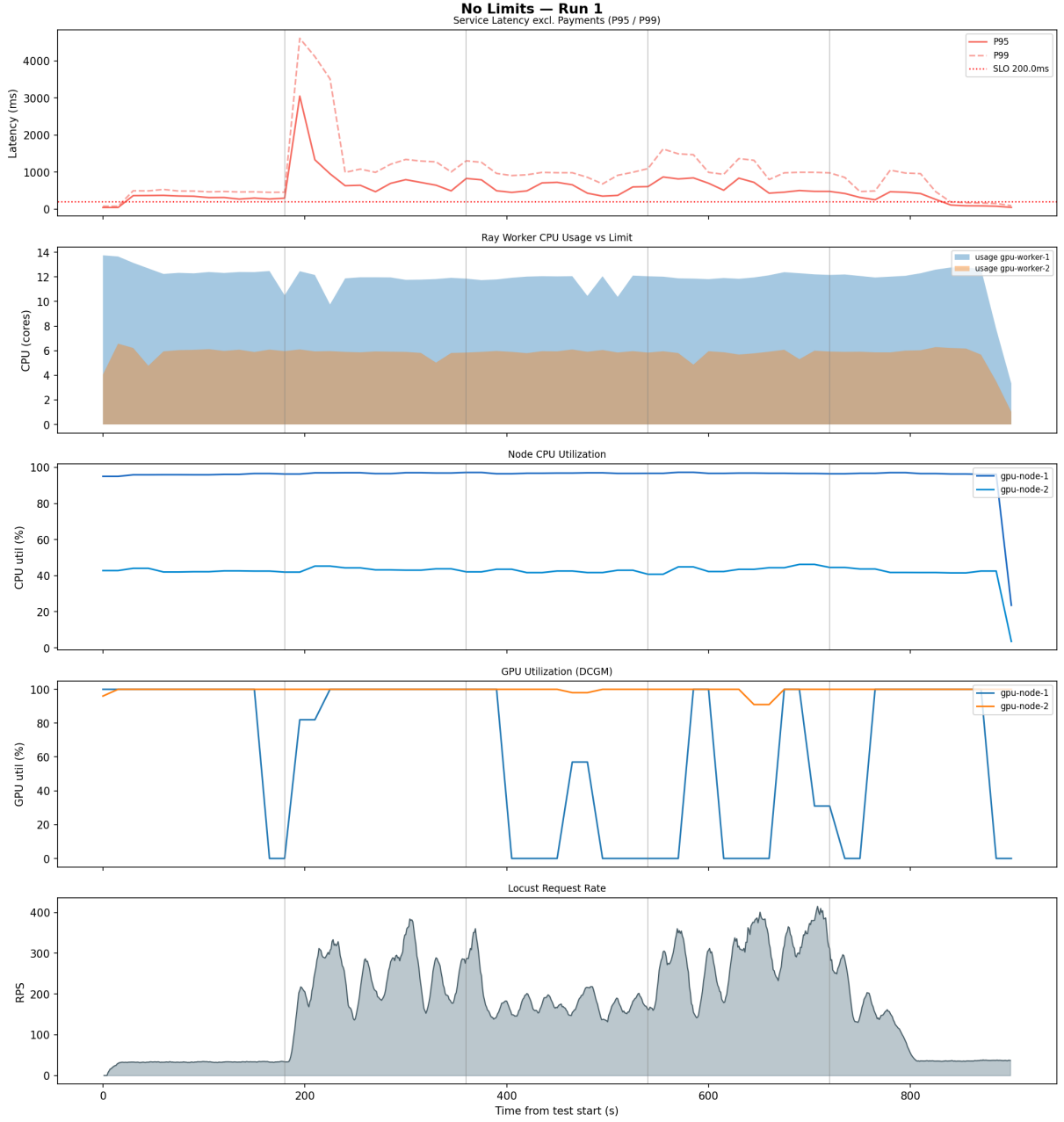


Figure 4.3: Co-location without CPU limits — run 1 (representative). Panels show P95/P99 latency excluding payments (top), Ray worker CPU usage vs. applied limit (second), node CPU utilization (third), GPU utilization from DCGM (fourth), and Locust request rate (bottom). P99 exceeds 200 ms from $t \approx 30$ s through the end of the run ($\approx 89\%$ of the test window), with a single 15s transient recovery in Phase 5; GPU utilization remains high throughout as Ray continues training despite severe latency violations. Runs 2 and 3 in Section F.3.

Idle CPU headroom. The Ray worker CPU usage panel reveals a per-worker asymmetry consistent with the Workload B baseline (Section 4.1.2): Worker 1 (`gpu-node-1`, RTX 3090) saturates the 6-core cap throughout all phases, while Worker 2 (`gpu-node-2`, RTX 3080) operates at ≈ 3.6 cores (60 % of its allocation) as the faster GPU drives greater preprocessing CPU demand. The wasted headroom is visible at the cluster level: mean node CPU utilization is 41.6 ± 0.3 %, substantially below the no-limits condition (66.5 %) and the unconstrained Workload B baseline (65.8 %). The static cap prevents Ray from reclaiming this surplus capacity during the low-traffic windows that constitute roughly 60 % of the test.

Throughput. With workers capped at 6 cores throughout, mean makespan is 1890 ± 4 s; more than double the 888 s unconstrained baseline (+113 %). Mean Locust RPS is 269 ± 4 , close to the no-contention Workload A baseline (278 ± 1), confirming that the static cap protects web application throughput at the cost of AI job completion time.

4.4 Co-location with Dynamic QoS Controller (qos-controller)

Figure 4.5 shows run 1 (runs 2 and 3 are in Section F.3). The bottom panel of each plot shows the QoS controller error signal and the applied CPU limit over time, making the inverse correlation between observed latency and allocated cores directly visible.

Per-phase controller behavior. *Phase 1* (30 users, $t \approx 0$ –180 s): observed P95 is 40–110 ms, well below the 150 ms target; the controller holds the limit at $C_{\max} = 10$ cores, allowing Ray workers full access to available CPU.

Phase 2 onset ($t \approx 180$ s): within the first control intervals of Phase 2, the bcrypt signup burst drives observed P95 above 900 ms; the error signal exceeds the deadband (see Section 3.5 for parameter definitions) by several hundred milliseconds and the controller clamps the limit to $C_{\min} = 2$ in a single step. The limit remains at 2 cores for roughly 225 s while latency gradually subsides.

Phase 3 ($t \approx 360$ s): with load reduced to 200 users, the controller begins releasing capacity at $t \approx 420$ s: $2 \rightarrow 4 \rightarrow 8 \rightarrow 9$ cores over three control intervals. A transient latency spike at $t \approx 480$ s causes a brief re-clamp to $C_{\min} = 2$ before a second recovery ($2 \rightarrow 3 \rightarrow 6 \rightarrow 7$) carries the limit to 7–9 cores by Phase 3 end.

Phase 4 ($t \approx 540$ s): as load ramps back to 400 users, the controller oscillates between 7–9 cores before clamping to $C_{\min} = 2$ at $t \approx 585$ s when P95 spikes. Recovery proceeds more quickly than after Phase 2 ($2 \rightarrow 5 \rightarrow 7 \rightarrow 10$ in approximately 1 min) as the PI integral accumulator has partially wound down (see Section 3.5.2); the limit reaches

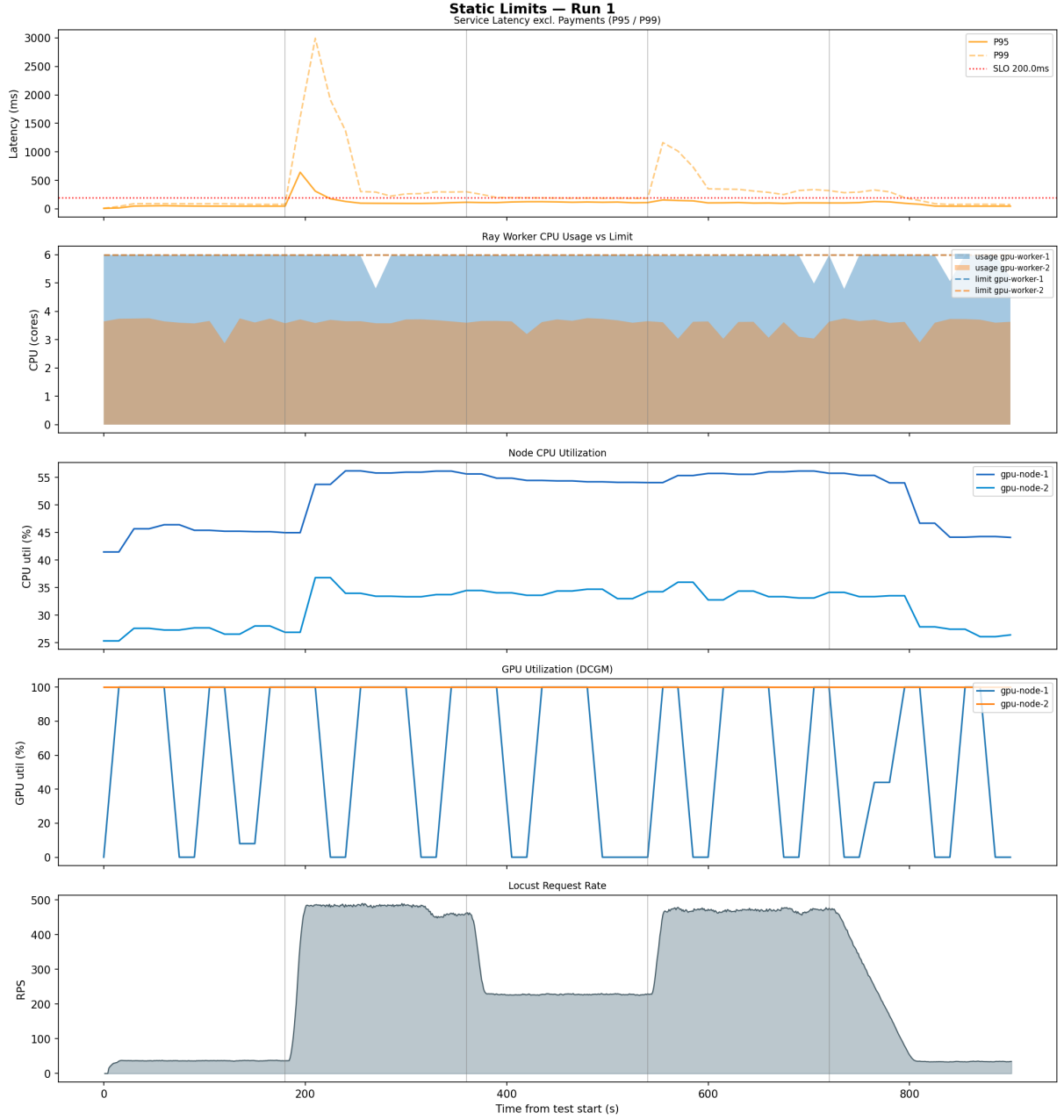


Figure 4.4: Co-location with static CPU limits — run 1 (representative). Panels show P95/P99 latency excluding payments (top), Ray worker CPU usage vs. applied 6-core limit (second), node CPU utilization (third), GPU utilization from DCGM (fourth), and Locust request rate (bottom). The flat limit line prevents Ray from using idle CPU headroom during low-traffic phases. Runs 2 and 3 in Section F.3.

$C_{\max} = 10$ before Phase 5 begins.

Phase 5 ($t \approx 720$ s): load tapers back to 30 virtual users; the limit is already at $C_{\max} = 10$ (having recovered during Phase 4) and remains there for the final 3 min of the test.

Across run 1, approximately 16 distinct limit adjustments were applied per worker. The `qos_patches_total` and `qos_oscillations_total` counter metrics were not included in the post-run CSV export and are therefore unavailable for quantitative comparison.

CPU reclamation and throughput. The controller’s ability to release capacity during Phases 1, 3, and 5 raises mean node CPU utilization to $50.6 \pm 4.3\%$; +9 percentage points above the static-limits condition (41.6%). Mean makespan is 1351 ± 89 s, a 29 % reduction relative to static limits (1890 s), confirming that CPU reclamation during idle web-traffic windows meaningfully shortens Ray job completion time.

SLO compliance. Mean P99 is 468 ± 87 ms and SLO compliance is $52.5 \pm 1.7\%$, with a violation total of 430 ± 17 s. Compliance is modestly higher than static limits (49.1%) with substantially lower run-to-run variance ($\pm 1.7\%$ vs. $\pm 3.5\%$). As with all co-location conditions, the remaining SLO violations are driven by the application-level `bcrypt` transients at phase transitions rather than by CPU contention from Ray workers.

4.5 Cross-Condition Comparison

Figures 4.6 and 4.7 place all five conditions on a common axis. Table 4.1 reports mean values and standard deviations across the three repetitions for each condition.

Noisy-neighbor severity. The no-limits condition demonstrates the worst-case noisy-neighbor effect: P99 rises to 906 ± 69 ms ($4.5\times$ the SLO boundary) and SLO compliance collapses to $12.0 \pm 0.9\%$. Total requests drop to 154 068 vs. 249 969 for the no-contention baseline (-38%) and the client failure rate rises to 40.5% . Adding even a static CPU cap reduces violation duration from 805 s to 455 s (-43%), confirming that unguarded co-location is untenable.

Static limits vs. dynamic QoS controller. At first glance, the QoS controller’s compliance improvement (+3.4 pp) and its higher mean P99 appear contradictory. The intuition is: during low-load phases the controller grants Ray more CPU, raising steady-state P99 slightly while never crossing the 200 ms boundary; but during spikes it reacts faster than the fixed cap, reducing worst-case violation time. Compliance counts windows below the threshold; mean P99 integrates the full P99 level (these two statistics measure different aspects of the same distribution).

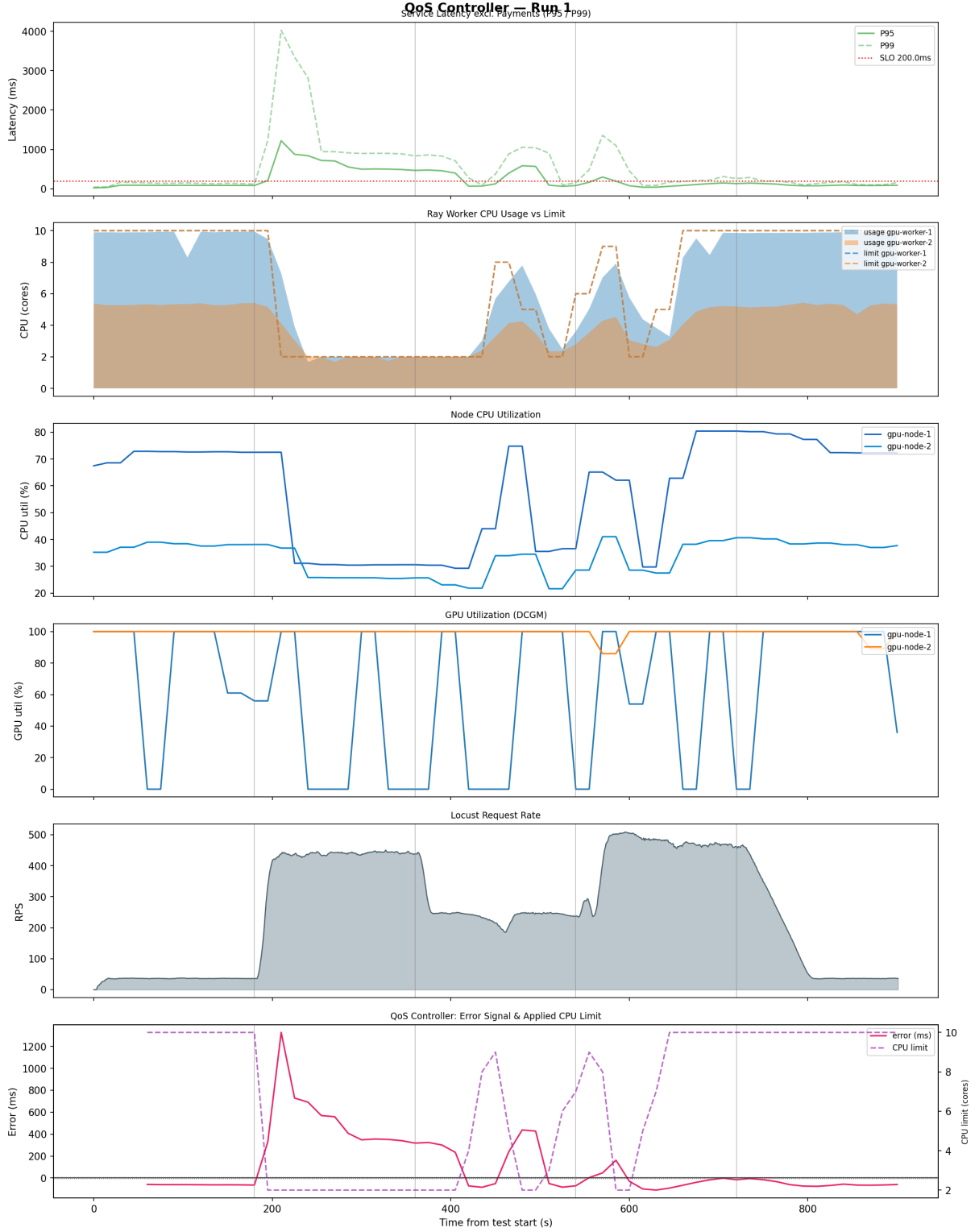


Figure 4.5: Co-location with dynamic QoS controller — run 1 (representative). Panels show P95/P99 latency excluding payments (top), Ray worker CPU usage vs. dynamic limit (second), node CPU utilization (third), GPU utilization from DCGM (fourth), Locust request rate (fifth), and QoS controller error signal with applied CPU limit (bottom). The limit tracks inversely with observed P95: held at 10 during Phase 1, clamped to 2 at Phase 2 onset, released gradually during Phase 3, clamped again at Phase 4, and returned to 10 in Phase 5. Runs 2 and 3 in Section F.3.

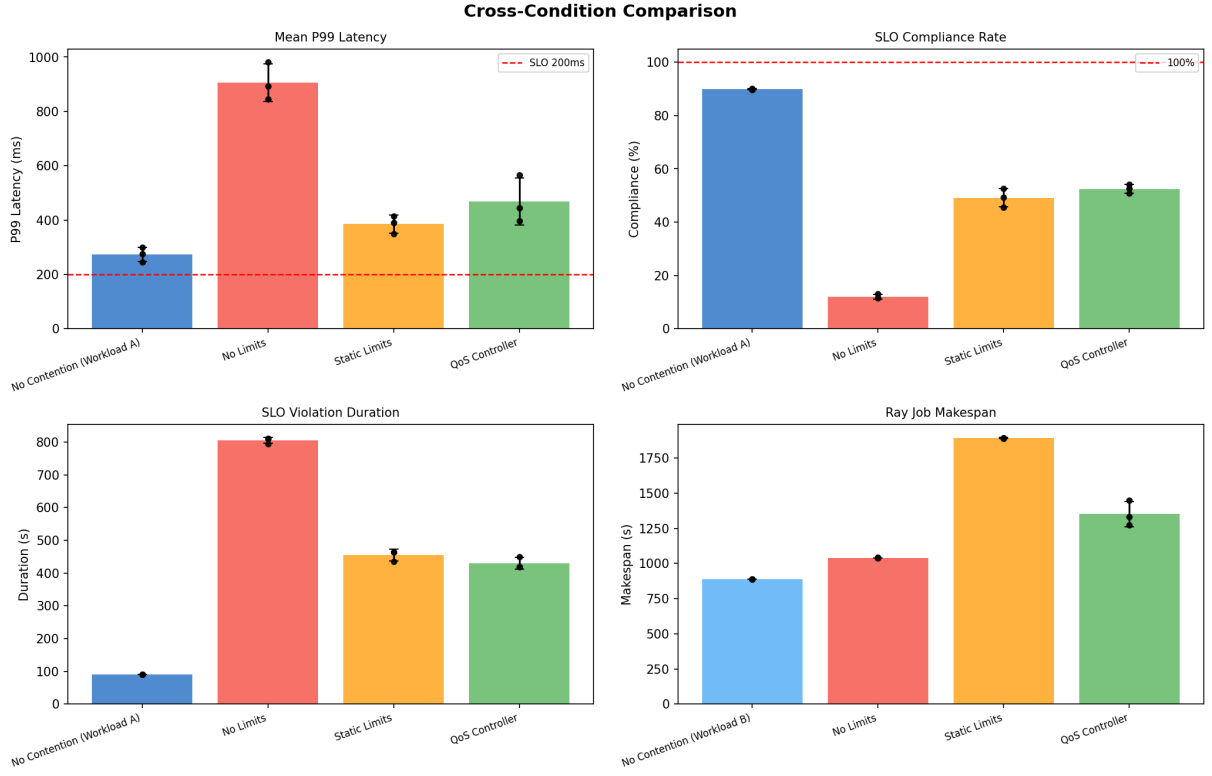


Figure 4.6: Cross-condition comparison of the four primary metrics (mean $\pm$ std over three runs). Top-left: mean P99 latency with 200 ms SLO reference. Top-right: SLO compliance rate. Bottom-left: cumulative SLO violation duration. Bottom-right: Ray job makespan (no-limits value is inflated, see Section 4.2).

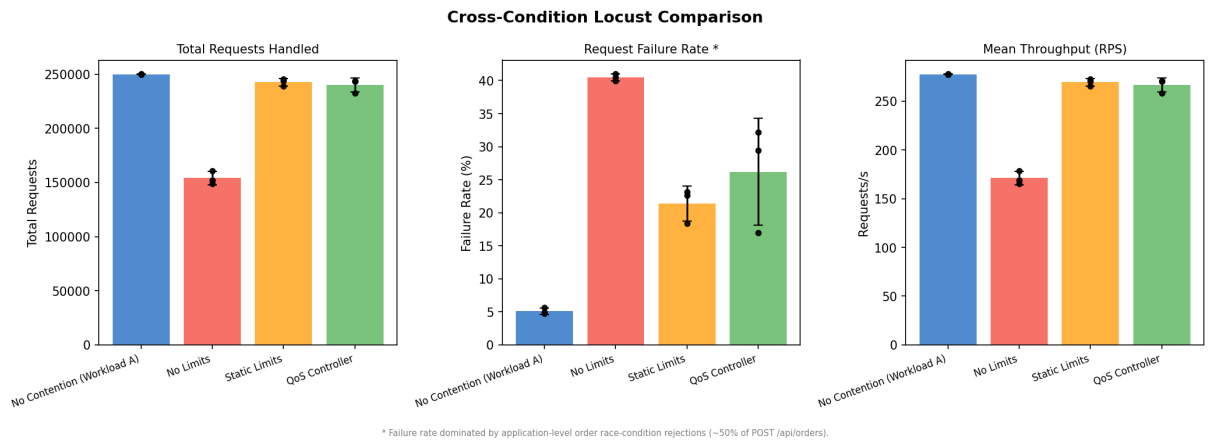


Figure 4.7: Cross-condition Locust client metrics. Failure rate is dominated by application-level order race-condition rejections ($\approx 50\%$ of POST /api/orders attempts) rather than server errors; the cross-condition delta is therefore more informative than the absolute value.

Condi	P95 (ms)	P99 (ms)	SLO Comp. (%)	Viol. (s)	Makespan (s)	CPU (%)	GPU (%)
No Cont. (wla)	89 ± 15	273 ± 27	89.9 ± 0.3	90 ± 0	—	10.5 ± 0.1	—
No Cont. (wlb)	—	—	—	—	888 ± 2	65.8 ± 0.3	80.3 ± 1.0
No Limits	504 ± 27	906 ± 69	12.0 ± 0.9	805 ± 9	$1040^\dagger$	66.5 ± 2.1	82.4 ± 3.5
Static Limits	125 ± 22	385 ± 33	49.1 ± 3.5	455 ± 17	1890 ± 4	41.6 ± 0.3	80.3 ± 1.4
QoS Controller	162 ± 69	468 ± 87	52.5 ± 1.7	430 ± 17	1351 ± 89	50.6 ± 4.3	81.9 ± 1.4

Table 4.1: Per-condition summary (mean $\pm$ std, $n = 3$ runs). Dashes indicate metrics not applicable to that condition. † No-limits makespan is inflated: the job completes before Locust ends but rayjob_end is recorded at Locust-exit time (Section 4.2). Full per-run data in Section F.2.

The QoS controller improves on the static-limits reference across both objectives simultaneously. SLO compliance increases from $49.1 \pm 3.5\%$ to $52.5 \pm 1.7\%$; a modest +3.4 percentage point (pp) gain with notably lower variance. The QoS controller’s mean P99 (468 ± 87 ms) is nevertheless higher than the static-limits reference (385 ± 33 ms). During the low-load phases (1, 3, and 5) when the controller grants up to $C_{\max} = 10$ cores, Ray consumes more CPU and raises steady-state P99 slightly above the levels observed under the fixed 6-core cap. These elevated but sub-violation windows inflate the mean P99 across the full run. At the same time, the controller curtails worst-case spikes more quickly by reacting to latency in real time, reducing total violation seconds. The net result is a lower violation count (better compliance) but a higher average P99 level. Violation duration falls by 25 s. More substantially, makespan decreases from 1890 ± 4 s to 1351 ± 89 s, a 29 % reduction, as the controller reclaims idle CPU capacity during Phases 1, 3, and 5 that the static cap leaves inaccessible. Mean node CPU utilization rises from 41.6 % to 50.6 % accordingly. GPU utilization remains stable at $\approx 80\%$ across all co-location conditions regardless of applied CPU limit, confirming that the bounded prefetch queue keeps the GPU pipeline supplied even under throttling. Client throughput (RPS, total requests) is statistically equivalent between the two conditions; however, the QoS controller shows wider run-to-run variance in request failure rate ($26.2 \pm 8.1\%$ vs $21.4 \pm 2.6\%$), consistent with the per-run variability in CPU-limit behavior visible in Figure 4.5.

Inherent SLO ceiling. The no-contention Workload A baseline achieves only $89.9 \pm 0.3\%$ SLO compliance despite running without any co-tenant. The remaining 10% of violations are caused by application-level `bcrypt` CPU demand at phase transitions, independent of Ray contention. This establishes an inherent SLO ceiling: no co-location strategy can achieve compliance above $\approx 90\%$ without addressing the application-level bottleneck. The gap between the wla baseline (89.9%) and the best co-location condition (QoS controller, 52.5%) quantifies the additional compliance cost attributable to the shared-hardware environment; a 37 pp shortfall.

Chapter 5

Discussion and Outlook

5.1 Evaluation Against Research Question and Hypotheses

The four hypotheses formulated in Section 3.6 provide a structured framework for evaluating the research question (Section 1.2); the results of Chapter 4 allow each to be assessed quantitatively against the observed data.

H1a — Noisy-neighbor severity (confirmed). H1a posits that co-location without CPU limits causes a statistically significant increase in mean P99 latency compared to the isolated Workload A baseline. The no-limits condition produces a mean P99 of 906 ± 69 ms, a $3.3\times$ increase over the wla baseline (273 ± 27 ms), and reduces SLO compliance from $89.9 \pm 0.3\%$ to $12.0 \pm 0.9\%$. H1a is confirmed with high confidence: the effect is large, consistent across all three repetitions, and observable even at the lowest WaveShape load level (Phase 1, 30 virtual users), where CPU starvation begins within the first 30 s of each run.

H1b — Static limits provide measurable SLO protection (confirmed). H1b posits that applying a static CPU limit reduces P99 latency relative to the no-limits condition. The static-limits condition achieves a mean P99 of 385 ± 33 ms and SLO compliance of $49.1 \pm 3.5\%$, compared to 906 ± 69 ms and $12.0 \pm 0.9\%$ without limits. H1b is confirmed: the fixed 6-core cap reduces mean P99 by 57% and increases SLO compliance by 37 pp. However, the 49.1% compliance falls substantially below the 89.9% wla isolation baseline. The shortfall is attributable to the application-level `bcrypt` phase-transition spikes (Section 3.3.4), which inflate P99 regardless of Ray co-location and are therefore outside the reach of any CPU scheduling policy.

H2 — QoS controller SLO compliance $\geq$ static limits (met). H2 posits that the QoS controller achieves P99 SLO compliance at least as high as static limits. The controller achieves $52.5 \pm 1.7\%$ compliance versus $49.1 \pm 3.5\%$ for static limits, a +3.4 pp improvement with notably lower run-to-run variance. H2 is met. The margin is modest because both conditions share the same inherent SLO floor (the 89.9% baseline established in Section 4); no CPU scheduling policy can prevent the application-level `bcrypt` events responsible for it. Within this constraint, the controller achieves marginally better protection during peak phases by dynamically throttling Ray as soon as the P95 signal crosses the 150 ms target, rather than maintaining a fixed limit that may either over-throttle during low load or provide insufficient headroom at phase transitions.

H3 — QoS controller makespan shorter than static limits (met). H3 posits that the controller achieves a shorter Ray training time by reclaiming idle CPU during low-load WaveShape phases. The controller achieves a mean makespan of 1351 ± 89 s versus 1890 ± 4 s for static limits, a 29% reduction. H3 is met with a sizeable margin. During Phases 1, 3, and 5 (the 9 min of low-to-medium load that constitute 60% of the test window) the controller relaxes the CPU limit towards $C_{\max} = 10$ cores (versus the fixed 6-core ceiling under static limits), allowing Ray workers to reclaim otherwise-stranded capacity; mean node CPU utilization rises from 41.6% to 50.6% accordingly (Table 4.1).

Research question answer. The four confirmed hypotheses collectively provide a positive answer to the research question: the PI controller dynamically reclaims idle CPU capacity for the co-located AI workload while simultaneously protecting the web-application SLO. H2 and H3 are met together, demonstrating that the dual objective of SLO protection and throughput maximization is achievable with a single feedback loop and no workload-specific knowledge beyond the observed P95 latency signal. The principal qualification is that the SLO protection margin over static limits is small (+3.4 pp) and bounded above by the 89.9% ceiling established by the wla isolation baseline (Table 4.1); the controller cannot address latency sources that originate inside the web application itself.

5.2 The Noisy Neighbor Effect in Practice

The no-limits condition provides a controlled demonstration of the noisy-neighbor phenomenon introduced in Section 2.4. Even with strict cgroup isolation at the container level, the two co-located workloads interact through shared physical hardware pathways, producing interference that is immediate, severe, and qualitatively different from what container-level resource accounting would predict.

Severity and onset. The most striking observation from the no-limits condition is not the magnitude of the mean P99 (906 ms, $4.5\times$ the SLO boundary) but its onset timing: SLO violations begin at $t \approx 30\text{--}60\text{ s}$ during Phase 1, with only 30 virtual users and a fraction of peak request volume. This is the only condition in this study where violations occur below peak load. The cause is that Ray workers reach full CPU utilization within the first 30 s of the run (consistent with the ramp-up pattern characterized in Section 4.1.2), immediately starving the Node.js event loops responsible for serving HTTP requests. At this load level, Ticket App CPU demand is negligible; the latency degradation is driven entirely by resource competition, not by request volume.

Hardware-level interference pathways. The interference is physical rather than logical. As described in Section 2.4, co-located workloads on shared x86 NUMA nodes contend for three hardware resources regardless of cgroup policy: the L3 processor cache, the memory bus, and the CPU scheduler run queue. Ray’s preprocessing pipeline operates 12 parallel threads that stream large tensor data through the L3 cache, displacing the Ticket App’s hot working set (connection pools, route handler code, and in-flight request state) and forcing subsequent HTTP handlers to re-fetch from DRAM. Simultaneously, Ray’s sustained memory-bus demand from DDP gradient synchronization and tensor transfers introduces stall cycles into the shared memory fabric, increasing the effective latency of every memory-bound operation across both workloads. These effects compound at the CPU scheduler level: Ray’s 12 preprocessing threads compete with the Node.js event loop for kernel scheduling slots, adding queueing delay to request handling even when no single thread is CPU-bound.

Cascade effects on client-facing metrics. The hardware interference cascades into client-visible service degradation beyond the P99 headline figure. The -38% RPS drop and 40.5% failure rate reported in Section 4.2 reflect Locust connection timeouts accumulating as server response time exceeds client timeout thresholds; at 906 ms mean P99, the PBKDF2-heavy order submission requests in particular cross the timeout boundary frequently. This RPS drop compounds the picture: despite fewer in-flight requests, latency remains severely elevated because the offending CPU pressure originates from Ray, not from the Ticket App itself.

Asymmetry of harm. A qualitatively important observation is that the Ray job completes successfully despite causing near-continuous SLO violations throughout the run. GPU utilization remains at $82.4 \pm 3.5\%$ and the training pipeline progresses without interruption. This asymmetry (the AI workload thrives while the co-tenant service degrades) illustrates why the noisy-neighbor problem is insidious in mixed-workload deployments: The offending workload experiences no degradation and generates no feedback signal in-

dicating that it is harming co-tenants. Without an external observer such as the QoS controller, the interference is invisible to the scheduler and undetectable by the AI job, making reactive or manual intervention the only remedy.

5.3 Limitations of the PI Controller

The PI controller achieves its dual objective through a single observable (the P95 HTTP latency signal) and a single actuator (the Ray worker CPU limit). This simplicity is a deliberate design choice suited to a data-sovereign cluster with no external API dependencies, but it imposes three structural limitations that bound the controller’s effectiveness.

Signal conflation. The controller cannot distinguish between latency that originates from Ray CPU contention and latency caused by application-internal CPU demand. At the Phase 1-to-2 and Phase 3-to-4 boundaries, the `bcrypt` phase-transition spike (Section 3.3.4) drives P95 above the 150 ms target regardless of Ray co-location. The controller correctly throttles the Ray CPU limit towards $C_{\min} = 2$ cores (the observed P95 signal genuinely exceeds the target) but the throttle provides no SLO benefit because Ray was not the cause. The only observable consequence is that Ray workers are unnecessarily starved of CPU capacity for the duration of the burst. This limitation is inherent to any single-signal feedback loop that conflates all latency sources into one observable; without a separate signal decomposing contention-driven from application-driven latency, the controller cannot respond selectively. Addressing this is discussed as a future direction in Section 5.7.

Measurement window dilution. A sliding-window metric reflects recent history rather than the instantaneous value: when traffic ramps up abruptly, the 60-second window dilutes the new spike with 60 s of calmer pre-spike data, so the controller sees a smaller error signal than actually exists for the first 15–30 s of the transition.

The PromQL query that feeds the error signal uses a 60 s `rate()` window. This window averages fast transients rather than imposing a 60 s pure delay: a latency spike is immediately visible in the signal but is diluted by 60 s of pre-spike history, reducing the effective error magnitude during fast-rising load. At the Phase 1-to-2 transition (where 400 virtual users are spawned at approximately 60 users/s over 7 s) the windowed P95 rises gradually over approximately 15 s rather than instantly. Measured reaction time from the Phase 1-to-2 boundary to the first Ray CPU patch is approximately 15–30 s, during which P99 can already exceed 200 ms. The dilution effect is most pronounced for fast, high-amplitude transitions: the low-load-to-peak onset produces the largest discrepancy between the true instantaneous P95 and the windowed signal observed by the controller.

Integral windup. Sustained under-reaction during a spike forces the PI integral accumulator to build up, keeping the Ray CPU limit suppressed below the operating range even after traffic subsides. Following the Phase 2-to-3 and Phase 4-to-5 transitions, the accumulated integral holds the CPU limit near $C_{\min} = 2$ cores for several additional minutes, producing a slow recovery to 7–10 cores that reduces Ray CPU availability for the first portion of each subsequent low-load phase. Anti-windup clamping (Section 3.5.2, Equation 3.5) limits the maximum accumulated integral, but does not eliminate the recovery lag entirely; the 5 s control period means that integral discharge proceeds incrementally over several control cycles before the CPU limit returns to the operating range.

5.4 The Stranded Resource Problem

The 29 % makespan reduction achieved by the QoS controller over static limits arises from reclaiming CPU capacity that is structurally unavailable under a fixed-limit policy (the *stranded resource* problem). Understanding this reclamation quantitatively requires examining the temporal distribution of workload demand across the WaveShape profile.

Characterization of stranded capacity. Under the static 6-core Ray cap, the per-node CPU allocation is fixed regardless of Ticket App demand. During Phases 1, 3, and 5 (the 9 min of low-to-medium traffic constituting 60 % of the test window) Ticket App CPU demand is substantially below its peak value. The wla isolation baseline reports a test-averaged mean node CPU utilization of 10.5 %, indicating that the web application consumes approximately 1–2 logical cores per node on average; during Phase 1 (30 virtual users), demand is lower still. With the static cap fixing Ray at 6 cores and the Ticket App consuming well under 2 cores, the remaining capacity on the node is inaccessible to either workload; stranded by the fixed ceiling regardless of actual demand.

Reclamation and throughput impact. The controller exploits this gap by relaxing the Ray CPU limit towards $C_{\max} = 10$ cores during low-load phases, directly reclaiming otherwise-stranded capacity. The aggregate effect is visible in the cross-condition data: mean node CPU utilization rises from 41.6 % to 50.6 % (+9 pp), and Ray training makespan falls from 1890 ± 4 s to 1351 ± 89 s, a 29 % reduction (Table 4.1). The mean reclaimed allocation of approximately 4–6 additional logical cores during low-load phases is consistent with the $C_{\max}$ -to-static-limit gap of 4 cores and the observed utilization increase.

Efficiency–variance trade-off. The throughput gain is not cost-free. The within-run standard deviation of the P99 time series rises from 593 ± 132 ms under static limits to 712 ± 101 ms under the QoS controller; an increase of approximately 20 %. The elevated

variance reflects two sources: transient under-throttling during the onset of peak-load phases before the controller has fully reacted (discussed further in Section 5.5), and minor oscillation around $C_{\max}$ during Phase 1 and Phase 5 as the controller saturates the upper bound. Both effects are transient and do not translate into a higher total SLO violation duration: violation time falls from 455 ± 17 s to 430 ± 17 s (Table 4.1). The wider P99 distribution means that individual measurement windows are more likely to land above the SLO boundary on any given sample, even as overall compliance improves.

5.5 Reaction Time and Control Loop Latency

The 15–30 s reaction window characterized in Section 5.3 is an inherent consequence of reactive control architecture, independent of the specific gains or implementation. This section examines the implications for SLO compliance and evaluates the proactive control alternative.

Detection-to-actuation anatomy. The reaction window comprises three stages. First, the 60 s `rate()` window partially attenuates the rising error signal during the onset of a spike: the signal is visible immediately but diluted by pre-spike history, so the controller observes only a fraction of the true instantaneous P95 for the first ≈ 15 s. Second, the 5 s polling interval delays the next PromQL sample by up to 5 s from the moment the signal first crosses the deadband. Third, after the API PATCH is accepted, the kubelet propagates the updated `cpu.max` to the Linux cgroup within a few milliseconds (Section 3.5.3). The dominant latency is therefore in the observation stage, not the actuation stage: faster actuation primitives such as direct eBPF cgroup writes (Section 5.7) would provide limited end-to-end benefit without also shortening the measurement window.

SLO compliance cost. During the reaction window the controller is aware that latency is elevated but has not yet fully throttled Ray. For each of the two peak-load onsets (Phase 1-to-2 and Phase 3-to-4), P99 can exceed 200 ms throughout the window before the throttle takes effect. A material fraction of the 430 s total SLO violation time recorded for the QoS controller condition (Table 4.1) falls within these two bounded windows, representing an irreducible cost of the reactive architecture under the current observation pipeline.

Reactive versus proactive control. A proactive controller with advance knowledge of the WaveShape profile could pre-throttle Ray before each phase transition, eliminating the reaction-window compliance cost. However, proactive control requires either a deterministic traffic schedule or an accurate real-time forecast from a time-series model. Forecast errors translate directly into either unnecessary pre-emptive throttling (reducing

Ray throughput without SLO benefit) or missed throttles that provide no better protection than the reactive case. Reactive control, by contrast, responds only to what the system actually observes and carries no assumptions about future demand. For the fixed experimental context, where the WaveShape profile is fully known, a predictive extension would be straightforward to add and would close most of the reaction-window compliance gap; this is discussed as a future direction in Section 5.7. In production deployments with unknown traffic patterns, the reactive design remains the appropriate baseline.

5.6 Comparison with Hyperscaler Approaches

The PI controller’s actuation mechanism (dynamically adjusting an existing container’s CPU limit without restarting it) occupies a design point distinct from conventional Kubernetes-native autoscaling strategies. Situating the approach within the broader landscape clarifies both its advantages and the conditions under which it is applicable.

Horizontal versus vertical scaling. Horizontal Pod Autoscaling (HPA) and event-driven autoscaling (KEDA) scale workloads by changing the number of running pod replicas (adding more copies under high load and removing them when idle). This is distinct from the approach used here, which changes the CPU resources granted to each existing pod without adding or removing containers.

The Kubernetes Horizontal Pod Autoscaler (HPA) and event-driven alternatives such as Kubernetes Event-driven Autoscaling (KEDA) scale workloads by adjusting the number of pod replicas. Adding replicas consumes spare capacity from the existing node pool or triggers the cluster autoscaler to provision new nodes; removing replicas frees resources but incurs pod scheduling and container startup latency. Work on combined vertical and horizontal strategies, such as KOSMOS [30], demonstrates that coordinating both axes can improve utilization, but it still operates by scaling replica counts or provisioning new pods. For the co-located AI training job in this thesis, horizontal scaling is structurally inapplicable: DDP training is tightly coupled across workers via NCCL AllReduce, and adding or removing a Ray worker requires the entire distributed job to restart. Node-level cluster autoscaling is similarly unsuitable for the private bare-metal cluster context: bare-metal provisioning takes tens of minutes and operates at a timescale incompatible with the 5 s control loop.

In-place vertical scaling as the enabling primitive. The traditional Vertical Pod Autoscaler (VPA) applies resource changes via its `Recreate` update mode, which evicts the pod and reschedules it with the updated resource specification [33]. This evict-and-reschedule cycle terminates the running container: in the Ray context, the pip runtime cache and the live DDP training session would both be destroyed on every throttle adjust-

ment, making per-cycle VPA rescaling impractical for a long-running distributed training job. The PI controller bypasses VPA entirely and instead PATCHes the Kubernetes `Pods/resize` subresource directly [32] (promoted to Beta in Kubernetes 1.34), the version deployed on this cluster. Because the `kubernetes` Python client does not expose this subresource, the controller issues the PATCH via the `requests` library using in-cluster service-account credentials (Section 3.5.3). The kubelet then propagates the updated limit to the Linux `cpu.max` cgroup entry without restarting the container, preserving the Ray workers’ pip runtime cache and live DDP session across every limit change and reducing actuation latency to a few milliseconds. This allows the controller to apply tens of patches per run without service disruption. A direct experimental comparison with a VPA-based strategy using the `InPlace` mode introduced in VPA 1.7.0 (Kubernetes 1.33+) [33] is a natural extension as that mode matures.

Data sovereignty advantage. The entire control loop (Prometheus scraping, PromQL evaluation, PI control computation, and Kubernetes API PATCH) executes within the private cluster. No external API calls are required; the only network dependencies are the intra-cluster paths to the Prometheus endpoint and the Kubernetes API server. This self-contained architecture satisfies strict data sovereignty requirements: latency metrics, which may indirectly reveal application load patterns and traffic volumes, never leave the cluster perimeter. Hyperscaler autoscaling products that delegate scaling decisions to a cloud-provider control plane are incompatible with this requirement, independent of where the workloads themselves run. For the Swiss regulatory context motivating this thesis (where data residency and processing-location constraints apply [4]) the in-cluster reactive controller is therefore the only viable autoscaling architecture.

5.7 Outlook and Future Work

The results and limitations discussed in the preceding sections point to four concrete directions for extending the controller beyond its current design.

Predictive control. The reactive lag discussed in Section 5.5 (approximately 15–30 s from the onset of a load-phase transition to the first effective throttle) is inherent to any observation-driven controller. A predictive extension would supplement the error signal with a short-horizon forecast of the incoming request rate, allowing the controller to pre-throttle Ray before an anticipated spike rather than reacting after it has begun. Prometheus already records the per-second request rate for the Ticket App services; a lightweight time-series model trained on historical traffic traces (for example, a Long Short-Term Memory network (LSTM) or an online exponential smoothing filter applied to the live RPS signal) could generate a lead-time prediction sufficient to eliminate most

of the reaction-window compliance cost. Pre-emptive throttling carries the risk of unnecessary CPU reduction if the forecast is wrong; a mixed strategy (pre-throttle to an intermediate limit rather than $C_{\min}$, then refine reactively) would bound the throughput cost of forecast errors.

Latency signal decomposition. As discussed in Section 5.3, the controller cannot distinguish whether a P95 elevation originates from Ray CPU contention or from application-internal demand such as the `bcrypt` signup burst at phase transitions. Prometheus already exposes per-service latency histograms for the Ticket App; a decomposition extension would maintain separate PromQL queries for the auth service (which processes `bcrypt` operations) and for the remaining services (orders, tickets), and suppress or dampen the Ray throttle response when the elevation is concentrated in the auth service alone. This requires no architectural modification to the controller; only an additional Prometheus query per control cycle and a conditional guard before the actuation step. The benefit is that the unnecessary 30–45s Ray throttle at each phase transition would be avoided, recovering the CPU capacity lost to signal conflation and shortening the post-transition recovery trajectory discussed in the integral windup paragraph of Section 5.3.

eBPF-based actuation. The Kubernetes API PATCH path for pod CPU limit changes passes through the API server, the kubelet watch loop, and the cgroup write, introducing a few hundred milliseconds of actuation latency after the control decision is made. As noted in Section 5.5, this is not the dominant bottleneck: the observation pipeline contributes the majority of the 15–30s reaction time. However, if the observation pipeline were also shortened (for example, by replacing the Prometheus 60s `rate()` window with an extended Berkeley Packet Filter (eBPF) program that measures per-request latency at the kernel network layer and maintains a running P95 estimate in a BPF map) the actuation path would become the next bottleneck. A companion BPF-based actuation step that writes directly to the Ray workers’ `cpu.max` cgroup entry, bypassing the Kubernetes API server entirely, would reduce the full signal-to-enforcement latency from the current 15–30s to under one second. This combination of kernel-space measurement and kernel-space actuation would make genuinely sub-second reactive control feasible without external API dependencies.

Multi-resource actuation. The PI controller currently throttles a single resource: Ray worker CPU. However, the hardware interference pathways identified in Section 2.4 span three dimensions: the L3 processor cache, the memory bus, and the CPU scheduler run queue. A multi-resource extension would address all three simultaneously. Applying a `cgroup memory.max` limit to Ray workers would cap their resident memory footprint, preventing them from displacing Ticket App pages from the shared last-level cache and re-

ducing memory-bus saturation during DDP gradient synchronization [28]. Independently, a Linux `tc` bandwidth shaper applied to Ray worker pods via a CNI bandwidth plugin or `net_cls` cgroups would throttle AllReduce gradient traffic during HTTP spike phases, reducing NIC contention without requiring as deep a CPU throttle and thereby shortening the post-spike recovery window. A multi-actuator PI controller would maintain one independent error signal per resource dimension with its own integral accumulator and saturation bounds, allowing the CPU, memory, and network actuators to respond at different timescales matched to the dynamics of each interference pathway. The key implementation challenge is characterizing the coupling between actuators: deep CPU throttling already reduces memory-bus traffic as a side effect, so a naive multi-actuator design risks over-throttling if the individual accumulators are not coordinated.

Bibliography

- [1] K. Jewargi, “Public cloud to cloud repatriation trend,” *Scholars Journal of Engineering and Technology*, 2023.
- [2] B. T. Horowitz, “Why some workloads are coming home: The case for cloud repatriation,” 2025. [Online]. Available: <https://biztechmagazine.com/article/2025/08/why-some-workloads-are-coming-home-case-cloud-repatriation>
- [3] Barclays, “Barclays 1h 2024 cio survey,” 2024. [Online]. Available: https://8198920.fs1.hubspotusercontent-na1.net/hubfs/8198920/Barclays_Cio_Survey_2024-1.pdf
- [4] Swiss Federal Administration, “New federal act on data protection (nfadp),” 2023. [Online]. Available: <https://www.kmu.admin.ch/kmu/en/home/facts-and-trends/digitization/data-protection/new-federal-act-on-data-protection-nfadp.html>
- [5] R. Aliano, “Why the private cloud in switzerland is the most secure option for swiss companies,” 2025. [Online]. Available: <https://www.umb.ch/en/blog/news/detail/why-the-private-cloud-in-switzerland-is-the-most-secure-option-for-swiss-companies>
- [6] X. Zhang, L. Li, Y. Wang, E. Chen, and L. Shou, “Zeus: Improving resource efficiency via workload colocation for massive kubernetes clusters,” *IEEE Access*, 2021.
- [7] M. Cerqueira de Abranches and P. Solis, “An algorithm based on response time and traffic demands to scale containers on a cloud computing system,” in *2016 IEEE 15th International Symposium on Network Computing and Applications (NCA)*, 2016.
- [8] Éléonore Daxhelet, “Swiss privacy regulator establishes new compliance standards,” 2026. [Online]. Available: <https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/news/swiss-privacy-regulator-establishes-new-compliance-standards>
- [9] DuoKey, “Privatim’s resolution declares most cloud deployments non-compliant,” 2026. [Online]. Available: <https://medium.com/@duokey.com/swiss-data-protection-authorities-declare-most-cloud-deployments-non-compliant-275af1bfdc8d>

-
- [10] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, “An updated performance comparison of virtual machines and linux containers,” in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2015.
 - [11] M. Baumgartner, A. Lillo, and S. Rumley, “Performance losses with virtualization: Comparing bare metal to VMs and containers,” in *High Performance Computing (ISC High Performance 2023)*, 2023.
 - [12] J. P. Walters, A. J. Younge, D. I. Kang, K. T. Yao, M. Kang, S. P. Crago, and G. C. Fox, “Gpu passthrough performance: A comparison of kvm, xen, vmware esxi, and lxc for cuda and opencl applications,” *IEEE International Conference on Cloud Computing*, 2014.
 - [13] SmartX SKS Team, “Kubernetes on vms vs. bare metal: Comparison of performance,” 2024. [Online]. Available: <https://makeitsimple.medium.com/kubernetes-on-vms-vs-bare-metal-comparison-of-performance-674a920b93e4>
 - [14] J. Lewis and M. Fowler, “Microservices,” <https://martinfowler.com/articles/microservices.html>, 2014, accessed May 2026.
 - [15] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2021.
 - [16] G. Blinowski, A. Ojdowska, and A. Przybyłek, “Monolithic vs. microservice architecture: A performance and scalability evaluation,” *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022.
 - [17] R. C. Thota, “Next generation multi-tenant saas with ai orchestrated workload isolation for scalable performance,” *World Journal of Advanced Engineering Technology and Sciences*, vol. 16, no. 2, pp. 452–462, 2025.
 - [18] Rafay Systems, “Demystifying fractional gpus in kubernetes: Mig, time slicing, and custom schedulers,” 2025. [Online]. Available: <https://rafay.co/ai-and-cloud-native-blog/demystifying-fractional-gpus-in-kubernetes-mig-time-slicing-and-custom-schedulers>
 - [19] NVIDIA Corporation, “Cuda multi-process service overview,” 2025. [Online]. Available: https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf
 - [20] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, “Ray: A distributed framework for emerging AI applications,” in *13th USENIX Symposium on*

- Operating Systems Design and Implementation (OSDI 18)*, 2018. [Online]. Available: <https://arxiv.org/abs/1712.05889>
- [21] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, “PyTorch distributed: Experiences on accelerating data parallel training,” *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.
- [22] D. Nichols, S. Singh, S.-H. Lin, and A. Bhatele, “A survey and empirical evaluation of parallel deep learning frameworks,” <https://arxiv.org/abs/2111.04949>, 2021, arXiv:2111.04949.
- [23] A. Gibiansky, “Bringing HPC techniques to deep learning,” <https://andrew.gibiansky.com/blog/machine-learning/baidu-allreduce/>, 2017, accessed May 2026.
- [24] PyTorch Contributors, “torch.distributed — distributed communication package,” 2025, accessed June 2026. [Online]. Available: <https://pytorch.org/docs/stable/distributed.html>
- [25] A. Isenko, R. Mayer, J. Jedele, and H.-A. Jacobsen, “Where is my training bottleneck? Hidden trade-offs in deep learning preprocessing pipelines,” in *Proceedings of the 2022 International Conference on Management of Data*, 2022. [Online]. Available: <http://dx.doi.org/10.1145/3514221.3517848>
- [26] Kubernetes Authors, “Pod quality of service classes,” 2025, accessed April 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>
- [27] —, “Resource management for pods and containers,” 2025, accessed April 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- [28] The Linux Kernel Authors, “Control group v2,” 2024, accessed April 2026. [Online]. Available: <https://docs.kernel.org/admin-guide/cgroup-v2.html>
- [29] —, “CFS scheduler,” 2024, accessed April 2026. [Online]. Available: <https://docs.kernel.org/scheduler/sched-design-CFS.html>
- [30] L. Baresi, D. Y. X. Hu, G. Quattrocchi, and L. Terracciano, “Kosmos: Vertical and horizontal resource autoscaling for kubernetes,” 2021.
- [31] S. Grider, “Microservices with node js and react,” <https://www.udemy.com/course/microservices-with-node-js-and-react/>, 2020, accessed May 2026.

- [32] Kubernetes Authors, “Resize CPU and memory resources assigned to containers,” 2025, accessed May 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/>
- [33] —, “Vertical pod autoscaling,” 2026, accessed June 2026. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/>

List of Figures

1.1	The Cloud Repatriation Trend: percentage of CIOs planning to repatriate workloads back to private infrastructure (Data based on the Barclays 2024 CIO Survey [3]).	6
2.1	The physical mechanics of the Noisy Neighbor problem: independent CPU cores contending for shared L3 Cache and Memory Bus bandwidth.	14
3.1	Hardware and network topology diagram showing the dual-NIC configuration, asymmetric cluster network bandwidth, and workload distribution across the control-plane and GPU nodes.	24
3.2	Architecture of Workload A. External traffic enters via NGINX Ingress, which routes requests by path prefix directly to the responsible backend service. Frontend routes (/) are served by the Next.js client. Services communicate asynchronously via Kafka (Event Bus); each service owns an isolated database instance. Tickets and Orders run two replicas each.	27
3.3	Sequence diagram of the Locust purchase flow. Filled arrows are synchronous HTTP calls (request/response); open arrows are asynchronous Kafka events. Between the ticket creation response and the order request, a <code>gevent.sleep(500ms)</code> pause allows the <code>ticket.created</code> event to propagate to <code>orders-srv</code> before the order is submitted; a second <code>gevent.sleep(300ms)</code> serves the same purpose for <code>order.created</code> before the payment request. The <code>expiration-srv</code> is omitted: it consumes <code>order.created</code> to start a countdown timer but is not part of the HTTP response path.	29
3.4	Observed requests-per-second (RPS) time series for the WaveShape load profile across a representative 15-minute run. Phase transitions at $t = 180\text{s}$, 360s , 540s , and 720s are clearly visible. Peak throughput of $\approx 500\text{ RPS}$ occurs during Phases 2 and 4.	31
3.5	CPU preprocessing pipeline for Workload B. Each training step draws a pre-processed batch from the bounded queue (<code>maxsize=2</code>); the daemon thread continuously refills it using 12 parallel <code>ThreadPoolExecutor</code> workers. Because torchvision C extensions release the GIL, all 12 threads run truly in parallel, saturating up to 12 CPU cores. The queue allows preprocessing of step $t+1$ to overlap with GPU work of step t	34

3.6	Closed-loop block diagram of the Dynamic QoS Controller. Every 5 s the PI controller computes an error $e_k = L_k - L_{\text{target}}$ from the Prometheus-observed P95 latency and adjusts the Ray worker CPU limits via the Kubernetes resize subresource. A deadband of ± 10 ms and a per-patch cooldown of 10 s suppress actuation under noisy or rapidly changing conditions.	37
4.1	No-contention Workload A baseline — run 1 (representative). Panels show P95/P99 latency excluding payments (top), node CPU utilization (middle), and Locust request rate (bottom). Two transient P99 spikes at phase transitions ($t \approx 180$ s and $t \approx 540$ s); steady-state P99 between spikes is ≈ 49 ms. Runs 2 and 3 in Section F.3.	45
4.2	No-contention Workload B baseline — run 1 (representative). Panels show Ray worker CPU usage vs. applied limit (top), node CPU utilization (middle), and GPU utilization from DCGM (bottom). The oscillating GPU utilization on <code>gpu-node-1</code> reflects the CPU-bound preprocessing pipeline periodically stalling the prefetch queue. Runs 2 and 3 in Section F.3. . . .	46
4.3	Co-location without CPU limits — run 1 (representative). Panels show P95/P99 latency excluding payments (top), Ray worker CPU usage vs. applied limit (second), node CPU utilization (third), GPU utilization from DCGM (fourth), and Locust request rate (bottom). P99 exceeds 200 ms from $t \approx 30$ s through the end of the run ($\approx 89\%$ of the test window), with a single 15 s transient recovery in Phase 5; GPU utilization remains high throughout as Ray continues training despite severe latency violations. Runs 2 and 3 in Section F.3.	48
4.4	Co-location with static CPU limits — run 1 (representative). Panels show P95/P99 latency excluding payments (top), Ray worker CPU usage vs. applied 6-core limit (second), node CPU utilization (third), GPU utilization from DCGM (fourth), and Locust request rate (bottom). The flat limit line prevents Ray from using idle CPU headroom during low-traffic phases. Runs 2 and 3 in Section F.3.	50
4.5	Co-location with dynamic QoS controller — run 1 (representative). Panels show P95/P99 latency excluding payments (top), Ray worker CPU usage vs. dynamic limit (second), node CPU utilization (third), GPU utilization from DCGM (fourth), Locust request rate (fifth), and QoS controller error signal with applied CPU limit (bottom). The limit tracks inversely with observed P95: held at 10 during Phase 1, clamped to 2 at Phase 2 onset, released gradually during Phase 3, clamped again at Phase 4, and returned to 10 in Phase 5. Runs 2 and 3 in Section F.3.	52

4.6	Cross-condition comparison of the four primary metrics (mean $\pm$ std over three runs). Top-left: mean P99 latency with 200 ms SLO reference. Top-right: SLO compliance rate. Bottom-left: cumulative SLO violation duration. Bottom-right: Ray job makespan (no-limits value is inflated, see Section 4.2).	53
4.7	Cross-condition Locust client metrics. Failure rate is dominated by application-level order race-condition rejections ($\approx 50\%$ of <code>POST /api/orders</code> attempts) rather than server errors; the cross-condition delta is therefore more informative than the absolute value.	53
F.1	No-contention Workload A (runs 2 and 3). Panels as in Figure 4.1.	121
F.2	No-contention Workload B (runs 2 and 3). Panels as in Figure 4.2.	121
F.3	Co-location without CPU limits (runs 2 and 3). Panels as in Figure 4.3.	121
F.4	Co-location with static CPU limits (runs 2 and 3). Panels as in Figure 4.4.	122
F.5	Co-location with dynamic QoS controller (runs 2 and 3). Panels as in Figure 4.5.	122

List of Tables

3.1	Hardware specifications of the three-node bare-metal Kubernetes cluster. .	22
3.2	Software component versions pinned across the cluster.	23
3.3	Workload A services, their technology stack, and primary responsibilities. Full HTTP API specifications are listed in Appendix C.1.	28
3.4	WaveShape load profile phases.	31
3.5	Ray worker CPU resource policy per experimental condition.	34
3.6	QoS controller tunable parameters, values, and selection rationale.	38
3.7	Prometheus metrics exported by the QoS controller on port 8080.	40
3.8	Five experimental conditions, their Ansible playbooks, workload configuration, and purpose.	41
3.9	Prometheus metrics exported per run, grouped by category.	42
4.1	Per-condition summary (mean $\pm$ std, $n = 3$ runs). Dashes indicate metrics not applicable to that condition. [†] No-limits makespan is inflated: the job completes before Locust ends but <code>rayjob_end</code> is recorded at Locust-exit time (Section 4.2). Full per-run data in Section F.2.	54
A.1	14-Week Project Schedule and Milestones	82
A.2	Project Risk Analysis, Mitigation Strategies, and Outcomes	83
B.1	Cluster network interface assignments per node.	85
C.1	Auth service HTTP endpoints.	94
C.2	Tickets service HTTP endpoints.	94
C.3	Orders service HTTP endpoints.	95
C.4	Payments service HTTP endpoints.	97
C.5	Kafka event topics.	98
C.6	Kafka consumer group IDs per listener.	99
F.1	Cross-condition summary loaded from <code>comparison.csv</code> (mean $\pm$ std, $n = 3$ runs). No-limits makespan is inflated (see Section 4.2).	119
F.2	Per-run measurements for all five experimental conditions ($n = 15$ runs total).	120

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
APT	Advanced Package Tool
BPF	Berkeley Packet Filter
cgroups	Linux Control Groups
CFS	Completely Fair Scheduler
CLOUD Act	Clarifying Lawful Overseas Use of Data Act
CNI	Container Network Interface
CPU	Central Processing Unit
CRD	Custom Resource Definition
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DCGM	NVIDIA Data Center GPU Manager
DDP	Distributed Data Parallel
DNS	Domain Name Service
DRAM	Dynamic Random-Access Memory
eBPF	Extended Berkeley Packet Filter
GIL	Global Interpreter Lock
GPU	Graphics Processing Unit
HPA	Horizontal Pod Autoscaler
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
JSON	JavaScript Object Notation
JWT	JSON Web Token
KEDA	Kubernetes Event-Driven Autoscaling
KVM	Kernel-based Virtual Machine
LLC	Last-Level Cache
LSTM	Long Short-Term Memory
MIG	Multi-Instance GPU

MPS	CUDA Multi-Process Service
MTU	Maximum Transmission Unit
NCCL	NVIDIA Collective Communications Library
nFADP	Swiss Federal Act on Data Protection (revised)
NIC	Network Interface Card
NUMA	Non-Uniform Memory Access
NVMe	Non-Volatile Memory Express
OOM	Out-Of-Memory
P95	95th Percentile
P99	99th Percentile
PBKDF2	Password-Based Key Derivation Function 2
PI	Proportional-Integral
PromQL	Prometheus Query Language
PV	Persistent Volume
PVC	Persistent Volume Claim
QoS	Quality of Service
RBAC	Role-Based Access Control
RPS	Requests Per Second
SLA	Service Level Agreement
SLO	Service Level Objective
SSH	Secure Shell
SSR	Server-Side Rendering
TCP	Transmission Control Protocol
TLS	Transport Layer Security
VM	Virtual Machine
VPA	Vertical Pod Autoscaler
VRAM	Video Random Access Memory
VU	Virtual User
YAML	YAML Ain't Markup Language

Appendix A

Project Management

A.1 Task Description

The following reproduces the original task description as provided at the start of the project. The thesis objectives as ultimately realized (including the pivot away from hypervisor virtualization and Chaos Testing toward a dynamic QoS controller) are described in Sections 1.2 and 1.3.

Motivation

Many companies seek to move away from public cloud providers (e.g., AWS, Azure) to better protect customer data, reduce operational costs, and maintain technological sovereignty. However, building a private cloud is challenging for instance because modern software has fundamentally different infrastructure requirements. As an example we can focus on the following two applications and their requirements:

- **Standard Web Applications** (e.g., e-commerce platforms) require stability, redundancy, and high availability.
- **AI/Data-Intensive Workloads** (e.g., data processing, AI model training) require large amounts of compute power and low overhead.

Public cloud providers satisfy both needs with mature services, but replicating this flexibility on self-hosted hardware is significantly harder. If the private infrastructure is too rigid, AI workloads become slow and inefficient. If it is too flexible, traditional applications may lose stability.

The core challenge is therefore to identify an architectural approach capable of reliably handling both workload types—without relying on hyperscale cloud providers.

Objective

The goal of this thesis goes beyond simply setting up a private infrastructure. It aims to systematically analyze architectural approaches for running mixed workloads on private hardware, evaluate their trade-offs, and implement a prototype based on the findings. The work of this thesis is structured into three main parts:

1. **Technology and Architecture Analysis:** Evaluate different virtualization and cluster-management approaches (e.g., Proxmox VE, alternative hypervisors, or bare-metal orchestration). The comparison will focus on balancing performance, maintainability, and usability.
2. **Design and Implementation of the Selected Architecture:** Implement the chosen approach using Infrastructure-as-Code (e.g., Terraform, Kubernetes) to ensure reproducibility, scalability, and automated deployment.
3. **Validation Through Realistic Workloads:** Demonstrate that the solution can support both stable web applications and compute-heavy data/AI workloads.

Methodology

1. Analysis and Technology Selection

- Define detailed system requirements (e.g., “must recover from node failure in <30 seconds,” “must sustain data throughput of at least X”).
- Compare candidate technologies (e.g., Proxmox VE vs. bare-metal Kubernetes) and justify the architectural choices based on these requirements.

2. Infrastructure Implementation

- Set up the physical hardware and network architecture aligned with the selected design.
- Automate environment provisioning using Infrastructure-as-Code (Terraform, Ansible, or similar).
- Deploy and configure Kubernetes to coordinate resources, ensuring—e.g., via scheduling policies or resource quotas—that AI workloads do not starve web applications.

3. Reference Workload Development: To demonstrate generality and flexibility, two contrasting workloads will be deployed:

- **Workload A – Web Application:** A microservice-based “Ticket Shop” (Next.js + MongoDB) to test reliability and availability.

- **Workload B – Data/AI Application:** A distributed AI computation job using a framework such as Ray or Apache Spark to test compute performance and resource consumption.
4. **Evaluation:** The final phase will assess the system’s robustness, performance, and workload isolation capabilities:
- **Chaos Testing:** Remove or shut down nodes to validate that the web application maintains availability.
 - **Performance Benchmarking:** Measure overhead introduced by virtualization and orchestration for the AI workload.
 - **Workload Interference Testing:** Run both workloads concurrently to test whether the AI task impacts the stability or responsiveness of the web application.

A.2 Repository Structure

The repository follows a flat, concern-separated layout in which each top-level directory maps to one architectural layer of the system.

```

bachelorthesis/
├── ansible/ - bare-metal provisioning (IaC)
│   ├── inventory/
│   │   └── hosts.yml - node IPs and roles
│   └── playbooks/
│       ├── 01-network-setup.yml - NIC config, MTU 9000
│       ├── 02-system-baseline.yml - OS hardening, swap off
│       ├── 03-containerd.yml - container runtime setup
│       ├── 04-kubernetes-components.yml - kubeadm, kubelet, kubect1
│       ├── 04b-kubelet-bind.yml - bind kubelet to cluster NIC
│       ├── 05-cluster-bootstrap.yml - kubeadm init + Calico CNI
│       ├── 06-telemetry-stack.yml - Prometheus + Grafana
│       ├── 07-gpu-operator.yml - NVIDIA GPU Operator + DCGM
│       ├── 07b-ingress-nginx.yml - NGINX Ingress (NodePort 30080)
│       ├── 07c-kuberay-operator.yml - KubeRay + persistent RayCluster
│       ├── 08a-test-no-contention-wla.yml - condition: Workload A alone
│       ├── 08b-test-no-contention-wlb.yml - condition: Workload B alone
│       ├── 09-test-no-limits.yml - condition: both, no CPU limits
│       ├── 10-test-static-limits.yml - condition: both, static limits
│       └── 11-test-qos.yml - condition: both, PI controller active
├── tasks/ - shared task fragments
│   ├── export-and-fetch.yml - Prometheus export + scp
│   ├── locust-test.yml - run Locust load test
│   └── teardown.yml - cleanup after each run
└── kubernetes/ - Kubernetes manifests

```

- workload-a/ - Ticket App services
 - auth-depl.yml - auth service
 - client-depl.yml - Next.js frontend
 - expiration-depl.yml - expiration worker
 - kafka-depl.yml - Kafka broker
 - mongo-depl.yml - MongoDB instances
 - orders-depl.yml - orders service
 - payments-depl.yml - payments service
 - redis-depl.yml - Redis (Bull queue)
 - tickets-depl.yml - tickets service
 - ingress-srv.yml - path-based routing rules
 - service-monitor.yml - Prometheus scrape config
 - configmap.yml - shared environment variables
 - secrets.yml - JWT secret, credentials
- workload-b/ - Ray cluster manifests
 - raycluster.yml - persistent RayCluster (deployed once)
 - rayjob.yml - per-run training job
 - ray-podmonitor.yml - Prometheus pod monitor
- qos-controller/ - QoS controller deployment
 - deployment.yml - controller pod spec
 - rbac.yml - RBAC with pods/resize permission
 - configmap.yml - Kp, Ki, targets, CPU limits
 - service.yml - metrics endpoint
 - servicemonitor.yml - Prometheus scrape
- qos-controller/ - Python QoS controller source
 - controller.py - PI loop: Prometheus -> API PATCH
 - Dockerfile - python:3.11-slim image
 - requirements.txt - Python dependencies
- workload-a/ - Ticket App npm monorepo
 - auth/ - user auth, JWT (Node.js/Express)
 - client/ - Next.js SSR frontend
 - common/ - shared library (@ticket-app/common)
 - expiration/ - order expiration worker (Bull)
 - orders/ - purchase state machine
 - payments/ - mocked payment gateway
 - tickets/ - ticket inventory and catalog
 - skaffold.yml - build + push all services to GHCR
- workload-b/ - distributed AI job
 - ray-job.py - ResNet-50 DDP training (TorchTrainer)
- scripts/ - load testing and data export
 - load-test.py - Locust WaveShape load profile
 - export-metrics.py - Prometheus to per-metric CSV
 - run-all-conditions.sh - orchestrator: 5 conditions x 3 runs
- notebooks/ - results analysis
 - results-analysis.ipynb - SLO compliance, stats, cross-condition plots
- results/ - generated per-condition, per-run CSVs
 - no-contention-wla/ - Workload A isolation baseline
 - no-contention-wlb/ - Workload B isolation baseline

```

├─ no-limits/ - co-located, no CPU cap
├─ static-limits/ - co-located, fixed limits
├─ dynamic/ - co-located, PI controller active
├─ report/ - LaTeX thesis source
│   ├── main.tex - root document
│   ├── preamble.tex - packages and custom commands
│   ├── references.bib - bibliography
│   ├── chapters/ - chapter and appendix .tex files
│   ├── figures/ - PNG figures
│   └── diagrams/ - Excalidraw source files

```

A.3 Challenges

Hardware and Network Constraints. The control-plane node (`cpu-node-1`) is equipped with a 10 Gbps Network Interface Card that was expected to auto-negotiate to 2.5 Gbps to match the cluster switch. Due to a hardware incompatibility, the card fell back to 1 Gbps. The practical impact on the thesis objectives was negligible: the Ray AI workers, which generate the bulk of throughput-intensive DDP AllReduce traffic, are hosted exclusively on `gpu-node-1` and `gpu-node-2`, both of which have fully functional 2.5 Gbps NICs. The 1 Gbps constraint affected only MongoDB replication traffic and Kubernetes API server calls, neither of which were throughput-critical under any of the five experimental conditions.

Technological Complexity and Learning Curve. Prior to this thesis, the author’s systems administration experience was limited to Proxmox VE hypervisor management. The pivot to bare-metal Kubernetes required acquiring proficiency across a broad stack with no prior exposure: Ansible for infrastructure-as-code provisioning, Calico CNI with multi-homed Jumbo-Frame routing, the KubeRay operator and Ray distributed training framework, DCGM GPU telemetry, and PromQL query design for the Prometheus-based control signal.

QoS Controller Engineering. The QoS controller required bridging three domains simultaneously. Designing a stable PI control loop demanded empirical gain tuning: an initial $K_p = 0.03$ produced oscillation on moderate errors and was reduced to $K_p = 0.02$; the $C_{\max}$ ceiling was lowered from 12 to 10 cores after observing that 12 cores caused Phase 1 P95 to exceed the 150 ms target even at low load. A separate integration obstacle was the Kubernetes in-place pod resize subresource, which is not exposed by the official Python `kubernetes` client; the controller was therefore written to issue raw HTTPS requests with in-cluster service-account credentials directly against the API server.

Workload Calibration. A substantial unanticipated challenge was making the Ticket App CPU-visible. In early iterations the Node.js handlers were almost entirely I/O-bound, producing under 2% node CPU utilization at 400 concurrent users. Without a meaningful CPU signal, Ray contention was unobservable by any scheduling policy and the core experiment was infeasible. This was resolved by introducing a deliberate synchronous PBKDF2 key derivation per order creation (Section 3.3), raising steady-state node CPU utilization and making contention empirically detectable.

Several latency pathologies unrelated to Ray contention were also discovered and corrected through iterative load testing: Missing MongoDB indexes on high-traffic query paths causing $O(n)$ collection scans as order and ticket collections grew during a run; an unbounded `GET /api/tickets` query returning 100 KB+ payloads at high ticket counts; and a 5-minute order expiration window that flooded Kafka with `expiration:order-expired` events mid-test at peak load. Each issue produced latency signatures in the Prometheus histograms that would have been indistinguishable from noisy-neighbor interference had they gone uncorrected, invalidating the experimental results.

A.4 Schedule

The following schedule (see Table A.1) outlines the 14-week project plan for this thesis. It details the designated phases and key milestones, progressing from the initial theoretical research to the practical implementation and empirical evaluation of the dynamic QoS controller. As noted in Section 1.3, the schedule reflects the pivot away from infrastructure chaos testing toward active software engineering and interference mitigation.

A.5 Risk analysis

Given the architectural complexity of establishing a self-hosted private cloud and developing a custom control loop, several project risks have been identified. Table A.2 outlines these potential technical and organizational risks, assessing their probability and impact, alongside predefined mitigation strategies.

Week	Phase	Milestones
1–2	Research & Theory	Completion of literature review, definition of data sovereignty constraints, and research into dynamic resource management and control loops.
3	Architecture Analysis	Definition of strict system requirements and comparative analysis of static orchestration versus dynamic QoS controllers.
4	Architecture Design	Finalization of the architectural blueprint, including workload co-location strategy and the Python controller logic.
5	Infrastructure (Hardware & OS)	Setup of physical hardware and network; implementation of initial Infrastructure-as-Code (IaC) for base provisioning.
6	Infrastructure (K8s & Telemetry)	Deployment of the Kubernetes cluster, functional container networking (CNI), NVIDIA GPU Operator, and the Prometheus/Grafana stack.
7–8	Workload A (Web App)	Development and deployment of the microservice-based Ticket App (Next.js + Express.js) and establishment of the Locust load-testing environment.
9	Workload B (AI Job)	Deployment of the distributed AI computation job (Ray) with configured GPU access and maximum CPU burst limits.
10	Custom Controller Dev.	Implementation and testing of the custom Python QoS Controller (querying Prometheus and patching Kubernetes pod limits).
11	Evaluation (Interference)	Execution of Workload Interference Testing (A/B testing the static baseline against the dynamic Python controller).
12	Data Aggregation & Analysis	Aggregation, visualization (Grafana dashboards), and statistical analysis of all interference benchmark data.
13	Buffer & Troubleshooting	Allocated buffer period; used primarily for Workload A calibration (PBKDF2 CPU load introduction, MongoDB index additions, Kafka expiration-window tuning), controller gain refinement (K_p , $C_{\max}$), and finalising system diagrams.
14	Final Polish	Final proofreading, formatting adjustments, citation verification, and formal submission of the thesis.

Table A.1: 14-Week Project Schedule and Milestones

Risk Description	Probability	Impact	Mitigation Strategy	Outcome
Control Loop Latency The Python controller reacts too slowly to sudden web traffic spikes, causing the web application's P95 latency to violate the 200 ms SLA before the AI workload is throttled.	High	Medium	Set the Prometheus scraping interval to 5 s. Keep the Python controller logic lightweight. Pre-calculate API patch payloads to minimize execution delay.	Partial: A 15–30 s reaction window is inherent to the 60 s Prometheus rate window (Section 5.5), not to controller overhead. The 5 s poll interval and lightweight logic prevented API server strain.
API Rate Limiting / Overload The custom controller aggressively spams the Kubernetes API server with patch requests, destabilising the control plane.	Medium	High	Implement a cooldown period in the controller logic (do not issue another patch for at least 10 s after a previous state change).	Mitigated: The 10 s cooldown bounded patch frequency; no API overload events were observed across all 15 experimental runs.
Scope Creep & Schedule Overrun Developing a custom software controller on top of complex bare-metal infrastructure within 14 weeks is highly ambitious.	High	Medium	Adopt a Minimum Viable Product (MVP) approach. Throttle only the AI pod's CPU limit; attempt GPU/MPS partitioning only if the CPU controller is complete ahead of schedule.	Partial: Unplanned Workload A calibration consumed approximately one additional week (see Section A.3). The buffer week and MVP scope prevented a schedule overrun.
Data Loss or Irreproducibility Loss of configuration state, benchmark data, or failure to reproduce the infrastructure setup.	Low	High	Mandate strict version control (Git) for all Ansible playbooks, Kubernetes manifests, and Python controller code. Export Prometheus metric data to CSV after every test run.	Mitigated: All 15 experimental runs were preserved via Git and per-run Prometheus CSV exports; no data loss occurred.
Heterogeneous CPU Architectures Mixing processor generations or microarchitectures can disrupt scheduling behavior or require per-architecture container image builds.	Low	Medium	Rely entirely on Kubernetes at the container layer; containers abstract the underlying CPU architecture. Build images for <code>linux/amd64</code> explicitly to avoid cross-compilation issues.	N/A: All three nodes use AMD Ryzen processors. No cross-vendor scheduling issues arose.

Table A.2: Project Risk Analysis, Mitigation Strategies, and Outcomes

Appendix B

Infrastructure Setup

This appendix documents the Ansible playbooks used to provision the bare-metal Kubernetes cluster, explaining what each playbook configures and the engineering rationale behind each decision. Each playbook includes built-in **VERIFY** tasks that assert correct configuration state before the playbook exits; all validation is automated and no manual verification steps are required after provisioning.

B.1 01-network-setup.yml

This playbook configures the cluster-facing NIC on each node. It deploys a dedicated Netplan configuration file at `/etc/netplan/99-cluster-network.yaml` that assigns a static IP address on the `10.0.1.0/24` subnet and sets the MTU to 9000 to enable Jumbo Frames. A dedicated file is used rather than modifying the OS-generated Netplan default so that the management interface configuration on `192.168.1.0/24` is never altered; the cluster-network setup can be fully reversed by the reset playbook simply by deleting this single file. Table B.1 lists the exact interface name and static IP assigned to each node.

MTU 9000 amortizes the per-frame kernel interrupt overhead across larger payloads. During Ray DDP gradient synchronization, the AllReduce collective streams gradient tensors between the two GPU workers over the cluster interface. Without Jumbo Frames, each 1500-byte Ethernet frame triggers a separate interrupt on the receiving NIC; at the tensor sizes produced by a ResNet-50 backward pass this interrupt pressure is known to consume non-trivial CPU cycles that would otherwise be available to the application workloads.

Verification

The playbook includes three built-in **VERIFY** tasks executed immediately after `netplan apply`. First, it greps the generated configuration file for the assigned static IP, confirming the Netplan template was written correctly. Second, it runs `ip link show`

Hostname	Cluster Interface	Speed
cpu-node-1	enp43s0f0	1 Gbps
gpu-node-1	enp9s0	2.5 Gbps
gpu-node-2	enp4s0	2.5 Gbps

Table B.1: Cluster network interface assignments per node.

`fast_iface` and asserts the string `mtu 9000` appears in the output; the task fails immediately if the kernel did not accept the Jumbo Frame setting. Third, it pings every other cluster node on the `10.0.1.0/24` subnet (using `failed_when: false`) to report inter-node reachability as an informational check without blocking the playbook on transient packet loss.

B.2 02-system-baseline.yml

This playbook applies the host-level prerequisites required by Kubernetes before any cluster components are installed. The following modifications are applied to all nodes:

- **Memory Management:** Linux swap space was completely disabled (`swapoff -a` and commented out in `/etc/fstab`). The `kubelet` refuses to start with swap active (`--fail-swap-on=true` is the default), making this a hard Kubernetes prerequisite; additionally, with swap enabled, memory limits become unreliable because the kernel may page container memory to disk rather than triggering the OOM killer.
- **Kernel Modules:** The `overlay` and `br_netfilter` kernel modules were loaded and persisted via `/etc/modules-load.d/k8s.conf` to support Container Network Interface (CNI) traffic across pod network namespaces.
- **Routing Parameters:** Sysctl parameters (`net.ipv4.ip_forward=1` and `net.bridge.bridge-nf-call-iptables=1`) were written to `/etc/sysctl.d/k8s.conf` to allow the Linux kernel to route packets between network namespaces and enforce iptables rules on bridged traffic.
- **Firewall Configuration:** The Uncomplicated Firewall (UFW) was disabled. Since the cluster runs on an isolated physical network (`10.0.1.0/24`) with no external exposure, the security benefit of UFW is minimal while the risk of silently dropping Kubernetes API component traffic due to misconfigured port rules is high.

Verification

The playbook includes five `VERIFY` tasks that must all pass before execution completes. It runs `swapon --show` and fails the play if the output is non-empty, ensuring no swap

device is active. It invokes `lsmod` twice, failing if `overlay` or `br_netfilter` is absent from the loaded module list. Finally, it queries `sysctl net.ipv4.ip_forward` and `sysctl net.bridge.bridge-nf-call-iptables` and fails if either does not return `= 1`. These assertions guarantee the host meets all Kubernetes prerequisites before the container runtime is installed in the next playbook.

B.3 03-containerd.yml

This playbook installs the container runtime and applies two targeted configuration patches. Containerd 2.2.3-1~ubuntu.24.04~noble is installed from the Docker APT repository. The playbook generates the full default `config.toml` and then applies the patches.

The first and most critical patch sets `SystemdCgroup = true`. Using `systemd` as the unified cgroup manager ensures the `kubelet` and the host OS agree on resource accounting paths. When the QoS controller issues a CPU limit change via the Kubernetes `Pods/resize` subresource, the `kubelet` receives the updated specification and writes the new quota to the pod's `cpu.max` entry in the cgroup v2 hierarchy; this propagation path requires the `kubelet` and the host cgroup manager to be aligned. Without this setting, the `kubelet` would use the `cgroupfs` driver while `systemd` manages its own cgroup tree, causing mismatches in resource accounting that prevent the in-place `resize` subresource from taking effect.

An `apt-mark hold` is placed on `containerd.io` immediately after installation to prevent unattended-upgrades or a manual `apt upgrade` from replacing the pinned version. Replacing the runtime version mid-experiment would alter container scheduling behavior without a corresponding playbook update, potentially invalidating the reproducibility guarantee that the IaC approach is designed to provide.

The playbook also verifies that the `sandbox (pause)` image is set to `registry.k8s.io/pause:3.10`, which is the correct version for Kubernetes v1.34 and the default shipped by containerd 2.2.x. No manual patch is required; the task confirms the default is intact.

Verification

After restarting the `containerd` daemon, the playbook runs four `VERIFY` tasks. It gathers `systemd` service facts and asserts the `containerd.service` state is `running`, failing with a descriptive error message if the service did not start. It then checks `containerd --version` output for the pinned version string `2.2.3`, catching cases where the APT hold failed or an incorrect package was installed. It greps `/etc/containerd/config.toml` for `SystemdCgroup = true` to confirm the cgroup driver patch was applied. Finally, it verifies the `sandbox` image is set to `registry.k8s.io/pause:3.10`; this confirms the default

is intact without requiring a manual patch.

B.4 04-kubernetes-components.yml & 04b-kubelet-bind.yml

Playbook `04-kubernetes-components.yml` installs the core Kubernetes binaries (`kubeadm`, `kubelet`, `kubect1`) strictly pinned to version `v1.34` via the APT package manager. An explicit `apt-mark hold` is placed on all four packages (`kubelet`, `kubeadm`, `kubect1`, `containerd.io`) to prevent accidental system upgrades from destabilising the environment across the multi-week experiment period.

Playbook `04b-kubelet-bind.yml` addresses a routing issue on multi-homed nodes. By default, the `kubelet` attempts to bind its `INTERNAL-IP` to the default gateway subnet (`192.168.1.0/24`). This causes all Kubernetes API communication and pod-to-pod traffic to flow over the 1 Gbps management interface rather than the dedicated cluster network. To prevent this, the playbook injects the `KUBELET_EXTRA_ARGS="--node-ip=10.0.1.x"` parameter into `/etc/default/kubelet`, the systemd environment file read by the `kubelet` service on startup. This ensures the `kubelet` advertises the correct `INTERNAL-IP` to the API server, routing all cluster traffic over the faster PCIe interface.

Verification

Playbook `04-kubernetes-components.yml` includes four built-in `VERIFY` tasks. Three `dpkg-query` calls inspect the installed package version of `kubeadm`, `kubelet`, and `kubect1` in turn, failing the play if the string `1.34` is absent from any result. A fourth task calls `apt-mark showhold` and fails if any of the four packages (`kubelet`, `kubeadm`, `kubect1`, or `containerd.io`) is not listed as held, confirming that the version pins from both this playbook and the preceding `containerd` playbook remain in effect.

Playbook `04b-kubelet-bind.yml` includes one `VERIFY` task: it reads `/etc/default/kubelet` and fails if the node's fast-network IP (`10.0.1.x`) is absent from the file, confirming the `kubelet` will advertise the correct `INTERNAL-IP` to the API server after its next restart.

B.5 05-cluster-bootstrap.yml

This playbook initializes the Kubernetes control plane and joins the worker nodes. The cluster is initialized using `kubeadm init`, specifying `10.0.1.11` for the API server advertisement address and `--pod-network-cidr=10.244.0.0/16` for the pod network. Calico `v3.29.3` was chosen for the CNI for its robust bare-metal routing support; this is the minimum release with official support for Kubernetes `1.33–1.34`.

Mitigating Split-Brain Routing: As described in Section 3.2.3, multi-homed nodes

require explicit interface binding to prevent Calico from building its overlay on the wrong subnet. A regex capture group injects `IP_AUTODETECTION_METHOD=cidr=10.0.1.0/24` into the Calico deployment manifest to force the correct PCIe interface. The Calico manifest is also patched to set `CALICO_IPV4POOL_CIDR=10.244.0.0/16` explicitly to match the kubeadm pod CIDR.

Worker nodes join the cluster via a token-based bootstrap mechanism: `kubeadm token create --print-join-command` is executed on the control plane and its output is stored as an Ansible fact. A second play then runs this command on each worker, guarded by an idempotency check on `/etc/kubernetes/kubelet.conf` so re-running the playbook on an already-joined node is safe.

Verification

The playbook includes four built-in `VERIFY` tasks spread across its three plays. After applying the Calico CNI, it polls `kubectl get nodes` with up to 18 retries at 10-second intervals, failing if any node remains `NotReady` after three minutes. In the same play it polls `kubectl get pods -n kube-system -l k8s-app=calico-node` with the same retry budget, failing if no Calico node pod reaches `Running`. It then calls `kubectl cluster-info` to confirm the API server endpoint is reachable. A final play waits up to two minutes for both `gpu-node-1` and `gpu-node-2` to appear in the cluster, confirming the worker join succeeded on both machines.

B.6 06-telemetry-stack.yml

This playbook deploys the `kube-prometheus-stack` Helm chart (version 69.3.0) into the `telemetry` namespace, providing Prometheus, Grafana, and the Prometheus Operator (including Node Exporter) for cluster-wide metrics collection. AlertManager is disabled as it is not required for the thesis. Scraping of `kubeControllerManager` and `kubeScheduler` is explicitly disabled; these components bind to `localhost` in a standard `kubeadm` deployment and are unreachable from the Prometheus pod, so scraping them would produce permanent connection-refused errors in the Prometheus target list.

Two Helm flags are set to enable cross-namespace `ServiceMonitor` and `PodMonitor` discovery:

- `serviceMonitorSelectorNilUsesHelmValues=false`
- `podMonitorSelectorNilUsesHelmValues=false`

Without these flags, the Helm chart sets the `serviceMonitorSelector` to match only monitors carrying the `release: telemetry` label; monitors deployed by other Helm releases (such as the DCGM exporter in `gpu-operator` or the NGINX ingress chart in

`ingress-nginx`) do not carry this label and would be silently ignored. Setting both flags to `false` leaves the selector empty (`{}`), instructing the Prometheus Operator to discover all `ServiceMonitor` and `PodMonitor` resources across all namespaces regardless of labels, which is required to pick up the DCGM exporter in `gpu-operator`, the NGINX ingress monitor in `ingress-nginx`, the Ray head monitor in `ray-system`, the QoS controller monitor in `qos-system`, and the ticket-app services monitor in `ticket-app`.

The cluster has no dynamic storage provisioner. Passing `storageSpec.emptyDir={}` via the Helm CLI causes a parse error; the CLI misinterprets `{}` as a JSON array rather than an empty object. Omitting `storageSpec` entirely causes the Prometheus Operator to default to ephemeral `emptyDir` storage, which is correct since Prometheus data is re-scraped from scratch after each experiment run.

Verification

The playbook includes five built-in **VERIFY** tasks. It first confirms the installed Helm binary version matches the pinned value `v3.17.0`. It then calls `kubectl wait` for the Prometheus pod and the Grafana pod in the `telemetry` namespace, failing immediately if either does not reach `Ready` within 120 seconds. A fourth task deploys an ephemeral `curlimages/curl` pod that queries the Prometheus HTTP API and fails if the response does not contain `"status":"success"`, confirming the API is reachable from within the cluster. A fifth task issues a URL-encoded PromQL query that counts the total number of idle CPU threads reported by Node Exporter; it fails if the HTTP request itself fails, confirming the scraping pipeline is active. The task fails if the HTTP request itself fails or the result set is empty; the expected idle-thread count is 48 (three nodes $\times$ 16 logical threads each), which an operator can inspect in the response body to confirm all three nodes are actively scraped, but this value is not asserted automatically.

B.7 07-gpu-operator.yml

This playbook deploys the NVIDIA GPU Operator to enable containerized GPU workloads without manually installing proprietary drivers or CUDA toolkits on the host OS. Manual host-level driver installation introduces severe configuration drift: driver versions may diverge between nodes, kernel updates may break driver compatibility, and CUDA toolkit versions must be manually coordinated with PyTorch requirements. The GPU Operator solves all three problems by managing the full driver lifecycle inside a privileged `DaemonSet` container.

Host Prerequisites and Driver Isolation: The open-source `nouveau` display driver was strictly blacklisted via `/etc/modprobe.d/` and the host `initramfs` was rebuilt to exclude it. The `nouveau` module conflicts with the proprietary NVIDIA driver; if `nouveau`

loads at boot time the NVIDIA driver installation fails. Linux kernel headers and build essentials were also installed on each worker node, allowing the GPU Operator's driver container to compile the proprietary NVIDIA kernel modules dynamically against the running host kernel while keeping the host OS pristine.

Operator Deployment and Runtime Integration: The operator was deployed via Helm. Two configuration values must be overridden from their defaults to match the custom containerd configuration established in playbook 03: the Helm values must point to `/etc/containerd/config.toml` and `/run/containerd/containerd.sock` as the actual containerd paths on the host. The DCGM exporter `ServiceMonitor` is enabled with the label `release: telemetry` (`dcgmExporter.serviceMonitor.additionalLabels.release=telemetry`), matching the cross-namespace discovery filter established in the telemetry stack playbook.

The Helm deployment uses `--wait --timeout 600s`, which blocks until every GPU Operator pod (including the driver DaemonSet) reaches `Running`. The GPU Operator's device-plugin DaemonSet registers the `nvidia.com/gpu` extended resource with the kubelet automatically once driver initialization completes; no explicit restart of `containerd` or `kubelet` is required.

Verification

The playbook includes three built-in `VERIFY` tasks executed after the Helm deployment completes. The first polls `kubectl get pods -n gpu-operator` with up to 30 retries at 20-second intervals (a total budget of 10 minutes), failing if any pod remains in `Pending` or `Init:` state; this accounts for the time required by the NVIDIA driver container to compile the proprietary kernel module against the running host kernel. The second task calls `kubectl get servicemonitor -n gpu-operator` and fails if no `ServiceMonitor` resource is present, confirming that the Prometheus Operator CRDs were available at Helm install time and the DCGM exporter monitor was created. The third task calls `kubectl get nodes -l nvidia.com/gpu.present=true` and fails if the output is empty, confirming that the GPU Operator device plugin detected and labeled both GPU worker nodes.

B.8 07b-ingress-nginx.yml

This playbook deploys the NGINX Ingress Controller (chart `ingress-nginx` v4.12.1, controller v1.12.x) into the `ingress-nginx` namespace via Helm. Chart version 4.12.1 is the first release series with official support for Kubernetes 1.33–1.34; earlier versions have known API compatibility issues with the newer admission framework.

Because the cluster runs on bare metal without a cloud provider integration, no ex-

ternal load balancer provisioner is available; a `type: LoadBalancer` service would remain in `Pending` state indefinitely. Instead, the controller is exposed via `NodePort`, binding HTTP traffic to port 30080 and HTTPS to port 30443 on every cluster node. Any cluster node IP serves as a valid entry point, making the ticket app accessible at `http://192.168.1.11:30080` regardless of pod scheduling.

Admission webhooks are disabled (`controller.admissionWebhooks.enabled=false`). The `ValidatingWebhookConfiguration` injected by a default NGINX deployment requires the Kubernetes API server to reach the webhook endpoint over an externally reachable IP. On bare metal, the webhook pod is assigned a cluster-internal IP with no corresponding external address, causing the API server to reject the TLS handshake and block all Ingress resource creation.

The Helm deployment enables controller metrics (`controller.metrics.enabled=true`) and creates a `ServiceMonitor` labeled `release: telemetry`, matching the cross-namespace discovery filter. This is required so Prometheus can scrape the NGINX metrics for operational dashboarding and for the cross-cutting integration check in playbook 07c; the QoS controller’s P95 latency signal is sourced from the application-level `http_request_duration_seconds_bucket` prom-client histogram emitted by the ticket-app microservices, not from the NGINX histogram (see Section 3.3.3).

Verification

The playbook includes three built-in `VERIFY` tasks. The first calls `kubectl wait pod -n ingress-nginx -l app.kubernetes.io/component=controller --for=condition=Ready --timeout=180s`, failing immediately if the controller pod does not become healthy within three minutes. The second queries `kubectl get servicemonitor -n ingress-nginx -l release=telemetry` and fails if the output is empty, confirming that the `ServiceMonitor` was created with the correct label for cross-namespace Prometheus discovery. The third task resolves the controller pod name at runtime and calls `wget -q0/dev/null http://localhost:10254/metrics` inside the pod; a non-zero exit code fails the play, confirming that the internal NGINX metrics endpoint is active and will respond to Prometheus scrape requests.

B.9 07c-kuberay-operator.yml

This playbook deploys the KubeRay operator (chart `kuberay-operator` v1.2.2) into the `ray-system` namespace and registers three Custom Resource Definitions: `RayCluster`, `RayJob`, and `RayService`. These CRDs enable Kubernetes-native lifecycle management of Ray distributed workloads.

Immediately after the operator is ready, the playbook applies a persistent `RayCluster`

manifest (`ray-training-cluster`) and waits up to 30 minutes for it to reach `Ready`. This cluster is deployed once and shared across all 15 experiment runs to preserve Ray’s pip runtime cache across conditions; the full rationale is in Section 3.2.4.

Each experiment playbook submits work to this persistent cluster via a `RayJob` resource that uses a `clusterSelector` targeting `ray-training-cluster` by label, rather than provisioning a new dedicated cluster per job. This decouples job lifecycle from cluster lifecycle: jobs can be deleted and resubmitted without evicting the pip cache.

The playbook applies a `PodMonitor` rather than a `ServiceMonitor` for the Ray head metrics port. A `PodMonitor` is used because the Ray head metrics port is not guaranteed to be backed by a stable named `Service` across all KubeRay cluster configurations, making direct pod scraping more robust. This playbook must therefore run after playbook 06: the `monitoring.coreos.com/v1` CRDs must already exist at `kubectl apply` time for the `PodMonitor` resource to be accepted by the API server.

Verification

The playbook includes two dedicated `VERIFY` tasks and four cross-cutting integration checks. The first dedicated task calls `kubectl get podmonitor -n ray-system ray-head-metrics` and fails if the `PodMonitor` is absent, confirming the Ray head metrics scrape target was applied successfully. The second calls `kubectl get crd -o name` and fails if any of `rayjobs.ray.io`, `rayclusters.ray.io`, or `rayservices.ray.io` is missing, confirming all three KubeRay CRDs were registered by the Helm install.

After a 3-minute pause to allow the Prometheus Operator to complete a scrape discovery cycle, four cross-cutting checks query Prometheus directly via ephemeral `curlimages/curl` pods. The first fails if `nginx.ingress.controller.config.hash` returns an empty result set, confirming the ingress `ServiceMonitor` from playbook 07b is actively scraped. The second queries `DCGM_FI_DEV_GPU_UTIL` but is marked `failed_when: false`; DCGM driver compilation can still be in progress at this point and an empty result is expected; the check is informational. The third confirms the Ray `PodMonitor` HTTP endpoint is reachable (result sets remain empty until a `RayCluster` begins emitting metrics); the fourth fails only if the HTTP request itself fails. A summary `debug` task prints the status of all three scraping targets to the Ansible output for operator review.

Appendix C

Workload A Reference

This appendix provides a complete reference for the Ticket Application (Workload A): the HTTP API exposed by each microservice, the Kafka event topology, the shared middleware implementations, and the Locust load test script.

C.1 HTTP API Reference

All backend services additionally expose `GET /health` (returns `200 {status: 'ok'}`) for Kubernetes readiness and liveness probes, and `GET /metrics` (returns 200 Prometheus text format) scraped by the `ServiceMonitor` in the `ticket-app` namespace. These infrastructure endpoints are omitted from the tables below.

The *Auth* column indicates whether `requireAuth` middleware is applied to the route. Routes marked “No” still run the global `currentUser` middleware but do not reject unauthenticated callers (see Section C.3.1).

C.1.1 Authentication Service (auth-srv)

The auth service manages user registration, login, and session lifecycle. Sessions are stored as JWTs in a `cookie-session` cookie; `secure: false` is required because the cluster terminates TLS at the physical network boundary, not at the NGINX ingress, so cookies travel over plain HTTP inside the cluster.

C.1.2 Tickets Service (tickets-srv)

The tickets service manages the ticket inventory. `GET /api/tickets` is bounded to 50 results to prevent full-collection scans as the MongoDB collection grows during a load test run. Tickets are locked (non-editable) while `ticket.orderId` is set (i.e., while a non-cancelled order exists for the ticket); `orderId` is cleared when the associated order is cancelled.

Method	Path	Auth	Request Body	Response
POST	/api/users/signup	No	email: string, password: string (4–20 chars)	201: user object; session JWT set in cookie
POST	/api/users/signin	No	email: string, password: string	200: user object; session JWT set in cookie
GET	/api/users/currentuser	No	—	200: { currentUser : { id , email } null }
POST	/api/users/signout	No	—	200: {}; session cookie cleared

Table C.1: Auth service HTTP endpoints.

Method	Path	Auth	Request Body	Response
POST	/api/tickets	Yes	title: string, price: number (>0)	201: { id , title , price , userId , version }
GET	/api/tickets	No	—	200: array of unreserved tickets (max 50)
GET	/api/tickets/:id	No	—	200: ticket object; 404 if not found
PUT	/api/tickets/:id	Yes	title: string, price: number (>0)	200: updated ticket; 400 if reserved; 401 if not owner; 404

Table C.2: Tickets service HTTP endpoints.

C.1.3 Orders Service (orders-srv)

The orders service manages the purchase state machine (`created` $\rightarrow$ `complete` | `cancelled`). `POST /api/orders` rejects tickets that are already reserved, synchronously computes a PBKDF2 receipt token (42 000 SHA-256 iterations) as deliberate CPU load, and publishes `order.created` to Kafka. Read endpoints call `.lean()` to return plain JavaScript objects and avoid Mongoose hydration overhead at high concurrency.

Method	Path	Auth	Request Body	Response
POST	/api/orders	Yes	ticketId: MongoDB ObjectId	201: {id, status, expiresAt, ticket, receiptToken}; 400 if ticket reserved; 404 if ticket not found
GET	/api/orders	Yes	—	200: array of caller's orders with populated ticket
GET	/api/orders/:orderId	Yes	—	200: order with populated ticket; 401 if not owner; 404
DELETE	/api/orders/:orderId	Yes	—	204: order status set to <code>cancelled</code> ; 401 if not owner; 404

Table C.3: Orders service HTTP endpoints.

PBKDF2 Receipt Token Implementation

The order creation route computes a PBKDF2 receipt token synchronously before returning the HTTP 201 response. The derivation input is the composite string `orderId:user` `Id:ticketId`, salted with the `JWT_KEY` environment variable, using 42 000 SHA-256 iterations and a 32-byte output length. The design rationale (making Workload A CPU-visible to the scheduler) is discussed in Section 3.3.

```

1  import crypto from 'crypto'
2  import express, { Request, Response } from 'express'
3  import {
4    BadRequestError,
5    NotFoundError,
6    requireAuth,
7    validateRequest,
8    kafkaWrapper,
9  } from '@ticket-app/common'
10 import { body } from 'express-validator'
11 import mongoose from 'mongoose'
12 import { Ticket } from '../models/ticket'

```

```

13 import { Order, OrderStatus } from '../models/order'
14 import { OrderCreatedPublisher } from '../events/publishers/order-created-publisher'
15
16 const router = express.Router()
17
18 const EXPIRATION_WINDOW_SECONDS = parseInt(process.env.EXPIRATION_WINDOW_SECONDS ||
  ↪ String(30 * 60), 10)
19
20 router.post(
21   '/api/orders',
22   requireAuth,
23   [
24     body('ticketId')
25       .not()
26       .isEmpty()
27       .custom((input: string) => mongoose.Types.ObjectId.isValid(input))
28       .withMessage('Ticket ID is required'),
29   ],
30   validateRequest,
31   async (req: Request, res: Response) => {
32     const { ticketId } = req.body
33
34     const ticket = await Ticket.findById(ticketId)
35     if (!ticket) {
36       throw new NotFoundError()
37     }
38
39     const isReserved = await ticket.isReserved()
40     if (isReserved) {
41       throw new BadRequestError('Ticket is already reserved.')
42     }
43
44     const expiration = new Date()
45     expiration.setSeconds(expiration.getSeconds() + EXPIRATION_WINDOW_SECONDS)
46
47     const order = Order.build({
48       userId: req.currentUser!.id,
49       status: OrderStatus.Created,
50       expiresAt: expiration,
51       ticket,
52     })
53     await order.save()
54
55     const receiptToken = crypto.pbkdf2Sync(
56       `${order.id}:${order.userId}:${ticket.id}`,
57       process.env.JWT_KEY || 'secret',
58       42000,
59       32,

```

```

60     'sha256',
61   ).toString('hex')
62
63   await new OrderCreatedPublisher(kafkaWrapper.producer).publish({
64     id: order.id,
65     version: order.version,
66     userId: order.userId,
67     status: order.status,
68     expiresAt: order.expiresAt.toISOString(),
69     ticket: {
70       id: ticket.id,
71       price: ticket.price,
72     },
73   })
74
75   res.status(201).send({ ...order.toJSON(), receiptToken })
76 },
77 )
78
79 export { router as createOrderRouter }

```

C.1.4 Payments Service (payments-srv)

The payments service simulates a payment gateway. Processing delay is mocked with a synchronous 500 ms pause per request; no real charge is performed. The generated charge ID follows the pattern `ch_mock_{random}`.

Method	Path	Auth	Request Body	Response
POST	/api/payments	Yes	token: string, orderId: string	201: {id: paymentId}; 400 if order cancelled; 401 if not owner; 404

Table C.4: Payments service HTTP endpoints.

C.1.5 Expiration Service (expiration-srv)

The expiration service has no HTTP endpoints. It is a pure background worker: it consumes `order.created` events, calculates the time remaining until `expiresAt`, and enqueues a Bull job (Redis-backed) to fire at that deadline. When the Bull job executes, it publishes an `expiration.complete` event, which causes the orders service to cancel the unpaid order. The expiration window (`EXPIRATION_WINDOW_SECONDS`) is set to 1800 s to prevent order expiration floods during the test window; see Section 3.3.2 for the rationale.

C.2 Kafka Event Reference

The six Kafka topics in Table C.5 form the complete asynchronous communication backbone of the Ticket Application. All inter-service state updates flow exclusively through these topics; no backend service calls another service’s HTTP API directly.

C.2.1 Event Topics

Topic	Published by	Consumed by	Purpose
<code>ticket.created</code>	tickets-srv	orders-srv	Replicate new ticket to orders DB
<code>ticket.updated</code>	tickets-srv	orders-srv	Sync ticket mutations (<code>orderId</code> , price, version)
<code>order.created</code>	orders-srv	tickets-srv, payments-srv, expiration-srv	Lock ticket; replicate order to payments DB; start expiry timer
<code>order.cancelled</code>	orders-srv	tickets-srv, payments-srv	Unlock ticket; mark order cancelled in payments DB
<code>expiration.complete</code>	expiration-srv	orders-srv	Cancel unpaid order after window expires
<code>payment.created</code>	payments-srv	orders-srv	Mark order status as <code>complete</code>

Table C.5: Kafka event topics.

C.2.2 Consumer Group IDs

Each listener class uses a unique `groupId` to ensure Kafka assigns all partitions of the subscribed topic to that listener. Sharing a `groupId` across listeners that subscribe to *different* topics triggers a Kafka consumer group rebalance; Kafka then assigns each listener only the partitions of the topic that the group was initially registered with, leaving other partitions unassigned and silently dropping all events on those topics.

C.3 Shared Middleware

All backend services share a common middleware library at `workload-a/common/src/middlewares/`, published as the private `@ticket-app/common` npm package and installed as a workspace dependency in each service.

C.3.1 Authentication Middleware

Two middleware functions implement session-based authentication. `currentUser` is registered globally in every service’s `app.ts`: it reads `req.session.jwt`, verifies the token

Service	Topic	Consumer Group ID
orders-srv	ticket.created	orders-ticket-created
orders-srv	ticket.updated	orders-ticket-updated
orders-srv	expiration.complete	orders-expiration-complete
orders-srv	payment.created	orders-payment-created
tickets-srv	order.created	tickets-order-created
tickets-srv	order.cancelled	tickets-order-cancelled
payments-srv	order.created	payments-order-created
payments-srv	order.cancelled	payments-order-cancelled
expiration-srv	order.created	expiration-service

Table C.6: Kafka consumer group IDs per listener.

with the `JWT_KEY` secret, and sets `req.currentUser = { id, email }` if valid. If the session is absent or the token is malformed, `currentUser` silently calls `next()` without setting `req.currentUser` and never throws. `requireAuth` is applied at the route level on all mutating endpoints; it throws `NotAuthorizedError` (HTTP 401) if `req.currentUser` was not populated by the preceding `currentUser` middleware.

```

1  import { Request, Response, NextFunction } from 'express';
2  import jwt from 'jsonwebtoken';
3
4  interface UserPayload {
5    id: string;
6    email: string;
7  }
8
9  declare global {
10    namespace Express {
11      interface Request {
12        currentUser?: UserPayload;
13      }
14    }
15  }
16
17  export const currentUser = (
18    req: Request,
19    res: Response,
20    next: NextFunction
21  ) => {
22    if (!req.session?.jwt) {
23      return next();
24    }
25
26    try {

```

```

27     const payload = jwt.verify(
28         req.session.jwt,
29         process.env.JWT_KEY!
30     ) as UserPayload;
31     req.currentUser = payload;
32 } catch (err) {}
33
34     next();
35 };

1  import { Request, Response, NextFunction } from 'express';
2  import { UnauthorizedError } from '../errors/unauthorized-error';
3
4  export const requireAuth = (
5      req: Request,
6      res: Response,
7      next: NextFunction
8  ) => {
9      if (!req.currentUser) {
10         throw new UnauthorizedError();
11     }
12
13     next();
14 };

```

C.3.2 Metrics Middleware

The metrics middleware instruments every HTTP request as a Prometheus histogram observation. The metric name is `http_request_duration_seconds` with three label dimensions: `method` (HTTP verb), `route` (Express route pattern), and `status_code`. Using `req.route.path` (the route template, e.g., `/api/orders/:orderId`) rather than `req.path` (the concrete URL) collapses all parameterized requests to a single time series per endpoint, keeping cardinality bounded as order IDs accumulate during a test run. The histogram buckets are [0.01, 0.05, 0.1, 0.2, 0.5, 1, 1.5, 2, 5] s, chosen to bracket the 200 ms SLO target at the 0.2 boundary and the payments mock delay at the 0.5 boundary. `collectDefaultMetrics()` additionally exports Node.js process metrics (heap usage, event loop lag, GC pressure) from every service. The QoS controller derives the P95 control signal from this histogram via PromQL.

```

1  import { Request, Response, NextFunction } from 'express'
2  import * as client from 'prom-client'
3
4  client.collectDefaultMetrics()
5
6  export const httpRequestDurationMicroseconds = new client.Histogram({
7      name: 'http_request_duration_seconds',

```

```

8   help: 'Duration of HTTP requests in seconds',
9   labelNames: ['method', 'route', 'status_code'],
10  buckets: [0.01, 0.05, 0.1, 0.2, 0.5, 1, 1.5, 2, 5],
11 })
12
13 export const metricsMiddleware = (req: Request, res: Response, next: NextFunction) =>
14   ↪ {
15     const end = httpRequestDurationMicroseconds.startTimer()
16
17     res.on('finish', () => {
18       const route = req.route ? req.baseUrl + req.route.path : req.path
19
20       end({
21         route,
22         status_code: res.statusCode,
23         method: req.method
24       })
25     })
26     next()
27   }
28
29 export const metricsRegister = client.register

```

C.4 Load Test Script

The load test is implemented in Locust (`scripts/load-test.py`). WaveShape subclasses `LoadTestShape` and overrides the CLI `--users` and `--spawn-rate` flags, encoding the five-phase wave profile directly in the script so it is independent of operator parameters. `TicketAppUser` runs two tasks at a 1:3 weight ratio: `browse_tickets` issues a single GET `/api/tickets`, while `full_purchase_flow` executes the three-step purchase sequence (ticket creation, order creation, payment submission). `gevent.sleep` is used for the inter-step coordination delays rather than `time.sleep` to avoid blocking the gevent greenlet pool that Locust uses internally for concurrent virtual users.

```

1  import os
2  import random
3  import string
4
5  import gevent
6  from locust import HttpUser, LoadTestShape, between, task
7
8
9  def _uid(n: int = 8) -> str:
10     return "".join(random.choices(string.ascii_lowercase + string.digits, k=n))

```

```

11
12
13 class WaveShape(LoadTestShape):
14     # Five-phase wave load profile (total ~15 minutes)
15     # (duration_s, users, spawn_rate)[]
16     stages = [
17         (180, 30, 5), # 0{3 min: low
18         (360, 400, 60), # 3{6 min: spike
19         (540, 200, 30), # 6{9 min: medium
20         (720, 400, 60), # 9{12 min: spike
21         (900, 30, 5), # 12{15 min: taper
22     ]
23
24     def tick(self):
25         t = self.get_run_time()
26         for duration, users, rate in self.stages:
27             if t < duration:
28                 return users, rate
29         return None
30
31
32 class TicketAppUser(HttpUser):
33     wait_time = between(0.5, 1.5)
34
35     def on_start(self) -> None:
36         email = f"locust_{_uid()}@test.com"
37         password = os.getenv("TEST_USER_PASSWORD", "Password123!")
38
39         resp = self.client.post(
40             "/api/users/signup",
41             json={"email": email, "password": password},
42             name="/api/users/signup",
43         )
44         if resp.status_code not in (200, 201):
45             resp = self.client.post(
46                 "/api/users/signin",
47                 json={"email": email, "password": password},
48                 name="/api/users/signin",
49             )
50         if resp.status_code not in (200, 201):
51             resp.failure("Auth failed | VU cannot proceed")
52             self._authenticated = False
53             return
54         self._authenticated = True
55
56     @task(1)
57     def browse_tickets(self) -> None:
58         if not getattr(self, "_authenticated", False):

```

```

59         return
60     self.client.get("/api/tickets", name="/api/tickets [GET]")
61
62     @task(3)
63     def full_purchase_flow(self) -> None:
64         if not getattr(self, "_authenticated", False):
65             return
66         title = f"Event {_uid(6)}"
67         price = round(random.uniform(10.0, 200.0), 2)
68         resp = self.client.post(
69             "/api/tickets",
70             json={"title": title, "price": price},
71             name="/api/tickets [POST]",
72         )
73         if resp.status_code != 201:
74             return
75
76         ticket_id = resp.json().get("id")
77         if not ticket_id:
78             return
79
80         gevent.sleep(0.5) # let ticket.created propagate to orders service
81
82         resp = self.client.post(
83             "/api/orders",
84             json={"ticketId": ticket_id},
85             name="/api/orders [POST]",
86         )
87         if resp.status_code != 201:
88             return
89
90         order_id = resp.json().get("id")
91         if not order_id:
92             return
93
94         gevent.sleep(0.3) # let order.created propagate to payments service
95
96         self.client.post(
97             "/api/payments",
98             json={"token": "tok_locust_test", "orderId": order_id},
99             name="/api/payments [POST]",
100         )

```

Appendix D

Workload B Reference

This appendix provides the complete source for the distributed AI training job (Workload B). The training script implements the full pipeline described in Section 3.4: the `TorchTrainer` entry point, the nine-stage CPU preprocessing pipeline with `ThreadPoolExecutor` parallelism, the prefetch queue daemon thread, the distributed ResNet-50 training loop with DDP AllReduce, and the `MetricsCollector` Ray actor that stores the final training summary from rank 0. All workers log a `training_complete` JSON event (including `total_time_sec`) independently; only rank 0 forwards its summary to the actor.

D.1 Training Script (ray-job.py)

```
1  import json
2  import logging
3  import os
4  import threading
5  import time
6  from concurrent.futures import ThreadPoolExecutor
7  from queue import Queue as _Queue
8
9  import ray
10 import torch
11 import torch.nn as nn
12 import torchvision.models as models
13 import torchvision.transforms.functional as TF
14 from ray import train
15 from ray.train import ScalingConfig
16 from ray.train.torch import TorchTrainer
17
18 logging.basicConfig(level=logging.INFO, format="%(message)s")
19 logger = logging.getLogger(__name__)
20
```

```

21
22 @ray.remote
23 class MetricsCollector:
24     def __init__(self):
25         self._data = None
26
27     def record(self, data):
28         self._data = data
29
30     def fetch(self):
31         return self._data
32
33
34 def _preprocess_chunk(chunk_size: int, seed_offset: int) -> torch.Tensor:
35     rng = torch.Generator()
36     rng.manual_seed(seed_offset)
37
38     # Simulate decoding a 768x768 image (2.25x more data than 512x512)
39     raw = torch.randint(0, 256, (chunk_size, 3, 768, 768),
40                        dtype=torch.uint8, generator=rng)
41     imgs = raw.float() / 255.0 # (N, 3, 768, 768), float32, CPU
42
43     # Random 640x640 crop (leaves 128px margin each side)
44     top = torch.randint(0, 129, (1,), generator=rng).item()
45     left = torch.randint(0, 129, (1,), generator=rng).item()
46     imgs = imgs[:, :, top:top + 640, left:left + 640]
47
48     # Random horizontal flip
49     if torch.rand(1, generator=rng).item() > 0.5:
50         imgs = torch.flip(imgs, dims=[3])
51
52     # Color jitter: brightness and contrast
53     brightness = 0.6 + torch.rand(1, generator=rng).item() * 0.8
54     contrast = 0.7 + torch.rand(1, generator=rng).item() * 0.6
55     imgs = torch.clamp(imgs * brightness, 0.0, 1.0)
56     imgs = torch.clamp((imgs - 0.5) * contrast + 0.5, 0.0, 1.0)
57
58     # Random rotation ±30° / expensive affine transform per image
59     angle = (torch.rand(1, generator=rng).item() * 60.0) - 30.0
60     imgs = torch.stack([TF.rotate(img, angle) for img in imgs])
61
62     # Gaussian blur kernel=21 / 2.6x more convolution work than kernel=13
63     imgs = torch.stack([TF.gaussian_blur(img, kernel_size=21) for img in imgs])
64
65     # Second Gaussian blur pass kernel=9 / additional CPU load
66     imgs = torch.stack([TF.gaussian_blur(img, kernel_size=9) for img in imgs])
67
68     # Center-crop to 224x224 for ResNet input / (640 - 224) / 2 = 208

```

```

69     imgs = imgs[:, :, 208:432, 208:432]
70
71     # ImageNet normalization
72     mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
73     std = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)
74     imgs = (imgs - mean) / std
75
76     return imgs # (N, 3, 224, 224), CPU
77
78
79 def cpu_preprocess_batch(batch_size: int, seed_offset: int,
80                          num_threads: int) -> torch.Tensor:
81     # each thread handles batch_size // num_threads images independently
82     chunk_size = batch_size // num_threads
83     seeds = [seed_offset + i * 1000 for i in range(num_threads)]
84
85     with ThreadPoolExecutor(max_workers=num_threads) as executor:
86         futures = [executor.submit(_preprocess_chunk, chunk_size, s)
87                    for s in seeds]
88         chunks = [f.result() for f in futures]
89
90     return torch.cat(chunks, dim=0) # (batch_size, 3, 224, 224)
91
92
93 def _prefetch_worker(q: _Queue, batch_size: int, num_threads: int, seed: int) -> None:
94     step = 0
95     while True:
96         batch = cpu_preprocess_batch(batch_size, seed + step * 7919, num_threads)
97         q.put(batch) # blocks when queue is full (backpressure)
98         step += 1
99
100
101 def train_loop_per_worker(config):
102     num_epochs = config["num_epochs"]
103     steps_per_epoch = config["steps_per_epoch"]
104     batch_size = config["batch_size"]
105     seed = config["seed"]
106     num_threads = config["num_preprocess_threads"]
107
108     torch.manual_seed(seed)
109     torch.cuda.manual_seed_all(seed)
110
111     world_rank = train.get_context().get_world_rank()
112     world_size = train.get_context().get_world_size()
113
114     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
115
116     logger.info(json.dumps({

```

```

117     "event": "worker_init",
118     "world_rank": world_rank,
119     "world_size": world_size,
120     "device": str(device),
121     "ddp_active": world_size > 1,
122     "num_preprocess_threads": num_threads,
123     "batch_size": batch_size,
124 )))
125
126 model = models.resnet50(weights=None)
127 model = model.to(device)
128 model = train.torch.prepare_model(model) # wraps in DDP when world_size > 1
129
130 criterion = nn.CrossEntropyLoss()
131 optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
132
133 total_images = 0
134 job_start = time.time()
135
136 _pq: _Queue = _Queue(maxsize=2)
137 threading.Thread(
138     target=_prefetch_worker,
139     args=(_pq, batch_size, num_threads, seed),
140     daemon=True,
141 ).start()
142
143 for epoch in range(num_epochs):
144     epoch_start = time.time()
145     epoch_loss = 0.0
146     epoch_images = 0
147
148     for step in range(steps_per_epoch):
149         step_start = time.time()
150
151         seed_offset = seed + epoch * steps_per_epoch + step
152         images = _pq.get().to(device)
153         labels = torch.randint(
154             0, 1000, (batch_size,),
155             generator=torch.Generator().manual_seed(seed_offset),
156         ).to(device)
157
158         optimizer.zero_grad()
159         outputs = model(images)
160         loss = criterion(outputs, labels)
161         loss.backward() # triggers DDP AllReduce gradient sync
162         optimizer.step()
163
164         step_time = time.time() - step_start

```

```

165         images_per_sec = batch_size / step_time
166         total_images += batch_size
167         epoch_images += batch_size
168         epoch_loss += loss.item()
169
170         train.report({
171             "epoch": epoch,
172             "step": step,
173             "images_per_sec": round(images_per_sec, 1),
174             "loss": round(loss.item(), 4),
175             "total_images": total_images,
176         })
177
178         epoch_time = time.time() - epoch_start
179         logger.info(json.dumps({
180             "event": "epoch_complete",
181             "epoch": epoch,
182             "world_rank": world_rank,
183             "epoch_time_sec": round(epoch_time, 2),
184             "avg_images_per_sec": round(epoch_images / epoch_time, 1),
185             "avg_loss": round(epoch_loss / steps_per_epoch, 4),
186         })))
187
188         total_time = time.time() - job_start
189         summary = {
190             "total_images": total_images,
191             "total_time_sec": round(total_time, 2),
192             "avg_throughput_images_per_sec": round(total_images / total_time, 1),
193         }
194         logger.info(json.dumps({"event": "training_complete", "world_rank": world_rank,
195                                ↪ **summary}))
196
197         if world_rank == 0:
198             collector = ray.get_actor("metrics_collector")
199             ray.get(collector.record.remote(summary))
200
201     def main():
202         num_epochs = int(os.environ.get("NUM_EPOCHS", "2"))
203         steps_per_epoch = int(os.environ.get("STEPS_PER_EPOCH", "200"))
204         batch_size = int(os.environ.get("BATCH_SIZE", "48"))
205         num_threads = int(os.environ.get("NUM_PREPROCESS_THREADS", "12"))
206         seed = int(os.environ.get("SEED", "42"))
207
208         # ensure batch_size is divisible by num_threads
209         batch_size = (batch_size // num_threads) * num_threads
210
211         ray.init(namespace="workload-b")

```

```
212
213     try:
214         ray.kill(ray.get_actor("metrics_collector"), no_restart=True)
215     except ValueError:
216         pass
217
218     collector = MetricsCollector.options(
219         name="metrics_collector", lifetime="detached"
220     ).remote()
221
222     trainer = TorchTrainer(
223         train_loop_per_worker=train_loop_per_worker,
224         train_loop_config={
225             "num_epochs": num_epochs,
226             "steps_per_epoch": steps_per_epoch,
227             "batch_size": batch_size,
228             "seed": seed,
229             "num_preprocess_threads": num_threads,
230         },
231         scaling_config=ScalingConfig(
232             num_workers=2,
233             use_gpu=True,
234             resources_per_worker={"GPU": 1, "CPU": 8},
235         ),
236     )
237
238     trainer.fit()
239
240     metrics = ray.get(collector.fetch.remote())
241
242     print(json.dumps({
243         "status": "COMPLETE",
244         "metrics": metrics,
245     }))
246
247
248 if __name__ == "__main__":
249     main()
```

Appendix E

Experiment Scripts

This appendix provides the complete source for the Prometheus metrics export script used by every run, followed by prose descriptions of the five Ansible playbooks for each experimental condition described in Section 3.6. Each playbook automates the full life-cycle of a single run: tearing down any previous state, deploying the relevant workloads, executing the 15-minute Locust WaveShape load test, waiting for Ray job completion, exporting metrics to CSV via the script below, and performing a final namespace cleanup. All playbooks accept a `run_id` extra variable (default: 1) that determines the output subdirectory under `results/`.

E.1 Prometheus Metrics Export Script

`scripts/export-metrics.py` is invoked after each run by the Ansible `export-and-fetch.yml` task. It queries Prometheus retroactively over the run's time window using `/api/v1/query\__range`, writing one CSV file per metric to the run directory. The `--condition` flag restricts the export to the subset of metrics relevant to that condition, avoiding empty files for metrics that are only meaningful in specific conditions (e.g. QoS diagnostics are only exported for the *dynamic* condition).

```
1  #!/usr/bin/env python3
2  """
3  Usage:
4  python3 scripts/export-metrics.py \
5      --start 2026-04-19T10:00:00Z \
6      --end   2026-04-19T10:30:00Z \
7      --out   results/baseline/
8  """
9
10 import argparse
11 import csv
12 import os
```

```

13 import sys
14 from datetime import datetime, timezone
15
16 import requests
17
18 PROMETHEUS_URL = os.getenv(
19     "PROMETHEUS_URL",
20     "http://telemetry-kube-prometheus-prometheus.telemetry:9090",
21 )
22
23 QUERIES: dict[str, str] = {
24     # Workload A latency (service-level prom-client histogram, excl. payments)
25     "p95_latency_ms": (
26         "histogram_quantile(0.95, sum(rate("
27
28         ↪ 'http_request_duration_seconds_bucket{namespace="ticket-app",service!="payments-srv"}'
29         "[60s])) by (le)) * 1000"
30     ),
31     "p99_latency_ms": (
32         "histogram_quantile(0.99, sum(rate("
33
34         ↪ 'http_request_duration_seconds_bucket{namespace="ticket-app",service!="payments-srv"}'
35         "[60s])) by (le)) * 1000"
36     ),
37     # Workload B (Ray) CPU
38     "ray_worker_cpu_usage_cores": (
39         "sum(rate(container_cpu_usage_seconds_total{"
40         'namespace="ray-system",container="ray-worker"'
41         "}[60s])) by (pod)"
42     ),
43     "ray_worker_cpu_limit_cores": (
44         "kube_pod_container_resource_limits{"
45         'namespace="ray-system",container="ray-worker",resource="cpu"'
46         "}"
47     ),
48     "ray_worker_cpu_usage_by_node": (
49         "sum by (node) (rate(container_cpu_usage_seconds_total{"
50         'namespace="ray-system",container="ray-worker"'
51         "}[60s]))"
52     ),
53     # Node CPU utilization (gpu-node-1: 10.0.1.12, gpu-node-2: 10.0.1.13)
54     "node_cpu_util_pct_gpu1": (
55         "(1 - avg(rate(node_cpu_seconds_total{"
56         'mode="idle",instance=~"10.0.1.12.*"'
57         "}[60s]))) * 100"
58     ),

```

```

59     "node_cpu_util_pct_gpu2": (
60         "(1 - avg(rate(node_cpu_seconds_total{"
61             'mode="idle",instance=~"10.0.1.13.*"'
62             "}[60s])))) * 100"
63     ),
64     "ticket_app_cpu_usage_by_node": (
65         "sum by (node) (rate(container_cpu_usage_seconds_total{"
66             'namespace="ticket-app"'
67             "}[15s]))"
68     ),
69
70     # GPU utilization (dcmg-exporter, provided by NVIDIA GPU Operator)
71     "gpu_util_pct": (
72         "avg by (Hostname, gpu) (DCGM_FI_DEV_GPU_UTIL)"
73     ),
74     "gpu_mem_used_mib": (
75         "avg by (Hostname, gpu) (DCGM_FI_DEV_FB_USED)"
76     ),
77     "gpu_mem_copy_util_pct": (
78         "avg by (Hostname, gpu) (DCGM_FI_DEV_MEM_COPY_UTIL)"
79     ),
80
81     # QoS controller diagnostics (only populated in qos-controller condition)
82     "qos_cpu_limit_cores": "qos_cpu_limit_cores",
83     "qos_error_ms": "qos_error_ms",
84     "qos_p95_observed_ms": "qos_p95_latency_ms",
85 }
86
87
88 CONDITION_QUERIES: dict[str, list[str]] = {
89     "no-contention-wla": [
90         "p95_latency_ms", "p99_latency_ms",
91         "node_cpu_util_pct_gpu1", "node_cpu_util_pct_gpu2",
92         "ticket_app_cpu_usage_by_node", "gpu_util_pct", "gpu_mem_used_mib",
93     ],
94     "no-contention-wlb": [
95         "ray_worker_cpu_usage_cores", "ray_worker_cpu_usage_by_node",
96         "node_cpu_util_pct_gpu1", "node_cpu_util_pct_gpu2",
97         "gpu_util_pct", "gpu_mem_used_mib", "gpu_mem_copy_util_pct",
98     ],
99     "no-limits": [
100         "p95_latency_ms", "p99_latency_ms",
101         "ray_worker_cpu_usage_cores", "ray_worker_cpu_usage_by_node",
102         "node_cpu_util_pct_gpu1", "node_cpu_util_pct_gpu2",
103         "ticket_app_cpu_usage_by_node", "gpu_util_pct", "gpu_mem_used_mib",
104         ↪ "gpu_mem_copy_util_pct",
105     ],
106     "static-limits": [

```

```

106         "p95_latency_ms", "p99_latency_ms",
107         "ray_worker_cpu_usage_cores", "ray_worker_cpu_limit_cores",
108         ↪ "ray_worker_cpu_usage_by_node",
109         "node_cpu_util_pct_gpu1", "node_cpu_util_pct_gpu2",
110         "ticket_app_cpu_usage_by_node", "gpu_util_pct", "gpu_mem_used_mib",
111         ↪ "gpu_mem_copy_util_pct",
112     ],
113     "dynamic": [
114         "p95_latency_ms", "p99_latency_ms",
115         "ray_worker_cpu_usage_cores", "ray_worker_cpu_limit_cores",
116         ↪ "ray_worker_cpu_usage_by_node",
117         "node_cpu_util_pct_gpu1", "node_cpu_util_pct_gpu2",
118         "ticket_app_cpu_usage_by_node", "gpu_util_pct", "gpu_mem_used_mib",
119         ↪ "gpu_mem_copy_util_pct",
120         "qos_cpu_limit_cores", "qos_error_ms", "qos_p95_observed_ms",
121     ],
122 }
123
124 def query_range(query: str, start: str, end: str, step: str) -> list[dict]:
125     resp = requests.get(
126         f"{PROMETHEUS_URL}/api/v1/query_range",
127         params={"query": query, "start": start, "end": end, "step": step},
128         timeout=30,
129     )
130     resp.raise_for_status()
131     data = resp.json()
132     if data["status"] != "success":
133         raise ValueError(f"Prometheus returned status={data['status']}: {data}")
134     return data["data"]["result"]
135
136 def ts_to_iso(ts: float) -> str:
137     return datetime.fromtimestamp(ts, tz=datetime.timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
138
139 def write_csv(name: str, results: list[dict], out_dir: str) -> None:
140     path = os.path.join(out_dir, f"{name}.csv")
141
142     if not results:
143         print(f" {name}: no data | empty CSV written")
144         with open(path, "w", newline="") as f:
145             csv.writer(f).writerow(["timestamp", "labels", "value"])
146         return
147
148     multi_series = len(results) > 1
149
150     with open(path, "w", newline="") as f:

```

```

150     writer = csv.writer(f)
151     if multi_series:
152         writer.writerow(["timestamp", "labels", "value"])
153         for series in results:
154             label_str = ",".join(
155                 f"{k}={v}" for k, v in series["metric"].items()
156             )
157             for ts, val in series["values"]:
158                 writer.writerow([ts_to_iso(float(ts)), label_str, val])
159         else:
160             writer.writerow(["timestamp", "value"])
161             for ts, val in results[0]["values"]:
162                 writer.writerow([ts_to_iso(float(ts)), val])
163
164     print(f"  {name}: {path}")
165
166
167 def main() -> None:
168     parser = argparse.ArgumentParser(
169         description="Export Prometheus metrics to CSV"
170     )
171     parser.add_argument("--start", required=True, help="Start time (ISO8601 or Unix)")
172     parser.add_argument("--end", required=True, help="End time (ISO8601 or Unix)")
173     parser.add_argument("--step", default="15s", help="Resolution (default: 15s)")
174     parser.add_argument("--out", default="results", help="Output directory")
175     parser.add_argument(
176         "--condition",
177         default=None,
178         choices=list(CONDITION_QUERIES.keys()),
179         help="Export only the queries relevant to this condition (default: all)",
180     )
181     args = parser.parse_args()
182
183     if args.condition:
184         active_queries = {k: QUERIES[k] for k in CONDITION_QUERIES[args.condition]}
185     else:
186         active_queries = QUERIES
187
188     os.makedirs(args.out, exist_ok=True)
189
190     print(f"Prometheus: {PROMETHEUS_URL}")
191     print(f"Range:      {args.start} → {args.end} (step={args.step})")
192     print(f"Condition:  {args.condition or 'all'}")
193     print(f"Output:     {args.out}/")
194     print()
195
196     errors = 0
197     for name, query in active_queries.items():

```

```

198         try:
199             results = query_range(query, args.start, args.end, args.step)
200             write_csv(name, results, args.out)
201         except Exception as exc:
202             print(f" ERROR {name}: {exc}", file=sys.stderr)
203             errors += 1
204
205     print()
206     if errors:
207         print(f"Done with {errors} error(s).", file=sys.stderr)
208         sys.exit(1)
209     else:
210         print("Done.")
211
212
213 if __name__ == "__main__":
214     main()

```

E.2 Workload A Isolation (08a)

08a-test-no-contention-wla.yml runs the Ticket App in isolation without any Ray co-tenant. Play 1 calls the shared `teardown.yml` task, which deletes the `ticket-app` and `qos-system` namespaces, any existing RayJob, and clears the Ray internal job registry via the dashboard API, ensuring no residual state carries over from a prior run. Play 2 applies all manifests from `kubernetes/workload-a/` and blocks until every `ticket-app` Deployment reports `Available` (300s timeout). Play 3 runs the Locust WaveShape test locally for 15 minutes against `http://192.168.1.11:30080`, writing `locust_stats.csv` and `locust_stats_history.csv` to `results/no-contention-wla/run-N/`. Play 4 invokes `export-and-fetch.yml` with `condition_name: no-contention-wla`, which calls `export-metrics.py` on `cpu-node-1` against the in-cluster Prometheus endpoint and fetches `p95_latency_ms`, `p99_latency_ms`, `node_cpu_util_pct_gpu{1,2}`, `ticket-app-cpu_usage_by_node`, `gpu_util_pct`, and `gpu_mem_used_mib` to the local results directory before deleting the `ticket-app` namespace. The persistent RayCluster (deployed once by playbook 07c) is left untouched throughout.

E.3 Workload B Isolation (08b)

08b-test-no-contention-wlb.yml runs the Ray distributed training job in isolation without Workload A. After the shared `teardown`, Ray worker CPU limits are reset to 16 cores via in-place `resize`, removing any cap set by a prior `static-limits` or `qos-controller` run. `ray-job.py` is copied from the repository to the Ray head pod via `kubectl cp`, and the RayJob is submitted. The playbook records the start timestamp, then waits

for **Running** deployment status (10 min timeout). Unlike co-location conditions that interleave Locust, this playbook immediately waits for **RayJob Complete** (45 min timeout; `failed_when: false` to capture partial results on timeout). The makespan record (`rayjob_start/rayjob_end`) is written to `makespan.txt`. Metrics cover Ray worker CPU usage by pod and node, per-node utilization, and all three GPU metrics; web-application latency metrics are omitted since Workload A is not running. The 45 min ceiling bounds the run even if DDP synchronization hangs; the job typically completes in under 15 min under isolation.

E.4 No-Limits Co-location (09)

`09-test-no-limits.yml` is the worst-case co-location baseline: both workloads run simultaneously with Ray workers permitted 16 CPU cores. After teardown and Workload A deployment (with an **Available** wait), workers are reset to 16 cores via in-place resize. `ray-job.py` is copied to the head pod and the **RayJob** submitted; the start timestamp is recorded and the playbook waits for **Running** state. Locust then runs its 15-minute WaveShape test. A separate post-Locust play waits up to 30 min for **RayJob Complete** (`failed_when: false`); in the no-limits condition the job typically finishes before Locust ends, so `kubectl wait` returns immediately and `rayjob_end` reflects the time shortly after Locust exits; producing the makespan inflation artefact documented in Section 4.2. Prometheus metrics are exported over the Locust time window (`test_start/test_end` set by the `locust-test` task). The teardown play deletes the **RayJob** and `ticket-app` namespace.

E.5 Static-Limits Co-location (10)

`10-test-static-limits.yml` is the fixed-cap reference condition. It follows the same structure as playbook 09 but throttles each Ray worker to exactly 6 CPU cores before submitting the job. The resize patch sets both `limits.cpu` and `requests.cpu` to "6", placing each worker pod in the Kubernetes Guaranteed QoS class and preventing the kernel from allocating more than 6 cores of CFS bandwidth to the Ray preprocessing threads. The 6-core ceiling is chosen to be measurably below the workers' natural demand of 8 cores, imposing a visible throughput penalty while still permitting the job to complete within the 15-minute test window. `ray_worker_cpu_limit_cores` is additionally exported as a sanity check confirming the 6-core cap held throughout the run.

E.6 QoS-Controller Co-location (11)

`11-test-qos.yml` is the experimental condition under evaluation. After teardown and Workload A deployment, Ray workers are set to 8 cores; above the natural floor of 2 cores observed under full throttle but below full capacity, giving the PI controller room to both reduce and restore CPU allocation from a known starting point. `ray-job.py` is copied to the head pod and the RayJob submitted. The QoS controller is then deployed by applying all manifests under `kubernetes/qos-controller/`, and the playbook blocks until the controller pod reports `Ready` (120 s timeout) before proceeding to Locust. This ordering ensures the controller is active before load generation begins, so the first Phase-2 spike is not unguarded. Locust runs its 15-minute WaveShape test. The post-Locust play exports the same metrics as playbook 10 with the addition of three QoS diagnostic metrics: `qos_cpu_limit_cores`, `qos_error_ms`, and `qos_p95_observed_ms`. The teardown play deletes the RayJob and both the `qos-system` and `ticket-app` namespaces.

Appendix F

Supplementary Results

This appendix provides the complete numerical record for all experimental runs, the full cross-condition comparison table with all columns read directly from the results CSV, and the per-run time-series plots for runs 2 and 3 of each condition (run 1 of each condition is shown as the representative in Chapter 4).

F.1 Cross-Condition Summary Table

Table F.1 is generated directly from `results/tables/comparison.csv` and shows all exported metrics for each condition.

F.2 Per-Run Measurements

Table F.2 lists every individual run across all five conditions, loaded directly from `results/tables/summary.csv`. Dashes indicate metrics not applicable to that condition.

F.3 Per-Run Time-Series Plots

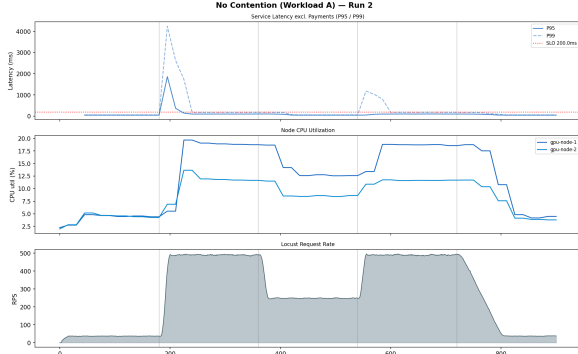
Figures F.1–F.5 show runs 2 and 3 for each condition in the same panel layout as the representative run 1 figures in Chapter 4: the no-contention Workload A baseline (Figure F.1), the no-contention Workload B baseline (Figure F.2), co-location without CPU limits (Figure F.3), co-location with static CPU limits (Figure F.4), and co-location with the dynamic QoS controller (Figure F.5).

Condition	P99 (ms)	SLO (%)	Viol. (s)	Makespan (s)	CPU (%)	GPU (%)
No Contention (Workload A)	273.11 $\pm$ 26.60	89.90 $\pm$ 0.26	90.00 $\pm$ 0.00	—	10.51 $\pm$ 0.07	0.00 $\pm$ 0.00
No Contention (Workload B)	—	—	0.00 $\pm$ 0.00	888.00 $\pm$ 1.73	65.78 $\pm$ 0.32	80.25 $\pm$ 0.98
No Limits	905.82 $\pm$ 69.20	12.03 $\pm$ 0.92	805.00 $\pm$ 8.66	1040.33 $\pm$ 0.58	66.52 $\pm$ 2.11	82.44 $\pm$ 3.46
Static Limits	385.18 $\pm$ 33.05	49.10 $\pm$ 3.45	455.00 $\pm$ 17.32	1890.33 $\pm$ 4.04	41.62 $\pm$ 0.34	80.32 $\pm$ 1.41
QoS Controller	468.05 $\pm$ 87.05	52.47 $\pm$ 1.65	430.00 $\pm$ 17.32	1351.33 $\pm$ 89.25	50.63 $\pm$ 4.30	81.89 $\pm$ 1.35

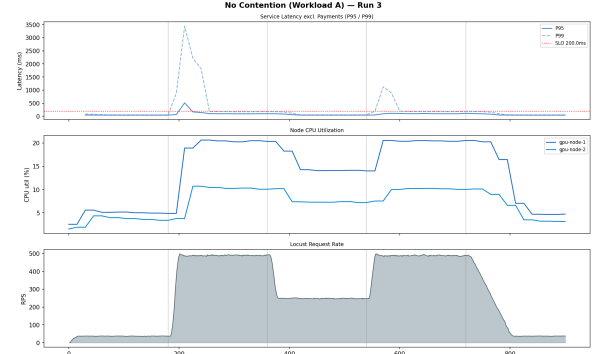
Table F.1: Cross-condition summary loaded from `comparison.csv` (mean $\pm$ std, $n = 3$ runs). No-limits makespan is inflated (see Section 4.2).

Condition	Run	Makespan (s)	P99 (ms)	SLO (%)	Viol. (s)	CPU (%)	GPU (%)
no-contention-wla	1		245.83	90.2	90.0	10.59	0.0
no-contention-wla	2		298.98	89.7	90.0	10.45	0.0
no-contention-wla	3		274.51	89.8	90.0	10.48	0.0
no-contention-wlb	1	890.0			0.0	65.67	79.33
no-contention-wlb	2	887.0			0.0	65.54	81.28
no-contention-wlb	3	887.0			0.0	66.14	80.15
no-limits	1	1041.0	980.86	11.5	810.0	68.88	81.26
no-limits	2	1040.0	844.52	13.1	795.0	65.88	79.72
no-limits	3	1040.0	892.07	11.5	810.0	64.81	86.34
static-limits	1	1891.0	349.59	52.5	435.0	41.64	78.72
static-limits	2	1886.0	391.06	45.6	465.0	41.94	80.84
static-limits	3	1894.0	414.9	49.2	465.0	41.27	81.39
dynamic	1	1449.0	564.71	50.8	450.0	45.83	80.56
dynamic	2	1331.0	443.6	52.5	420.0	51.95	83.26
dynamic	3	1274.0	395.85	54.1	420.0	54.12	81.85

Table F.2: Per-run measurements for all five experimental conditions ($n = 15$ runs total).

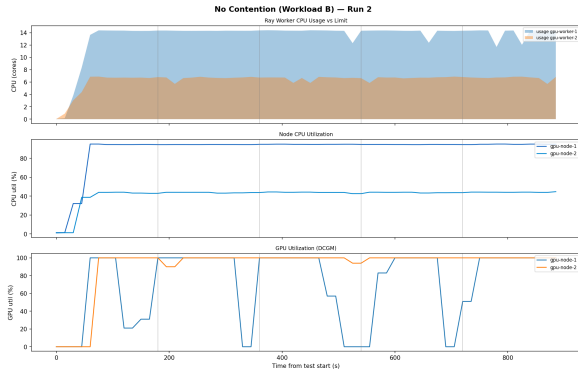


(a) Run 2

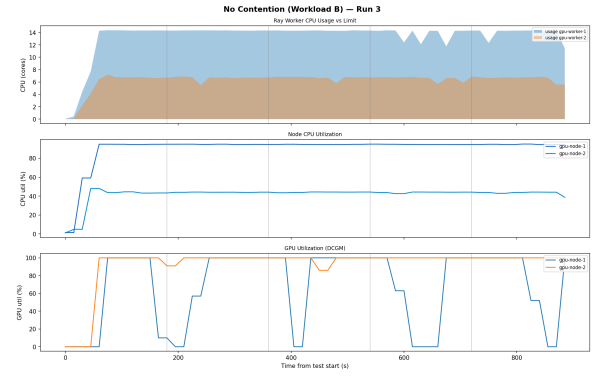


(b) Run 3

Figure F.1: No-contention Workload A (runs 2 and 3). Panels as in Figure 4.1.

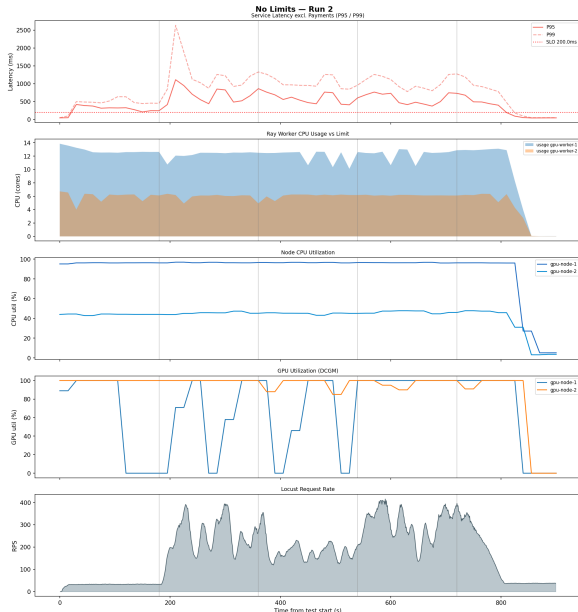


(a) Run 2

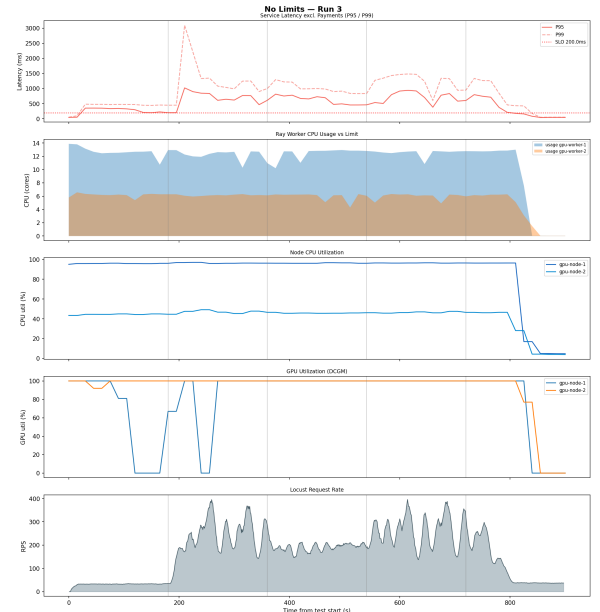


(b) Run 3

Figure F.2: No-contention Workload B (runs 2 and 3). Panels as in Figure 4.2.

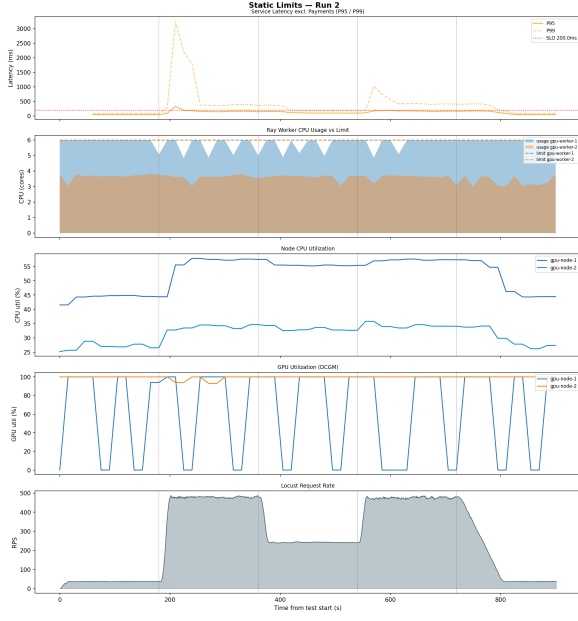


(a) Run 2

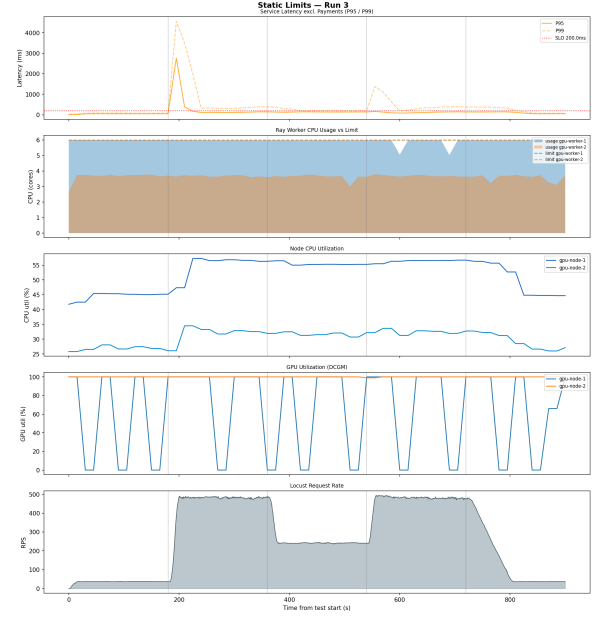


(b) Run 3

Figure F.3: Co-location without CPU limits (runs 2 and 3). Panels as in Figure 4.3.

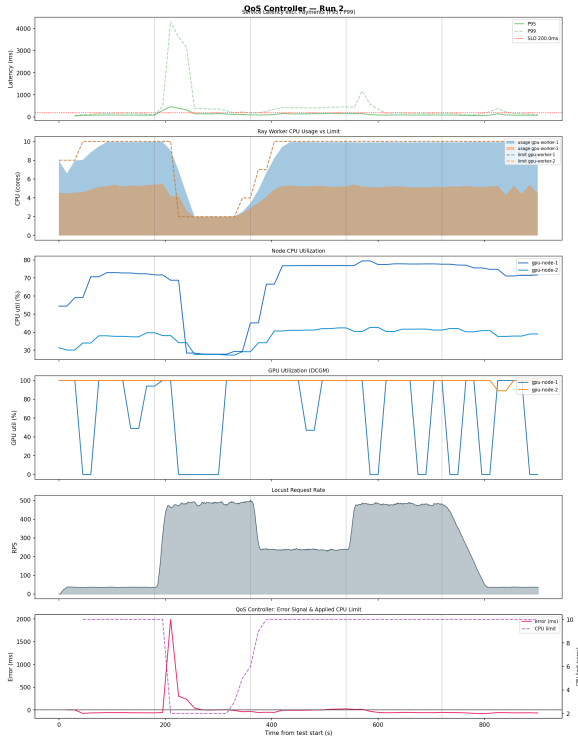


(a) Run 2

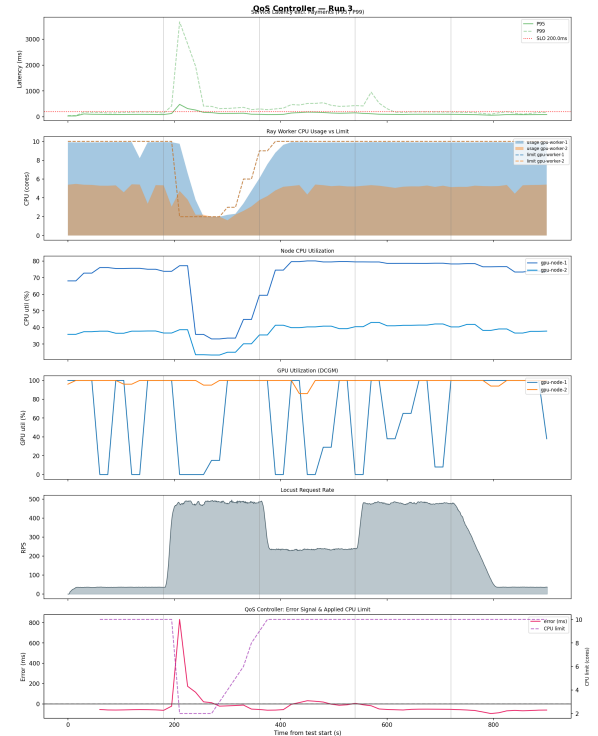


(b) Run 3

Figure F.4: Co-location with static CPU limits (runs 2 and 3). Panels as in Figure 4.4.



(a) Run 2



(b) Run 3

Figure F.5: Co-location with dynamic QoS controller (runs 2 and 3). Panels as in Figure 4.5.